

Compliance-Based Cloud Security Proof of Concept: AWS & Kubernetes Security Architecture

Abiyu Salam

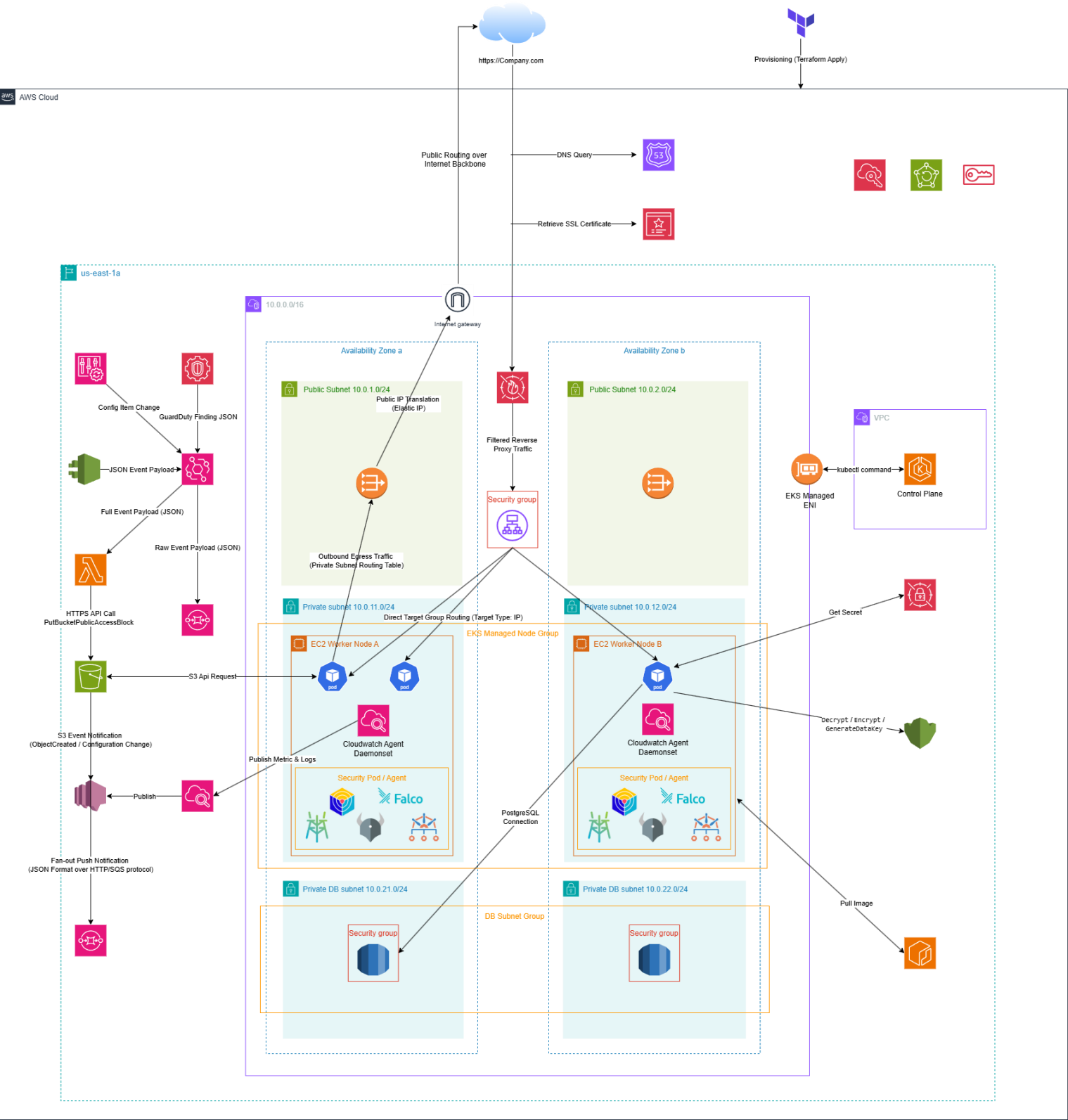


Table of Content

Phase 1 — Networking Foundation.....	5
Phase 2 — Security	11
Phase 3 — Compute.....	18
Phase 4 — Traffic Management & Edge Security	33
Phase 5 — Database.....	45
Phase 6 — Storage & Object Security.....	52
Phase 7 — Domain & SSL.....	67
Phase 8 — Secrets Management & Encryption.....	81
Phase 9 — Monitoring & Observability.....	101
Phase 10 — Audit, Compliance & Threat Detection	139
Phase 11 — Automated Security Response	151

Step 1. Onboarding

1.1 Pindah User

```
su - floci
```

1.2 Cek apakah service floci core sudah jalan

```
floci status
```

1.3 Jika belum jalan, nyalakan service-nya

```
floci start
```

1.4 Start GUI

```
cd ~/floci-ui
```

```
pnpm dev
```

1.5 Buka di web <http://localhost:4500>

1.6 Env Var AWS

```
export AWS_ACCESS_KEY_ID=test
```

```
export AWS_SECRET_ACCESS_KEY=test
```

```
export AWS_DEFAULT_REGION=us-east-1
```

```
ENDPOINT="http://localhost:4566"
```

```
export AWS_ACCOUNT_ID="000000000000"
```

Phase 1 — Networking Foundation

Step 2. Membuat VPC dengan CIDR 10.0.0.0/16

```
aws --endpoint-url=http://localhost:4566 ec2 create-vpc \  
    --cidr-block 10.0.0.0/16 \  
    --tag-specifications  
'ResourceType=vpc,Tags=[{Key=Name,Value=enterprise-vpc}]'
```

2.1 Validasi

```
aws --endpoint-url=http://localhost:4566 ec2 describe-vpcs
```

Step 3. Create Subnet

3.1 Ambil VPC ID

```
VPC_ID=$(aws ec2 describe-vpcs \  
    --endpoint-url=http://localhost:4566 \  
    --filters "Name=tag:Name,Values=enterprise-vpc" \  
    --query "Vpcs[0].VpcId" \  
    --output text)
```

3.2 Public Subnets

```
PUBLIC_SUBNET_ID_A=$(aws --endpoint-url=http://localhost:4566 ec2  
create-subnet \  
    --vpc-id $VPC_ID --cidr-block 10.0.1.0/24 --availability-zone us-  
east-1a \  
    --tag-specifications  
'ResourceType=subnet,Tags=[{Key=Name,Value=public-subnet-a}]' \  
    --query 'Subnet.SubnetId' --output text)  
  
PUBLIC_SUBNET_ID_B=$(aws --endpoint-url=http://localhost:4566 ec2  
create-subnet \  
    --vpc-id $VPC_ID --cidr-block 10.0.2.0/24 --availability-zone us-  
east-1b \  
    --tag-specifications  
'ResourceType=subnet,Tags=[{Key=Name,Value=public-subnet-b}]' \  
    --query 'Subnet.SubnetId' --output text)
```

3.3 Private App Subnets

```
PRIVATE_APP_SUBNET_ID_A=$(aws --endpoint-url=http://localhost:4566 ec2  
create-subnet \  
    --vpc-id $VPC_ID --cidr-block 10.0.11.0/24 --availability-zone us-  
east-1a \  
    --tag-specifications  
'ResourceType=subnet,Tags=[{Key=Name,Value=private-app-subnet-a}]' \  
    --query 'Subnet.SubnetId' --output text)
```

```

    --tag-specifications
'ResourceType=subnet,Tags=[{Key=Name,Value=private-app-subnet-a}]' \

    --query 'Subnet.SubnetId' --output text)

PRIVATE_APP_SUBNET_ID_B=$(aws --endpoint-url=http://localhost:4566 ec2
create-subnet \

    --vpc-id $VPC_ID --cidr-block 10.0.12.0/24 --availability-zone us-
east-1b \

    --tag-specifications
'ResourceType=subnet,Tags=[{Key=Name,Value=private-app-subnet-b}]' \

    --query 'Subnet.SubnetId' --output text)

```

3.4 Private DB Subnets

```

PRIVATE_DB_SUBNET_ID_A=$(aws --endpoint-url=http://localhost:4566 ec2
create-subnet \

    --vpc-id $VPC_ID --cidr-block 10.0.21.0/24 --availability-zone us-
east-1a \

    --tag-specifications
'ResourceType=subnet,Tags=[{Key=Name,Value=private-db-subnet-a}]' \

    --query 'Subnet.SubnetId' --output text)

PRIVATE_DB_SUBNET_ID_B=$(aws --endpoint-url=http://localhost:4566 ec2
create-subnet \

    --vpc-id $VPC_ID --cidr-block 10.0.22.0/24 --availability-zone us-
east-1b \

    --tag-specifications
'ResourceType=subnet,Tags=[{Key=Name,Value=private-db-subnet-b}]' \

    --query 'Subnet.SubnetId' --output text)

```

3.5 Validasi

cara lama → <VPC_ID> lihat di

Global (seluruh vpc masuk)

aws --endpoint-url=http://localhost:4566 ec2 describe-vpcs → describe
seluruh vpc yg ada di arsitektur aws kita

ATAU

```
aws --endpoint-url=http://localhost:4566 ec2 describe-vpc --query
"Vpcs[*].VpcId" --output text → coba dong show vpcID dari seluruh VPC
```

Local (Khusus)

```
aws ec2 describe-vpcs \

    --endpoint-url=http://localhost:4566 \

    --filters "Name=tag:Name,Values=enterprise-vpc"
```

ATAU

```
aws ec2 describe-vpcs \  
    --endpoint-url=http://localhost:4566 \  
    --filters "Name=tag:Name,Values=enterprise-vpc" \  
    --query "Vpcs[*].VpcId" \  
    --output text
```

Step 4. Create Internet Gateway

4.1 IGW_ID biar cepet

```
IGW_ID=$(aws ec2 create-internet-gateway \  
    --endpoint-url=http://localhost:4566 \  
    --tag-specifications 'ResourceType=internet-  
gateway,Tags=[{Key=Name,Value=enterprise-igw}]' \  
    --query 'InternetGateway.InternetGatewayId' \  
    --output text)
```

4.2 Attach Internet Gateway ke VPC

```
aws --endpoint-url=http://localhost:4566 ec2 attach-internet-gateway \  
    --internet-gateway-id $IGW_ID \  
    --vpc-id $VPC_ID
```

4.3 Validasi

```
aws --endpoint-url=http://localhost:4566 ec2 describe-internet-  
gateways
```

Step 5. Create NAT Gateway

--- 5.1 NAT GATEWAY AZ-A ---

5.1.2 Allocate Elastic IP A

```
ALLOCATION_ID_A=$(aws --endpoint-url=http://localhost:4566 ec2  
allocate-address \  
    --domain vpc \  
    --query 'AllocationId' --output text)
```

5.1.3 Create NAT Gateway A (Pake Subnet AZ-A dari Step 3)

```
NAT_GATEWAY_ID_A=$(aws --endpoint-url=http://localhost:4566 ec2  
create-nat-gateway \  
    --subnet-id $PUBLIC_SUBNET_ID_A \  
    --allocation-id $ALLOCATION_ID_A \
```

```

    --tag-specifications
'ResourceType=natgateway,Tags=[{Key=Name,Value=enterprise-nat-a}]' \

    --query 'NatGateway.NatGatewayId' --output text)

# --- 5.2 NAT GATEWAY AZ-B ---

# 5.2.1 Allocate Elastic IP B
ALLOCATION_ID_B=$(aws --endpoint-url=http://localhost:4566 ec2
allocate-address \

    --domain vpc \

    --query 'AllocationId' --output text)

# 5.2.3 Create NAT Gateway B (Pake Subnet AZ-B dari Step 3)
NAT_GATEWAY_ID_B=$(aws --endpoint-url=http://localhost:4566 ec2
create-nat-gateway \

    --subnet-id $PUBLIC_SUBNET_ID_B \

    --allocation-id $ALLOCATION_ID_B \

    --tag-specifications
'ResourceType=natgateway,Tags=[{Key=Name,Value=enterprise-nat-b}]' \

    --query 'NatGateway.NatGatewayId' --output text)

# 5.3 Validasi
aws --endpoint-url=http://localhost:4566 ec2 describe-nat-gateways

```

Step 6. Create Route Tables

```

# 6.1.1 PUBLIC ROUTE TABLE (1 RT untuk AZ-A & AZ-B)
PUBLIC_RT_ID=$(aws --endpoint-url=http://localhost:4566 ec2 create-
route-table \

    --vpc-id $VPC_ID \

    --tag-specifications 'ResourceType=route-
table,Tags=[{Key=Name,Value=public-rt}]' \

    --query 'RouteTable.RouteTableId' --output text)

# 6.1.2 Route 0.0.0.0/0 ke Internet Gateway
aws --endpoint-url=http://localhost:4566 ec2 create-route \

    --route-table-id $PUBLIC_RT_ID \

    --destination-cidr-block 0.0.0.0/0 \

    --gateway-id $IGW_ID

# 6.1.3 Associate ke Public Subnet AZ-A & AZ-B
aws --endpoint-url=http://localhost:4566 ec2 associate-route-table \

    --subnet-id $PUBLIC_SUBNET_ID_A \

```

```
--route-table-id $PUBLIC_RT_ID
aws --endpoint-url=http://localhost:4566 ec2 associate-route-table \
--subnet-id $PUBLIC_SUBNET_ID_B \
--route-table-id $PUBLIC_RT_ID
```

6.2.1 PRIVATE ROUTE TABLE AZ-A (Mengarahkan ke NAT GW A)

```
PRIVATE_RT_ID_A=$(aws --endpoint-url=http://localhost:4566 ec2 create-
route-table \
--vpc-id $VPC_ID \
--tag-specifications 'ResourceType=route-
table,Tags=[{Key=Name,Value=private-rt-a}]' \
--query 'RouteTable.RouteTableId' --output text)
```

6.2.2 Route 0.0.0.0/0 ke NAT Gateway AZ-A

```
aws --endpoint-url=http://localhost:4566 ec2 create-route \
--route-table-id $PRIVATE_RT_ID_A \
--destination-cidr-block 0.0.0.0/0 \
--nat-gateway-id $NAT_GATEWAY_ID_A
```

6.2.3 Associate ke Private App & DB Subnet AZ-A

```
aws --endpoint-url=http://localhost:4566 ec2 associate-route-table \
--subnet-id $PRIVATE_APP_SUBNET_ID_A \
--route-table-id $PRIVATE_RT_ID_A
aws --endpoint-url=http://localhost:4566 ec2 associate-route-table \
--subnet-id $PRIVATE_DB_SUBNET_ID_A \
--route-table-id $PRIVATE_RT_ID_A
```

6.3.1 PRIVATE ROUTE TABLE AZ-B (Mengarahkan ke NAT GW B)

```
PRIVATE_RT_ID_B=$(aws --endpoint-url=http://localhost:4566 ec2 create-
route-table \
--vpc-id $VPC_ID \
--tag-specifications 'ResourceType=route-
table,Tags=[{Key=Name,Value=private-rt-b}]' \
--query 'RouteTable.RouteTableId' --output text)
```

6.3.2 Route 0.0.0.0/0 ke NAT Gateway AZ-B

```
aws --endpoint-url=http://localhost:4566 ec2 create-route \
--route-table-id $PRIVATE_RT_ID_B \
--destination-cidr-block 0.0.0.0/0 \
```

```
--nat-gateway-id $NAT_GATEWAY_ID_B
```

6.3.3 Associate ke Private App & DB Subnet AZ-B

```
aws --endpoint-url=http://localhost:4566 ec2 associate-route-table \
```

```
--subnet-id $PRIVATE_APP_SUBNET_ID_B \
```

```
--route-table-id $PRIVATE_RT_ID_B
```

```
aws --endpoint-url=http://localhost:4566 ec2 associate-route-table \
```

```
--subnet-id $PRIVATE_DB_SUBNET_ID_B \
```

```
--route-table-id $PRIVATE_RT_ID_B
```

6.4 Validasi

6.4.1 Cek Route Table

```
aws --endpoint-url=http://localhost:4566 ec2 describe-route-tables
```

6.4.2 Cek Association Route Table

```
aws --endpoint-url=http://localhost:4566 ec2 describe-route-tables --  
query "RouteTables[*].Associations"
```

6.4.3 Cek Route

```
aws --endpoint-url=http://localhost:4566 ec2 describe-route-tables --  
query "RouteTables[*].Routes"
```

Phase 2 — Security

Step 7. Security Group

7.1.1 SG untuk Application Load Balancer (ALB)

```
SG_ALB_ID=$(aws --endpoint-url=$ENDPOINT ec2 create-security-group \
    --group-name sg-eks-alb \
    --description "Security Group for EKS Ingress ALB" \
    --vpc-id "$VPC_ID" \
    --query 'GroupId' --output text)
```

7.1.2 Izinkan HTTPS dari Internet ke ALB

```
aws --endpoint-url=$ENDPOINT ec2 authorize-security-group-ingress \
    --group-id "$SG_ALB_ID" --protocol tcp --port 443 --cidr 0.0.0.0/0
```

7.2.1 SG untuk EKS Worker Nodes (EC2)

```
SG_EKS_NODE_ID=$(aws --endpoint-url=$ENDPOINT ec2 create-security-
group \
    --group-name sg-eks-nodes \
    --description "Security Group for EKS Worker Nodes" \
    --vpc-id "$VPC_ID" \
    --query 'GroupId' --output text)
```

7.2.2 Izinkan traffic dari ALB ke EKS Worker Nodes (NodePort / Traffic Ingress)

```
aws --endpoint-url=$ENDPOINT ec2 authorize-security-group-ingress \
    --group-id "$SG_EKS_NODE_ID" --protocol tcp --port 30000-32767 --
source-group "$SG_ALB_ID"
```

7.2.3 Izinkan komunikasi antar Worker Node di dalam Cluster (Kubelet / Pod to Pod)

```
aws --endpoint-url=$ENDPOINT ec2 authorize-security-group-ingress \
    --group-id "$SG_EKS_NODE_ID" --protocol -1 --source-group
"$SG_EKS_NODE_ID"
```

7.3.1 SG untuk Database (RDS)

```
SG_RDS_ID=$(aws --endpoint-url=$ENDPOINT ec2 create-security-group \
    --group-name sg-eks-rds \
    --description "Security Group for RDS Database" \
    --vpc-id "$VPC_ID" \
    --query 'GroupId' --output text)
```

7.3.2 Izinkan PostgreSQL (7001) HANYA dari EKS Worker Nodes

```
aws --endpoint-url=$ENDPOINT ec2 authorize-security-group-ingress \
    --group-id "$SG_RDS_ID" --protocol tcp --port 7001 --source-group
"$SG_EKS_NODE_ID"
```

7.4 Validasi

7.4.1 Validasi SG ALB (Port 443 HTTPS Only)

```
aws --endpoint-url=$ENDPOINT ec2 describe-security-groups \
    --group-ids "$SG_ALB_ID" \
    --query
'SecurityGroups[0].IpPermissions[?ToPort==`443`].{Protocol:IpProtocol,
Port:ToPort, Source:IpRanges[0].CidrIp}' \
    --output table
```

7.4.2 Validasi SG EKS Worker Nodes (Traffic dari ALB & Intra-Node)

```
aws --endpoint-url=$ENDPOINT ec2 describe-security-groups \
    --group-ids "$SG_EKS_NODE_ID" \
    --query 'SecurityGroups[0].IpPermissions[0].{Protocol:IpProtocol,
FromPort:FromPort, ToPort:ToPort,
SourceSG:UserIdGroupPairs[0].GroupId}' \
    --output table
```

7.4.3 Validasi SG RDS Database (Traffic HANYA dari EKS Nodes)

```
aws --endpoint-url=$ENDPOINT ec2 describe-security-groups \
    --group-ids "$SG_RDS_ID" \
    --query
'SecurityGroups[0].IpPermissions[?ToPort==`7001`].{Protocol:IpProtocol
, Port:ToPort, Allowed_Source_SG:UserIdGroupPairs[0].GroupId}' \
    --output table
```

Step 8. IAM

8.1 Prepare Environment

```
mkdir ~/iam
cd ~/iam
```

8.2 Prepare EKS Cluster IAM Role

```
cat <<'EOF' > eks-cluster-trust-policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "eks.amazonaws.com"
```

```

    },
    "Action": "sts:AssumeRole"
  }
]
}
EOF

```

8.3 Trust Policy buat EKS Worker Node (EC2)

```

cat <<'EOF' > node-trust-policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ec2.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
EOF

```

8.4 Custom App Policy (S3 + Secrets + Logs)

```

cat <<'EOF' > enterprise-app-policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadWriteApplicationBucket",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::enterprise-app-bucket",
        "arn:aws:s3:::enterprise-app-bucket/*"
      ]
    },
    {
      "Sid": "ReadDatabaseSecret",
      "Effect": "Allow",
      "Action": ["secretsmanager:GetSecretValue"],
      "Resource": "*"
    },
    {
      "Sid": "CloudWatchLogs",
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "*"
    }
  ]
}
EOF

```

```
}  
EOF
```

8.5 Provision EKS Cluster Role

8.5.1 Create EKS Cluster Role

```
aws --endpoint-url="$ENDPOINT" iam create-role \  
    --role-name EnterpriseEKSClusterRole \  
    --assume-role-policy-document file://eks-cluster-trust-policy.json
```

8.5.2 Attach EKS Cluster Policy

```
aws --endpoint-url="$ENDPOINT" iam attach-role-policy \  
    --role-name EnterpriseEKSClusterRole \  
    --policy-arn arn:aws:iam::aws:policy/AmazonEKSClusterPolicy
```

8.5.3 Validasi, Expected: AmazonEKSClusterPolicy

```
aws --endpoint-url="$ENDPOINT" iam list-attached-role-policies \  
    --role-name EnterpriseEKSClusterRole \  
    --output table
```

8.6 Provision EKS Node Role

8.6.1 Create Node Role

```
aws --endpoint-url="$ENDPOINT" iam create-role \  
    --role-name EnterpriseEKSNodeRole \  
    --assume-role-policy-document file://node-trust-policy.json
```

8.6.2 Attach EKS Worker Node Policy

```
aws --endpoint-url="$ENDPOINT" iam attach-role-policy \  
    --role-name EnterpriseEKSNodeRole \  
    --policy-arn arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy
```

8.6.3 Attach ECR Read Policy

```
aws --endpoint-url="$ENDPOINT" iam attach-role-policy \  
    --role-name EnterpriseEKSNodeRole \  
    --policy-arn  
arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly
```

8.6.4 Attach CNI Policy, Ini kita pertahankan sebagai AWS-compatible node-role baseline. Tetapi jangan menganggap policy ini berarti Floci sedang menjalankan AWS VPC CNI sungguhan; real EKS Floci menggunakan k3s. Floci sendiri menjelaskan bahwa real mode menjalankan k3s container.

```
aws --endpoint-url="$ENDPOINT" iam attach-role-policy \  
    --role-name EnterpriseEKSNodeRole \  
    --policy-arn
```

```

--policy-arn arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy

# 8.7.1 Create Instance Profile, Ini tetap kita buat karena secara AWS
architecture worker EC2 memang biasanya menggunakan instance profile.

aws --endpoint-url="$ENDPOINT" iam create-instance-profile \
    --instance-profile-name EnterpriseEKSNODEInstanceProfile

# 8.7.2 Tambahkan role, Tapi: CreateNodegroup Floci menerima nodeRole,
bukan instance profile. Jadi object ini kita simpan sebagai AWS
architecture representation, bukan karena Floci akan attach profile
tersebut ke container k3s.

aws --endpoint-url="$ENDPOINT" iam add-role-to-instance-profile \
    --instance-profile-name EnterpriseEKSNODEInstanceProfile \
    --role-name EnterpriseEKSNODERole

# 8.8 Provision Application IAM Policy

# 8.8.1 Create Customer Managed Policy

aws --endpoint-url="$ENDPOINT" iam create-policy \
    --policy-name EnterpriseAppPolicy \
    --policy-document file://enterprise-app-policy.json

# 8.8.2 Ambil ARN

export
ENTERPRISE_APP_POLICY_ARN="arn:aws:iam::${AWS_ACCOUNT_ID}:policy/EnterpriseAppPolicy"

# 8.8.3 Validasi

aws --endpoint-url="$ENDPOINT" iam get-policy \
    --policy-arn "$ENTERPRISE_APP_POLICY_ARN" \
    --query
'Policy.{Name:PolicyName,Arn:Arn,DefaultVersion:DefaultVersionId}' \
    --output table

# 8.9 Validate IAM Foundation

# 8.9.1 EKS Cluster Role

echo

echo "=== EKS CLUSTER ROLE ==="

aws --endpoint-url="$ENDPOINT" iam get-role \
    --role-name EnterpriseEKSClusterRole \
    --query
'Role.{RoleName:RoleName,Principal:AssumeRolePolicyDocument.Statement[
0].Principal.Service}' \
    --output table

```

8.9.2 Cluster Role Policies, , Expected: AmazonEKSClusterPolicy

echo

echo "=== EKS CLUSTER ROLE POLICIES ==="

```
aws --endpoint-url="$ENDPOINT" iam list-attached-role-policies \
    --role-name EnterpriseEKSClusterRole \
    --query 'AttachedPolicies[].PolicyName' \
    --output table
```

8.9.3 Node Role Trust, Expected: ec2.amazonaws.com

echo

echo "=== EKS NODE ROLE ==="

```
aws --endpoint-url="$ENDPOINT" iam get-role \
    --role-name EnterpriseEKSNodeRole \
    --query
'Role.{RoleName:RoleName,Principal:AssumeRolePolicyDocument.Statement[
0].Principal.Service}' \
    --output table
```

8.9.4 Node Role Policies, Expected baseline: AmazonEKSWorkerNodePolicy, AmazonEKS_CNI_Policy, AmazonEC2ContainerRegistryReadOnly

echo

echo "=== EKS NODE ROLE POLICIES ==="

```
aws --endpoint-url="$ENDPOINT" iam list-attached-role-policies \
    --role-name EnterpriseEKSNodeRole \
    --query 'AttachedPolicies[].PolicyName' \
    --output table
```

8.9.5 Instance Profile, Expected: EnterpriseEKSNodeInstanceProfile, EnterpriseEKSNodeRole

echo

echo "=== EKS NODE INSTANCE PROFILE ==="

```
aws --endpoint-url="$ENDPOINT" iam get-instance-profile \
    --instance-profile-name EnterpriseEKSNodeInstanceProfile \
    --query
'InstanceProfile.{ProfileName:InstanceProfileName,Role:Roles[0].RoleNa
me}' \
    --output table
```

8.9.6 Application Policy Metadata

echo

```

echo "=== APPLICATION POLICY ==="

aws --endpoint-url="$ENDPOINT" iam get-policy \
    --policy-arn "$ENTERPRISE_APP_POLICY_ARN" \
    --query
'Policy.{Name:PolicyName,Arn:Arn,DefaultVersion:DefaultVersionId}' \
    --output table

```

8.9.7 Application Policy Statements

```

echo

echo "=== APPLICATION POLICY STATEMENTS ==="

APP_POLICY_VERSION=$(aws --endpoint-url="$ENDPOINT" iam get-policy \
    --policy-arn "$ENTERPRISE_APP_POLICY_ARN" \
    --query 'Policy.DefaultVersionId' \
    --output text)

aws --endpoint-url="$ENDPOINT" iam get-policy-version \
    --policy-arn "$ENTERPRISE_APP_POLICY_ARN" \
    --version-id "$APP_POLICY_VERSION" \
    --query
'PolicyVersion.Document.Statement[].{Sid:Sid,Effect:Effect,Action:Action,Resource:Resource}' \
    --output table

```

Phase 3 — Compute

Pada Phase 3 kita akan membangun compute layer dari enterprise application. Sampai Phase 2, kita baru punya:

VPC

Public Subnet A/B

Private App Subnet A/B

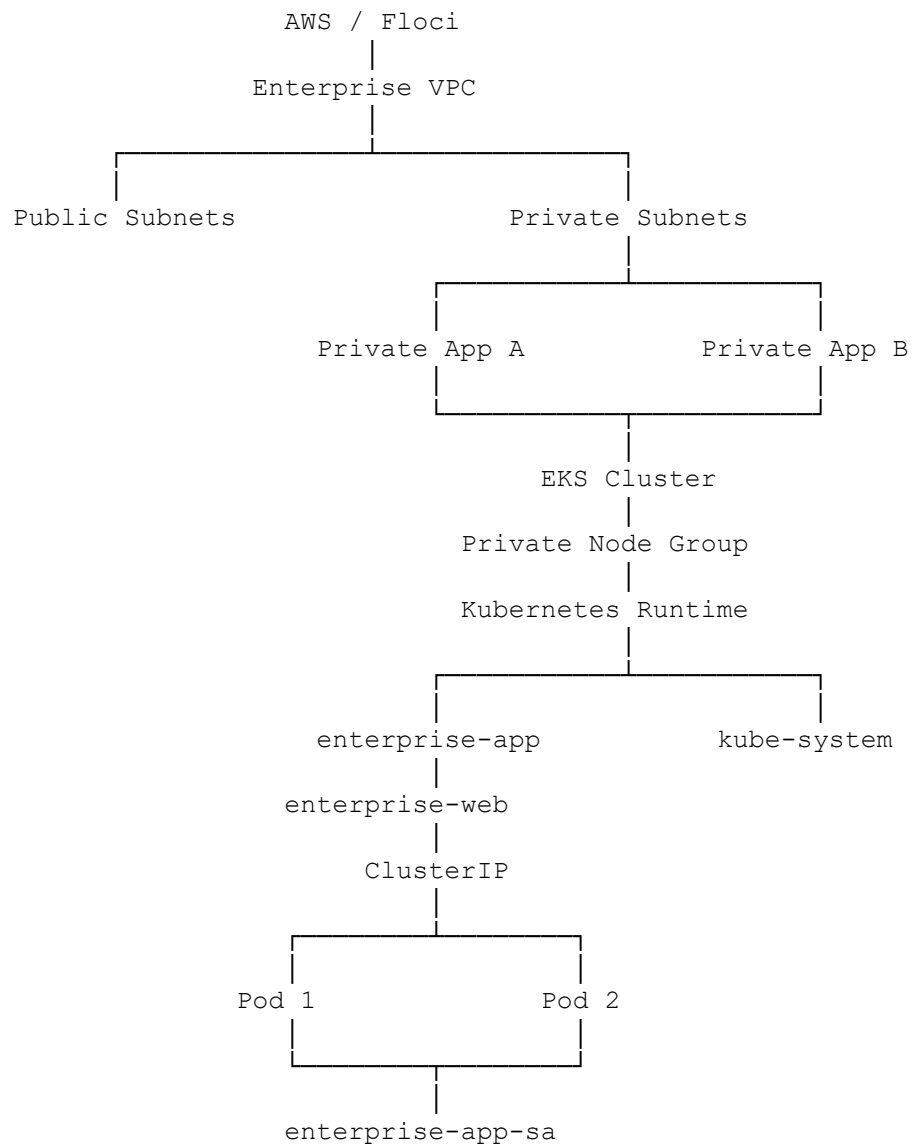
Private DB Subnet A/B

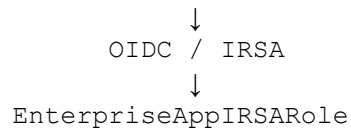
Security Groups

IAM Roles

IAM Policies

Sekarang kita akan memasukkan compute platform ke dalam architecture tersebut:



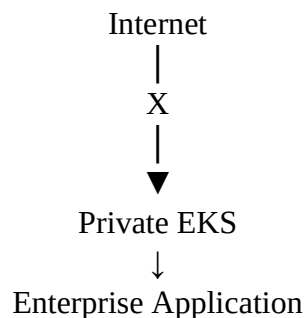


Phase 3 selesai kalau:

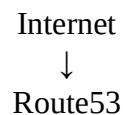
Component	Status
EKS Cluster	☒ ACTIVE
Private App Subnet A/B	☒
Private Node Group	☒ ACTIVE
Kubernetes API	☒
`kubectl`	☒
Kubernetes Nodes	☒ Ready
`enterprise-app` namespace	☒
ServiceAccount	☒
OIDC issuer	☒
IRSA role	☒
nginx Deployment	☒ 2/2
ClusterIP	☒
Internet exposure	☒
ALB	☒
WAF	☒
CloudWatch	☒
GuardDuty	☒
KMS integration	☒

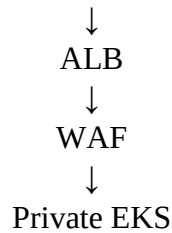
Step 9. Create EKS Cluster

Pada step ini kita membuat EKS control plane. Ini adalah titik di mana subnet yang sudah kita buat di Phase 1 akhirnya digunakan oleh EKS configuration. Kita akan menempatkan EKS ke Private App Subnet A dan Private App Subnet B. Kenapa bukan public subnet? Karena workload aplikasi kita tidak boleh langsung exposed ke Internet. Architecture yang kita inginkan:



Nanti baru pada phase berikutnya kita membuat:





9.1 Set Environment Variables, Pastikan variable dari Phase sebelumnya masih tersedia. Expected → VPC: vpc-xxxxxxx, Private App A: subnet-xxxxxxx, Private App B: subnet-xxxxxxx, EKS SG: sg-xxxxxxx. Kalau variable kosong, jangan lanjut.

```
export ENDPOINT="http://localhost:4566"
export AWS_REGION="us-east-1"
export AWS_ACCOUNT_ID="000000000000"
export EKS_CLUSTER_NAME="enterprise-eks"
echo "VPC: $VPC_ID"
echo "Private App A: $PRIVATE_APP_SUBNET_ID_A"
echo "Private App B: $PRIVATE_APP_SUBNET_ID_B"
echo "EKS SG: $SG_EKS_NODE_ID"
```

9.2 Create EKS Cluster, Kita sekarang membuat EKS cluster menggunakan → EnterpriseEKSClusterRole, Private App Subnet A, Private App Subnet B, EKS Security Group. Jalankan:

```
aws --endpoint-url="$ENDPOINT" eks create-cluster \
    --name "$EKS_CLUSTER_NAME" \
    --role-arn
"arn:aws:iam::${AWS_ACCOUNT_ID}:role/EnterpriseEKSClusterRole" \
    --resources-vpc-config
"subnetIds=$PRIVATE_APP_SUBNET_ID_A,$PRIVATE_APP_SUBNET_ID_B,securityG
roupIds=$SG_EKS_NODE_ID"
```

9.3 Validate EKS Cluster Status Kita cek apakah cluster berhasil dibuat. Yang paling penting: Status = ACTIVE

```
aws --endpoint-url="$ENDPOINT" eks describe-cluster \
    --name "$EKS_CLUSTER_NAME" \
    --query
'cluster.{Name:name,Status:status,Version:version,Endpoint:endpoint}' \
    --output table
```

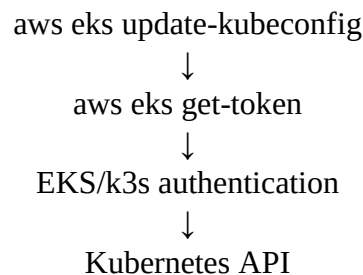
9.4 Validate Subnets, Sekarang kita pastikan cluster menyimpan subnet yang benar.

```
aws --endpoint-url="$ENDPOINT" eks describe-cluster \
    --name "$EKS_CLUSTER_NAME" \
```

```
--query 'cluster.resourcesVpcConfig.subnetIds' \
--output table
```

Step 10. Connect kubectl

Sekarang kita berpindah dari AWS API ke Kubernetes API. Ini checkpoint sangat penting. Karena EKS bukan cuma resource AWS. Kita ingin membuktikan bahwa cluster Kubernetes yang berada di balik EKS abstraction benar-benar bisa kita operasikan. Floci mendukung workflow native:



dan real mode menggunakan k3s.

10.1 Create kubeconfig, Expected: Added new context ...

```
aws --endpoint-url="$ENDPOINT" eks update-kubeconfig \
    --name "$EKS_CLUSTER_NAME"
```

10.2 Validate Current Context, Expected: arn:aws:eks:us-east-1:000000000000:cluster/enterprise-eks. Nama context bisa sedikit berbeda, jadi jangan terpaku 100%. Yang penting context menunjuk ke: **enterprise-eks**

```
kubectl config current-context
```

10.3 Validate Kubernetes Nodes, harus STATUS = Ready

```
kubectl get nodes
```

10.4 Validate All Kubernetes Components, tidak boleh ada CrashLoopBackOff, ImagePullBackOff, Pending

```
kubectl get pods -A
```

Step 11. Create Private Node Group

Sekarang kita membuat EKS managed node group metadata. Secara AWS architecture, node group adalah kumpulan worker nodes yang menjalankan workload Kubernetes.

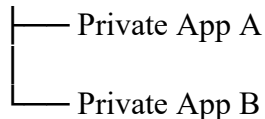
Architecture:

EKS Cluster



Private Node Group





Catatan Floci: nodegroup ini direpresentasikan sebagai metadata; jangan menganggap command ini membuat dua EC2 instance sungguhan. Real Kubernetes runtime-nya berasal dari k3s container yang dibuat ketika cluster dibuat.

11.1 Create Private Node Group, Kita sedang mendefinisikan Node Group: enterprise-private-nodegroup; Location: Private App A & Private App B; Desired: 2; Minimum: 2; Maximum: 4

```
aws --endpoint-url="$ENDPOINT" eks create-nodegroup \
    --cluster-name "$EKS_CLUSTER_NAME" \
    --nodegroup-name enterprise-private-nodegroup \
    --node-role
"arn:aws:iam::${AWS_ACCOUNT_ID}:role/EnterpriseEKSTNodeRole" \
    --subnets \
        "$PRIVATE_APP_SUBNET_ID_A" \
        "$PRIVATE_APP_SUBNET_ID_B" \
    --instance-types t3.micro \
    --scaling-config minSize=2,maxSize=4,desiredSize=2
```

11.2 Validate Node Group, Cari bagian nodegroupName, status, subnets, nodeRole, instanceTypes, scalingConfig

```
aws --endpoint-url="$ENDPOINT" eks describe-nodegroup \
    --cluster-name "$EKS_CLUSTER_NAME" \
    --nodegroup-name enterprise-private-nodegroup
```

11.3 Validate Node Group – Compact, Supaya lebih gampang dibaca, Yang paling penting: Status = ACTIVE

```
aws --endpoint-url="$ENDPOINT" eks describe-nodegroup \
    --cluster-name "$EKS_CLUSTER_NAME" \
    --nodegroup-name enterprise-private-nodegroup \
    --query
'nodegroup.{Name:nodegroupName,Status:status,Subnets:subnets,Role:node
Role,InstanceTypes:instanceTypes,Desired:scalingConfig.desiredSize,Min
:scalingConfig.minSize,Max:scalingConfig.maxSize}' \
    --output table
```

11.4 Validate Kubernetes Nodes, Kita ingin memastikan Kubernetes API tetap sehat setelah nodegroup dibuat.

```
kubectl get nodes -o wide
```

Step 12. Validate EKS OIDC

Ini bukan "enable OIDC". OIDC sudah dibuat otomatis ketika EKS cluster dibuat. Yang kita lakukan sekarang adalah mengambil OIDC issuer yang dibuat Floci. Floci mendokumentasikan bahwa setiap cluster mendapatkan OIDC identity provider dan `DescribeCluster` mengembalikan cluster.identity.oidc.issuer

Issuer tersebut berbentuk:

`https://oidc.eks.us-east-1.amazonaws.com/id/<generated-id>`

12.1 Retrieve OIDC Issuer, Expected: `https://oidc.eks.us-east-1.amazonaws.com/id/XXXXXXXXXXXXXX`. ID tersebut harus generated, bukan FLOCI MOCK_OIDC

```
export EKS_OIDC_ISSUER=$(aws --endpoint-url="$ENDPOINT" eks describe-cluster \
```

```
--name "$EKS_CLUSTER_NAME" \
```

```
--query 'cluster.identity.oidc.issuer' \
```

```
--output text)
```

```
echo "$EKS_OIDC_ISSUER"
```

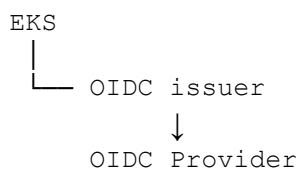
12.2 Extract OIDC Provider AWS IAM trust policy tidak menggunakan `https://`, Expected: `oidc.eks.us-east-1.amazonaws.com/id/XXXXXXXXXXXXXX`

```
export OIDC_PROVIDER="${EKS_OIDC_ISSUER#https://}"
```

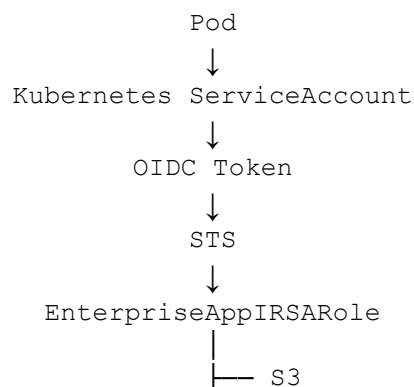
```
echo "$OIDC_PROVIDER"
```

Step 13. Create IRSA Role

Sekarang baru kita membuat IRSA. kita sudah tahu:



Architecture yang ingin kita bangun:



└─ Secrets Manager
└─ CloudWatch Logs

Floci secara eksplisit mendukung AssumeRoleWithWebIdentity dan memvalidasi sub, aud, issuer, signature, dan expiry token.

13.1 Create IRSA Trust Policy, Definisi → Kita mengatakan " Hanya Kubernetes ServiceAccount bernama `enterprise-app-sa` di namespace `enterprise-app` yang boleh menggunakan IAM role ini melalui OIDC." Jadi bukan semua Pod dan bukan semua ServiceAccount, tetapi: enterprise-app (enterprise-app-sa) → EnterpriseAppIRSARole. Ini adalah least privilege identity boundary.

```
mkdir ~/phase3

cd ~/phase3

cat <<EOF > irsa-trust-policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::${AWS_ACCOUNT_ID}:oidc-provider/${OIDC_PROVIDER}"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "${OIDC_PROVIDER}:sub": "system:serviceaccount:enterprise-app:enterprise-app-sa",
          "${OIDC_PROVIDER}:aud": "sts.amazonaws.com"
        }
      }
    }
  ]
}
EOF
```

13.2 Create IRSA Role

```
aws --endpoint-url="$ENDPOINT" iam create-role \
  --role-name EnterpriseAppIRSARole \
  --assume-role-policy-document file://irsa-trust-policy.json
```

13.3 Attach Application Policy

```
aws --endpoint-url="$ENDPOINT" iam attach-role-policy \
  --role-name EnterpriseAppIRSARole \
  --policy-arn
"arn:aws:iam::${AWS_ACCOUNT_ID}:policy/EnterpriseAppPolicy"

aws --endpoint-url="$ENDPOINT" iam list-attached-role-policies \
  --role-name EnterpriseAppIRSARole \
```

```

--output table

# 13.4 Validate IRSA Role, Target harus menunjukkan "Action:
sts:AssumeRoleWithWebIdentity" dan " Principal:
arn:aws:iam::000000000000:oidc-provider/${OIDC_PROVIDER}", serta "sub:
system:serviceaccount:enterprise-app:enterprise-app-sa", "aud:
sts.amazonaws.com"

aws --endpoint-url="$ENDPOINT" iam get-role \
    --role-name EnterpriseAppIRSARole \
    --query 'Role.AssumeRolePolicyDocument' \
    --output json

```

Step 14. Create Enterprise Application Namespace

Sekarang kita mulai masuk ke Kubernetes workload. Kita tidak akan menggunakan namespace default. Kita membuat isolation boundary **enterprise-app**

14.1 Create Namespace

```
kubectl create namespace enterprise-app
```

14.2 Validate Namespace

```
kubectl get namespace enterprise-app
```

Step 15. Create Application ServiceAccount

ServiceAccount ini nanti menjadi identity yang mendapatkan **EnterpriseAppIRSARole**

15.1 Create ServiceAccount

```

cat <<EOF > enterprise-app-serviceaccount.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: enterprise-app-sa
  namespace: enterprise-app
  annotations:
    eks.amazonaws.com/role-arn:
arn:aws:iam::${AWS_ACCOUNT_ID}:role/EnterpriseAppIRSARole
EOF

```

15.2 Apply

```
kubectl apply -f enterprise-app-serviceaccount.yaml
```

15.3 Validate ServiceAccount, Target → metadata: (name: enterprise-app-sa, namespace: enterprise-app, annotations: eks.amazonaws.com/role-arn: ...)

```

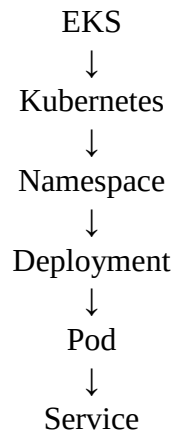
kubectl get serviceaccount \
    enterprise-app-sa \

```

```
-n enterprise-app \  
-o yaml
```

Step 16. Deploy Test Workload

Sekarang kita ingin membuktikan:



Kita belum membuat Internet exposure.

16.1 Create Enterprise Web Deployment, Perhatikan, gue tambahkan: serviceAccountName: enterprise-app-sa. Ini penting supaya workload kita benar-benar menggunakan identity yang tadi kita buat.

```
cat <<'EOF' > enterprise-web.yaml  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: enterprise-web  
  namespace: enterprise-app  
spec:  
  replicas: 2  
  selector:  
    matchLabels:  
      app: enterprise-web  
  template:  
    metadata:  
      labels:  
        app: enterprise-web  
    spec:  
      serviceAccountName: enterprise-app-sa  
      containers:  
        - name: web  
          image: nginx:latest  
          ports:  
            - containerPort: 80  
          resources:  
            requests:  
              cpu: 100m  
              memory: 128Mi  
            limits:
```

```

        cpu: 500m
        memory: 256Mi
---
apiVersion: v1
kind: Service
metadata:
  name: enterprise-web
  namespace: enterprise-app
spec:
  type: ClusterIP
  selector:
    app: enterprise-web
  ports:
    - port: 80
      targetPort: 80
EOF

```

16.2 Apply Workload

```
kubectl apply -f enterprise-web.yaml
```

16.3 Validate Deployment, target → READY = 2/2 dan AVAILABLE = 2

```

kubectl get deployment \
    enterprise-web \
    -n enterprise-app

```

16.4 Validate Pods, target READY = 1/1 dan STATUS = Running

```

kubectl get pods \
    -n enterprise-app \
    -o wide

```

16.5 Validate Service

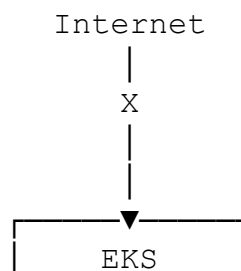
```

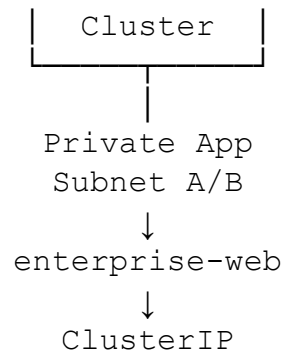
kubectl get svc \
    enterprise-web \
    -n enterprise-app

```

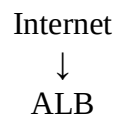
Step 17. Validate Private Workload Architecture

Sekarang kita ingin membuktikan bahwa workload kita belum exposed ke Internet. Architecture saat ini:

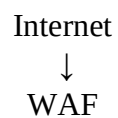




Belum ada:



Belum ada:



Belum ada NodePort dan belum ada LoadBalancer, Ini sengaja. Phase 3 hanya membuktikan compute layer.

17.1 Validate Service Type

```
kubectl get svc \
    enterprise-web \
    -n enterprise-app \
    -o jsonpath='{.spec.type}{"\n"}'
```

Step 18. Test Internal Application Connectivity

Sekarang kita membuktikan bahwa application benar-benar hidup. Kita akan membuat temporary client di dalam cluster.

18.1 Create Temporary Test Pod

```
kubectl run internal-client \
    -n enterprise-app \
    --image=curlimages/curl:latest \
    --restart=Never \
    --rm \
```

```

    -it \

    -- sh

# 18.2 Di dalam pod lalu ketik dibawah ini, target → keluar welcome
nginx, Ini membuktikan Pod → Kubernetes DNS → ClusterIP → enterprise-
web → nginx

curl http://enterprise-web

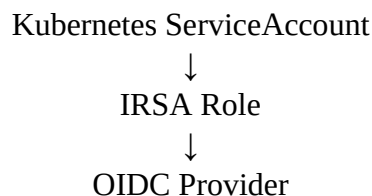
# 18.3 Exit Test Pod, Karena kita menggunakan --rm, temporary pod akan
otomatis dihapus.

exit

```

Step 19. Validate IRSA Foundation

Sekarang kita melakukan identity checkpoint. Kita belum perlu melakukan full application → S3 → STS test. Yang kita pastikan dulu adalah:



```

# 19.1 Validate ServiceAccount, Expected
arn:aws:iam::000000000000:role/EnterpriseAppIRSARole

kubectl get serviceaccount \
    enterprise-app-sa \
    -n enterprise-app \
    -o jsonpath='{.metadata.annotations.eks\.amazonaws\.com/role-
arn}{"\n"}'

# 19.2 Validate IAM Role, Expected EnterpriseAppIRSARole

aws --endpoint-url="$ENDPOINT" iam get-role \
    --role-name EnterpriseAppIRSARole \
    --query 'Role.RoleName' \
    --output text

# 19.3 Validate OIDC Issuer, Expected: https://oidc.eks.us-east-
1.amazonaws.com/id/XXXXXXXXX

echo "$EKS_OIDC_ISSUER"

```

Step 20. Final Phase 3 Validation

Ini adalah checkpoint utama. Kalau semua bagian di bawah berhasil, Phase 3 dianggap selesai.

20.1 EKS Cluster, Target → Name enterprise-eks, Status ACTIVE

```
echo "====="
echo "=== EKS CLUSTER ==="
echo "====="
aws --endpoint-url="$ENDPOINT" eks describe-cluster \
    --name "$EKS_CLUSTER_NAME" \
    --query 'cluster.{Name:name,Status:status,Version:version}' \
    --output table
```

20.2 EKS Network, Target → Private App A, Private App B

```
echo "====="
echo "=== EKS SUBNETS ==="
echo "====="
aws --endpoint-url="$ENDPOINT" eks describe-cluster \
    --name "$EKS_CLUSTER_NAME" \
    --query 'cluster.resourcesVpcConfig.subnetIds' \
    --output table
```

20.3 Node Group, Target → Name enterprise-private-nodegroup, Status ACTIVE, Desired 2, Min 2, Max 4

```
echo "====="
echo "=== NODE GROUP ==="
echo "====="
aws --endpoint-url="$ENDPOINT" eks describe-nodegroup \
    --cluster-name "$EKS_CLUSTER_NAME" \
    --nodegroup-name enterprise-private-nodegroup \
    --query
'nodegroup.{Name:nodegroupName,Status:status,Subnets:subnets,Desired:scalingConfig.desiredSize,Min:scalingConfig.minSize,Max:scalingConfig.maxSize}' \
    --output table
```

20.4 Kubernetes Nodes, Target → STATUS = Ready

```
echo "====="
echo "=== KUBERNETES NODES ==="
echo "====="
```

```
kubectl get nodes -o wide
```

20.5 Kubernetes Namespace, Target → enterprise-app Active

```
echo "=====
```

```
echo "=== NAMESPACE ==="
```

```
echo "=====
```

```
kubectl get namespace enterprise-app
```

20.6 Application, Target → Deployment enterprise-web 2/2 dan Pods: 1/1 Running, 1/1 Running

```
echo "=====
```

```
echo "=== APPLICATION ==="
```

```
echo "=====
```

```
kubectl get deployment \
```

```
    -n enterprise-app
```

```
kubectl get pods \
```

```
    -n enterprise-app \
```

```
    -o wide
```

20.7 Service, Target → enterprise-web ClusterIP

```
echo "=====
```

```
echo "=== SERVICES ==="
```

```
echo "=====
```

```
kubectl get svc \
```

```
    -n enterprise-app
```

20.8 OIDC, Target → Issuer: https://oidc.eks.us-east-1.amazonaws.com/id/XXXXXXXXX. Provider: oidc.eks.us-east-1.amazonaws.com/id/XXXXXXXXX

```
echo "=====
```

```
echo "=== OIDC ==="
```

```
echo "=====
```

```
echo "Issuer:"
```

```
echo "$EKS_OIDC_ISSUER"
```

```
echo
```

```
echo "Provider:"
```

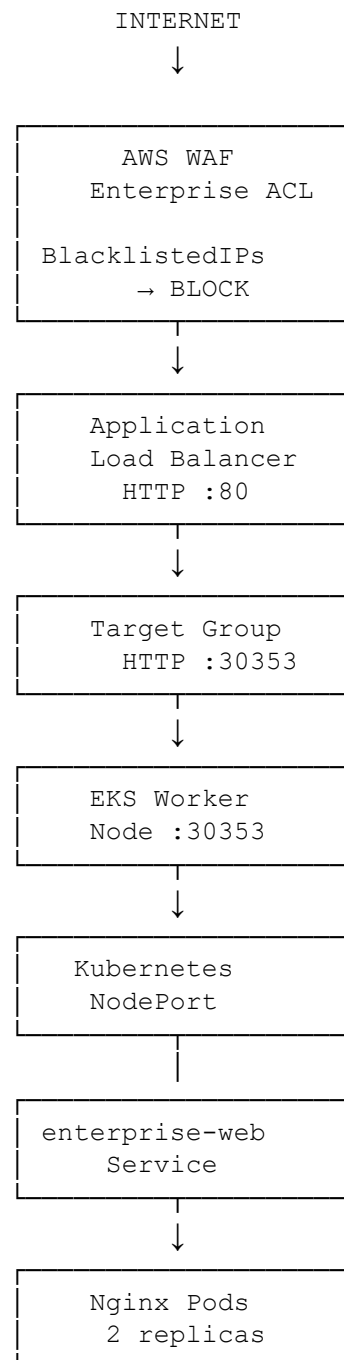
```
echo "$OIDC_PROVIDER"
```

20.9 IRSA, Expected → arn:aws:iam::000000000000:role/EnterpriseAppIRSARole dan EnterpriseAppPolicy

```
echo "====="
echo "=== IRSA ==="
echo "====="
kubectl get serviceaccount \
    enterprise-app-sa \
    -n enterprise-app \
    -o jsonpath='{.metadata.annotations.eks\.amazonaws\.com/role-arn}'
aws --endpoint-url="$ENDPOINT" iam list-attached-role-policies \
    --role-name EnterpriseAppIRSARole \
    --query 'AttachedPolicies[].PolicyName' \
    --output table
```

Phase 4 — Traffic Management & Edge Security

Phase ini membangun traffic management dan perimeter security layer di atas workload EKS yang telah dibuat pada Phase 3. Secara arsitektur, workload yang sebelumnya hanya dapat diakses melalui Kubernetes ClusterIP akan diekspos melalui NodePort, kemudian direpresentasikan sebagai target backend bagi AWS Application Load Balancer (ALB). Setelah itu, AWS WAF akan diposisikan sebagai security control pada layer aplikasi sebelum traffic mencapai ALB. Target architecture:



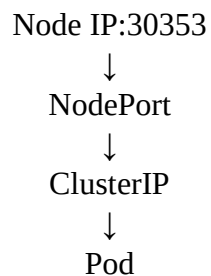
Floci limitation: Pada environment emulator yang digunakan, provisioning ALB dan WAF dapat dilakukan melalui AWS API, tetapi data-plane enforcement tidak sepenuhnya merepresentasikan AWS production. Berdasarkan pengujian, request ke WAF tidak benar-benar diblokir dan ALB listener tidak dapat digunakan sebagai external HTTP endpoint. Oleh karena itu, Phase 4 membedakan antara control-plane validation dan data-plane validation.

Phase 4 Status

Layer	Result	Validation
NodePort	☑ PASS	NODE_IP:30353 → HTTP 200
Kubernetes routing	☑ PASS	Pod response received
Target Group	☑ PASS	AWS API
Target registration	☑ PASS	AWS API
ALB provisioning	☑ PASS	AWS API
ALB listener	☑ PASS	AWS API
ALB external forwarding	⚠ Not verified	Floci limitation
WAF IP Set	☑ PASS	AWS API
WAF Web ACL	☑ PASS	AWS API
WAF → ALB association	☑ PASS	AWS API
WAF blocking	⚠ Not verified	Floci returned 200

Step 21. Expose Enterprise Application melalui NodePort

Menyediakan endpoint pada Kubernetes Worker Node yang dapat digunakan sebagai backend oleh external traffic-management layer. Pada Phase 3, application service menggunakan ClusterIP karena workload masih bersifat internal: ClusterIP → Pod. Pada Phase 4, service diubah menjadi:



21.1 Convert Service menjadi NodePort, Parameter type: NodePort → membuat Kubernetes membuka port pada setiap node untuk meneruskan traffic ke Service. port: 80 → port Service yang digunakan secara internal oleh Kubernetes. targetPort: 80 → port aplikasi di dalam Pod. nodePort: 30353 → port eksternal pada Worker Node yang akan digunakan sebagai backend endpoint. selector → memastikan traffic diarahkan ke Pod dengan label app: enterprise-web. Dengan demikian: Node :30353 → Service :80 → Pod :80

```

kubect1 apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:

```

```

    name: enterprise-web
    namespace: enterprise-app
spec:
  type: NodePort
  selector:
    app: enterprise-web
  ports:
    - name: http
      port: 80
      targetPort: 80
      nodePort: 30353
EOF

```

21.2 Validasi Service, Expected → NAME: enterprise-web; TYPE: NodePort; CLUSTER-IP: 10.x.x.x; EXTERNAL-IP: <none>; PORT(S): 80:30353/TCP

```
kubectl get svc enterprise-web -n enterprise-app
```

21.3 Validasi tambahan, Expected → NodePort 30353

```

kubectl get svc enterprise-web \
  -n enterprise-app \
  -o jsonpath='{.spec.type}{"\n"}{.spec.ports[0].nodePort}{"\n"}'

```

Step 22. Validate Kubernetes NodePort Data Plane

Memastikan perubahan dari ClusterIP menjadi NodePort benar-benar menghasilkan jalur komunikasi yang berfungsi sebelum kita memasukkan ALB. Ini merupakan data-plane checkpoint utama.

22.1 Retrieve NodePort, Expected → NODE_PORT=30353

```

export NODE_PORT=$(
  kubectl get svc enterprise-web \
    -n enterprise-app \
    -o jsonpath='{.spec.ports[0].nodePort}'
)

```

```
echo "NODE_PORT=$NODE_PORT"
```

22.2 Retrieve Worker Node IP, Expected → Contoh: NODE_IP=172.x.x.x

```

export NODE_IP=$(
  kubectl get nodes \
    -o
  jsonpath='{.items[0].status.addresses[?(@.type=="InternalIP")].address}'
)
echo "NODE_IP=$NODE_IP"

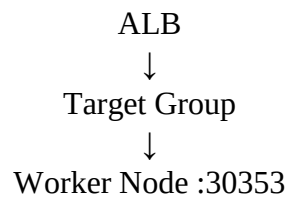
```

22.3 Test NodePort, Expected → HTTP/1.1 200 OK. dan response body Nginx: Welcome to nginx! Kesimpulannya adalah Jika mendapatkan HTTP 200, maka jalur Worker Node IP → :30353 → Kubernetes NodePort → enterprise-web Service → Nginx Pod telah tervalidasi.

```
curl -v "http://${NODE_IP}:${NODE_PORT}"
```

Step 23. Create ALB Target Group

Membuat backend pool yang mendefinisikan target application endpoint yang akan menerima traffic dari ALB. Dalam desain ini, target bukan Pod secara langsung, melainkan Worker Node pada NodePort:



23.1 Retrieve VPC ID, Expected → VPC_ID=vpc-xxxxxxxx

```
VPC_ID=$(aws ec2 describe-vpcs \
    --endpoint-url=http://localhost:4566 \
    --filters "Name=tag:Name,Values=enterprise-vpc" \
    --query "Vpcs[0].VpcId" \
    --output text)
echo "VPC_ID=$VPC_ID"
```

23.2 Create Target Group, Parameter penting → --name - nama logical target group. --protocol HTTP adalah protocol komunikasi antara ALB dan backend. --port "\$NODE_PORT" adalah backend menggunakan NodePort, yaitu 30353, bukan port 80. --vpc-id adalah menempatkan target group pada VPC enterprise. --target-type instance adalah target direpresentasikan sebagai node/instance, bukan IP Pod. --health-check-path "/" adalah endpoint yang digunakan untuk memeriksa kesehatan backend. --query adalah mengambil ARN target group dari response API.

```
export TG_ARN=$(
    aws --endpoint-url="$ENDPOINT" \
    elbv2 create-target-group \
    --name enterprise-target-group \
    --protocol HTTP \
    --port "$NODE_PORT" \
    --vpc-id "$VPC_ID" \
    --target-type instance \
    --health-check-path "/" \
```

```

    --query 'TargetGroups[0].TargetGroupArn' \
    --output text
)
echo "TG_ARN=$TG_ARN"

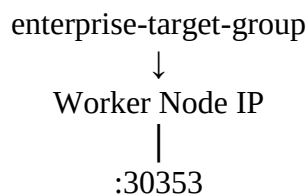
# 23.3 Validasi Target Group, Expected → Response menunjukkan:
TargetGroupName = enterprise-target-group; Protocol = HTTP; Port =
30353; TargetType = instance

aws --endpoint-url="$ENDPOINT" \
    elbv2 describe-target-groups \
    --target-group-arns "$TG_ARN"

```

Step 24. Register EKS Worker Node sebagai Target

Menghubungkan target group dengan endpoint NodePort yang telah terbukti dapat melayani request. Architecture:



24.1 Register Target, Parameter → --target-group-arn adalah target group yang menerima registration. Id="\$NODE_IP" adalah endpoint Worker Node yang menjadi backend. Port="\$NODE_PORT" adalah port NodePort tempat application traffic diterima.

```

aws --endpoint-url="$ENDPOINT" \
    elbv2 register-targets \
    --target-group-arn "$TG_ARN" \
    --targets Id="$NODE_IP",Port="$NODE_PORT"

```

24.2 Validasi Registration, Expected → Target terdaftar dengan: Target: Id = <NODE_IP> dan Port = 30353. Status dapat berupa: initial atau healthy tergantung kemampuan health-check data plane Floci. Jika target tetap unhealthy, validasi kembali secara independen menggunakan curl http://\$NODE_IP:\$NODE_PORT. Jika direct NodePort tetap 200 OK, berarti Kubernetes data plane sehat dan masalah berada pada jalur ELB emulator.

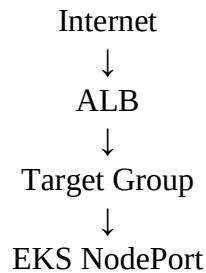
```

aws --endpoint-url="$ENDPOINT" \
    elbv2 describe-target-health \
    --target-group-arn "$TG_ARN"

```

Step 25. Provision Internet-Facing Application Load Balancer

Merepresentasikan entry point aplikasi pada perimeter network. Architecture:



ALB ditempatkan pada public subnets agar secara arsitektur dapat menerima traffic dari Internet.

25.1 Create ALB, Parameter → --subnets adalah menempatkan ALB pada public subnets, Dua subnet digunakan untuk merepresentasikan deployment multi-AZ. --security-groups adalah menerapkan security group ALB yang telah dibuat sebelumnya. --scheme internet-facing adalah mendefinisikan ALB sebagai Internet-facing. --type application adalah menggunakan Application Load Balancer.

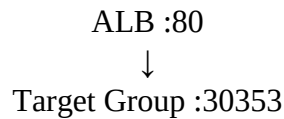
```
export ALB_ARN=$(
  aws --endpoint-url="$ENDPOINT" \
    elbv2 create-load-balancer \
    --name enterprise-alb \
    --subnets \
      "$PUBLIC_SUBNET_ID_A" \
      "$PUBLIC_SUBNET_ID_B" \
    --security-groups "$SG_ALB_ID" \
    --scheme internet-facing \
    --type application \
    --query 'LoadBalancers[0].LoadBalancerArn' \
    --output text
)
echo "ALB_ARN=$ALB_ARN"

# 25.2 Validasi ALB, Expected → State: Code = active dan Scheme =
internet-facing Type = application

aws --endpoint-url="$ENDPOINT" \
  elbv2 describe-load-balancers \
  --load-balancer-arns "$ALB_ARN"
```

Step 26. Create ALB HTTP Listener

Membuat listener yang menerima HTTP request pada ALB dan meneruskannya ke target group. Traffic model:



26.1 Create Listener, Parameter → --protocol HTTP adalah protocol request yang diterima listener. --port 80 adalah port listener. Type=forward adalah meneruskan request ke backend. TargetGroupArn adalah target group tujuan forwarding.

```
export LISTENER_ARN=$(
  aws --endpoint-url="$ENDPOINT" \
    elbv2 create-listener \
      --load-balancer-arn "$ALB_ARN" \
      --protocol HTTP \
      --port 80 \
      --default-actions \
        Type=forward,TargetGroupArn="$TG_ARN" \
      --query 'Listeners[0].ListenerArn' \
      --output text
)

echo "LISTENER_ARN=$LISTENER_ARN"
```

26.2 Validasi Listener, Expected → Protocol = HTTP dan Port = 80. Default action: Type = forward, TargetGroup = enterprise-target-group

```
aws --endpoint-url="$ENDPOINT" \
  elbv2 describe-listeners \
    --load-balancer-arn "$ALB_ARN"
```

Step 27. Validate ALB Data Plane

Menguji apakah ALB emulator benar-benar meneruskan HTTP traffic menuju NodePort. Ini sengaja menjadi step terpisah karena control plane dan data plane harus divalidasi secara independen.

27.1 Retrieve ALB DNS

```
export ALB_DNS=$(
  aws --endpoint-url="$ENDPOINT" \
    elbv2 describe-load-balancers \
```

```

--load-balancer-arns "$ALB_ARN" \
--query 'LoadBalancers[0].DNSName' \
--output text
)

```

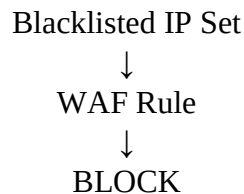
```
echo "ALB_DNS=$ALB_DNS"
```

27.2 Test ALB, Expected pada AWS real → HTTP/1.1 200 OK, dengan response dari application. Expected pada environment Floci → Berdasarkan eksperimen yang sudah dilakukan: Connection refused. Ini bukan berarti konfigurasi AWS resource gagal. Control plane sudah tervalidasi: ALB ✓ Listener ✓ Target Group ✓ Target ✓. Tetapi ALB external data plane: ALB socket → NodePort NOT FUNCTIONALLY VERIFIED. Jadi di laporan: ALB control-plane provisioning was successfully validated; however, end-to-end ALB data-plane forwarding could not be validated in the Floci emulator environment. Direct Worker Node-to-NodePort testing successfully demonstrated the underlying Kubernetes application data plane.

```
curl -v "http://${ALB_DNS}"
```

Step 28. Create WAF IP Set

Membuat daftar alamat IP yang dikategorikan sebagai blocked sources. Architecture:



28.1 Create IP Set, Parameter → --name adalah identifier IP set. --scope REGIONAL adalah WAF digunakan untuk regional resources seperti ALB. --ip-address-version IPV4 adalah IP set menggunakan IPv4. --addresses "1.2.3.4/32" adalah satu alamat IPv4 yang digunakan sebagai contoh blocked source. /32 adalah menunjukkan satu alamat IP individual.

```

export IP_SET_ARN=$(
aws --endpoint-url="$ENDPOINT" \
  wafv2 create-ip-set \
  --name "BlacklistedIPs" \
  --scope REGIONAL \
  --ip-address-version IPV4 \
  --addresses "1.2.3.4/32" \
  --query 'Summary.ARN' \
  --output text
)

```

```
)

echo "IP_SET_ARN=$IP_SET_ARN"

# 28.2 Validasi IP Set, Expected → BlacklistedIPs dengan ARN:
arn:aws:wafv2:...:regional/ipset/BlacklistedIPs/...

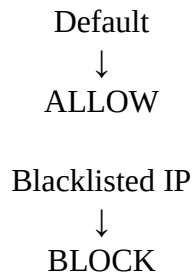
aws --endpoint-url="$ENDPOINT" \

    wafv2 list-ip-sets \

    --scope REGIONAL
```

Step 29. Create AWS WAF Web ACL

Membuat policy enforcement layer yang menentukan bagaimana WAF menangani HTTP request. Pada tahap ini kita membuat policy:



29.1 Create Web ACL, Parameter penting → --default-action Allow={} adalah request yang tidak terkena rule akan diizinkan. BlockBadIPs adalah rule security yang memblokir IP dari IP set. Priority: 1 adalah urutan evaluasi rule. IPSetReferenceStatement adalah menggunakan BlacklistedIPs sebagai source data. Action: Block adalah request yang match akan ditolak. VisibilityConfig adalah mengaktifkan sampled request dan metric untuk monitoring.

```
export WEB_ACL_ARN=$(

aws --endpoint-url="$ENDPOINT" \

    wafv2 create-web-acl \

    --name "Enterprise-WebACL" \

    --scope REGIONAL \

    --default-action Allow={} \

    --visibility-config \

        SampledRequestsEnabled=true,CloudWatchMetricsEnabled=true,MetricName=E
        nterpriseWebACL \

    --rules '[

        {

            "Name": "BlockBadIPs",

            "Priority": 1,
```

```

    "Statement": {
        "IPSetReferenceStatement": {
            "ARN": "'"$IP_SET_ARN"' "
        }
    },
    "Action": {
        "Block": {}
    },
    "VisibilityConfig": {
        "SampledRequestsEnabled": true,
        "CloudWatchMetricsEnabled": true,
        "MetricName": "BlockBadIPsMetric"
    }
}
]' \
--query 'Summary.ARN' \
--output text
)

echo "WEB_ACL_ARN=$WEB_ACL_ARN"

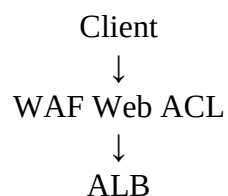
# 29.2 Validasi Web ACL, Expected → Enterprise-WebACL

aws --endpoint-url="$ENDPOINT" \
    wafv2 list-web-acls \
    --scope REGIONAL

```

Step 30. Associate WAF dengan ALB

Menghubungkan Web ACL dengan ALB sehingga secara AWS architecture model, request yang menuju ALB akan melewati WAF policy. Architecture:



30.1 Associate

```

aws --endpoint-url="$ENDPOINT" \
    wafv2 associate-web-acl \

```

```

--web-acl-arn "$WEB_ACL_ARN" \

--resource-arn "$ALB_ARN"

# 30.2 Validasi Association, Expected → Response mengandung Enterprise-
WebACL atau WEB_ACL_ARN

aws --endpoint-url="$ENDPOINT" \

    wafv2 get-web-acl-for-resource \

    --resource-arn "$ALB_ARN"

```

Step 31. Validate WAF Enforcement

Menguji apakah rule BlockBadIPs benar-benar melakukan traffic inspection dan blocking. Ini adalah security validation yang penting karena keberadaan Web ACL saja belum membuktikan enforcement.

31.1 Request menggunakan Blacklisted IP, Expected pada AWS real → Request yang source IP-nya benar-benar terdeteksi sebagai 1.2.3.4 akan BLOCK dan umumnya menghasilkan HTTP response 403. Actual pada Floci → Pada Eksperimen kita menghasilkan HTTP/1.1 200 OK. Ini menunjukkan bahwa pada environment Floci yang sedang digunakan: WAF IP Set ✓ WAF Web ACL ✓ WAF Rule ✓ WAF → Resource ✓ Actual HTTP blocking X. Jadi jangan mencoba mengakali hasil ini dengan menganggap X-Forwarded-For sebagai source IP sebenarnya. Header tersebut hanyalah HTTP header. Pada AWS WAF production, source address yang dievaluasi secara default berasal dari connection source IP, kecuali konfigurasi forwarded-IP handling digunakan. Untuk project ini, hasil 200 OK dicatat sebagai: WAF control-plane configuration successfully provisioned and associated with the ALB. However, request-level WAF enforcement could not be demonstrated in the Floci emulator.

```

curl -I \

    -H "X-Forwarded-For: 1.2.3.4" \

    http://localhost:4566/

```

Step 32. Final Phase 4 Validation

Memastikan seluruh resource yang menjadi tanggung jawab Phase 4 telah diprovision dan masing-masing layer telah divalidasi secara independen.

32.1 Kubernetes, Expected → TYPE = NodePort, PORT = 80:30353/TCP

```
kubectl get svc enterprise-web -n enterprise-app
```

32.2 Kubernetes Data Plane, Expected → HTTP/1.1 200 OK

```
curl -v "http://${NODE_IP}:${NODE_PORT}"
```

32.3 Target Group, Expected → enterprise-target-group, HTTP, 30353, instance

```
aws --endpoint-url="$ENDPOINT" \
```

```

    elbv2 describe-target-groups \
    --target-group-arns "$TG_ARN"

# 32.4 Target Registration, Expected → Target = NODE_IP:30353. Health state tergantung kemampuan data plane Floci.

aws --endpoint-url="$ENDPOINT" \
    elbv2 describe-target-health \
    --target-group-arn "$TG_ARN"

# 32.5 ALB, Expected → enterprise-alb, internet-facing, application, active

aws --endpoint-url="$ENDPOINT" \
    elbv2 describe-load-balancers \
    --load-balancer-arns "$ALB_ARN"

# 32.6 ALB Listener, Expected → HTTP :80 dan Forward → enterprise-target-group

aws --endpoint-url="$ENDPOINT" \
    elbv2 describe-listeners \
    --load-balancer-arn "$ALB_ARN"

# 32.7 WAF, Expected → Enterprise-WebACL

aws --endpoint-url="$ENDPOINT" \
    wafv2 get-web-acl-for-resource \
    --resource-arn "$ALB_ARN"

# 32.8 WAF Enforcement, Expected AWS production behavior → HTTP 403 sedangkan Actual Floci behavior → HTTP 200

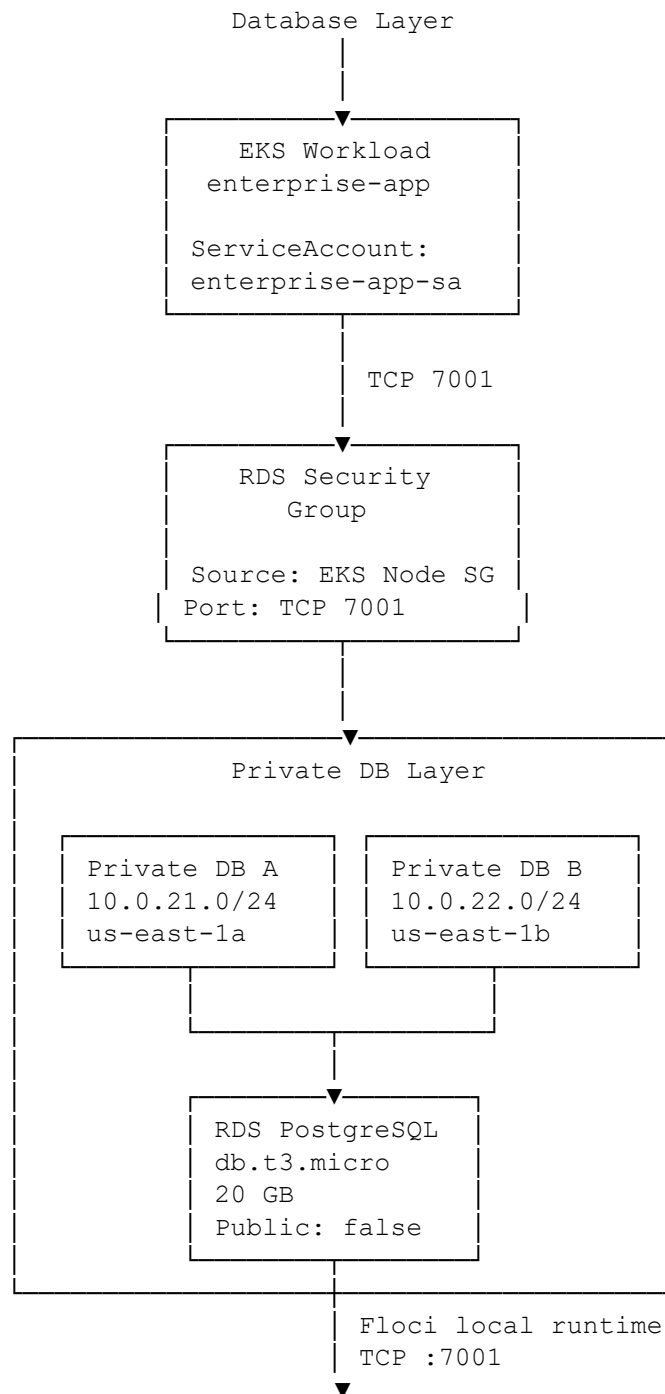
curl -I \
    -H "X-Forwarded-For: 1.2.3.4" \
    http://localhost:4566/

```

Phase 5 — Database

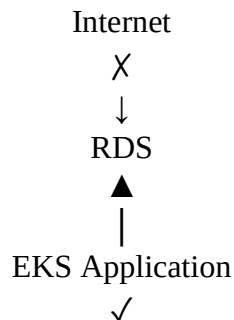
Phase ini digunakan untuk membangun database layer menggunakan Amazon RDS PostgreSQL pada private DB subnets. Database dikonfigurasi agar tidak publicly accessible dan hanya menerima koneksi PostgreSQL dari EKS workload melalui database security group. Selain melakukan validasi terhadap RDS control plane, phase ini juga memverifikasi konektivitas aktual menuju PostgreSQL data plane yang disediakan oleh Floci.

Architecture:



172.17.0.2:7001

Security principle:



Component	Validation	Expected	Status
Private DB Subnet A	EC2 API	10.0.21.0/24 / us-east-1a	☒
Private DB Subnet B	EC2 API	10.0.22.0/24 / us-east-1b	☒
DB Subnet Group	RDS API	2 subnets / 2 AZ	☒
RDS PostgreSQL	RDS API	postgres / db.t3.micro	☒
Storage	RDS API	20 GB	☒
Public Accessibility	RDS API	FALSE	☒
RDS Security Group	EC2 API	TCP 7001 from EKS SG	☒
PostgreSQL Endpoint	RDS API	172.17.0.2:7001	☒
PostgreSQL Data Plane	TCP test	172.17.0.2:7001 reachable	☒

Step 33. Configure Private RDS DB Subnet Group

Membuat logical grouping dari subnet yang diperuntukkan bagi RDS. Pada AWS production, DB Subnet Group umumnya menggunakan subnet dari minimal dua Availability Zone agar database dapat mendukung deployment Multi-AZ dan failover architecture. Karena project kita sudah memiliki:

Private DB A → 10.0.21.0/24 → us-east-1a

Private DB B → 10.0.22.0/24 → us-east-1b

kita gunakan keduanya.

33.1 Pastikan kedua subnet tersedia, Expected → DB Subnet A = subnet-xxxxxxx, DB Subnet B = subnet-yyyyyyyyy

```
export PRIVATE_DB_SUBNET_ID_A=$(aws \
  --endpoint-url="$ENDPOINT" \
  ec2 describe-subnets \
  --filters "Name=tag:Name,Values=private-db-subnet-a" \
  --query 'Subnets[0].SubnetId' \
  --output text)
export PRIVATE_DB_SUBNET_ID_B=$(aws \
```

```

--endpoint-url="$ENDPOINT" \
ec2 describe-subnets \
--filters "Name=tag:Name,Values=private-db-subnet-b" \
--query 'Subnets[0].SubnetId' \
--output text)
echo "DB Subnet A = $PRIVATE_DB_SUBNET_ID_A"
echo "DB Subnet B = $PRIVATE_DB_SUBNET_ID_B"

# 33.2 Validate Availability Zone
aws --endpoint-url="$ENDPOINT" \
    ec2 describe-subnets \
    --subnet-ids \
        "$PRIVATE_DB_SUBNET_ID_A" \
        "$PRIVATE_DB_SUBNET_ID_B" \
    --query 'Subnets[].[ID:SubnetId,AZ:AvailabilityZone,CIDR:CidrBlock]' \
    --output table

# 33.3 Create DB Subnet Group, Parameter → --db-subnet-group-name adalah
nama logical grouping untuk subnet database. --db-subnet-group-
description adalah deskripsi resource. --subnet-ids adalah subnet tempat
RDS diperbolehkan ditempatkan. Dua subnet digunakan untuk
merepresentasikan deployment pada dua Availability Zone.
aws --endpoint-url="$ENDPOINT" \
    rds create-db-subnet-group \
    --db-subnet-group-name enterprise-db-subnet-group \
    --db-subnet-group-description "Enterprise DB Subnet Group" \
    --subnet-ids \
        "$PRIVATE_DB_SUBNET_ID_A" \
        "$PRIVATE_DB_SUBNET_ID_B"

# 33.4 Validasi DB Subnet Group, Expected → Response menunjukkan
DBSubnetGroupName = enterprise-db-subnet-group. dengan dua subnet yaitu
subnet-xxxx → us-east-1a dan subnet-yyyy → us-east-1b
aws --endpoint-url="$ENDPOINT" \
    rds describe-db-subnet-groups \
    --db-subnet-group-name enterprise-db-subnet-group

```

Step 34. Enforce RDS Network Access Control

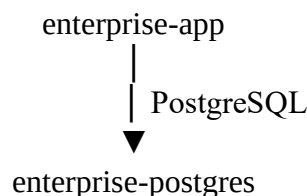
Memastikan database hanya menerima koneksi dari application layer. Security Group yang sudah dibuat pada Phase 1: SG_RDS memiliki rule: TCP 7001 dan Source = SG_EKS_NODE. Jadi kita tidak membuka: 0.0.0.0/0 → TCP 7001 karena itu akan mengekspos PostgreSQL secara langsung ke Internet.

34.1 Validate RDS Security Group, Expected → Harus terdapat rule:
Protocol : tcp, Port : 7001, Source : SG_EKS_NODE_ID

```
aws --endpoint-url="$ENDPOINT" \  
    ec2 describe-security-groups \  
    --group-ids "$SG_RDS_ID" \  
    --query 'SecurityGroups[0].IpPermissions'
```

Step 35. Deploy Private RDS PostgreSQL Instance

Membuat database PostgreSQL sebagai persistent data layer untuk enterprise application. Logical architecture:



35.1 Set Database Credentials Untuk sementara kita pake yg ada dibawah ini. Security note: Ini hanya digunakan sebagai bootstrap lab. Credential database akan dipindahkan ke AWS Secrets Manager + KMS pada phase security/data berikutnya. Jangan gunakan password ini untuk production.

```
export DB_USERNAME="postgres"  
export DB_PASSWORD="Password123!"
```

35.2 Create RDS Instance, Parameter → --db-instance-identifier → Identifier database instance. --engine postgres → Menggunakan PostgreSQL. --db-instance-class db.t3.micro → Instance class kecil untuk lab. --allocated-storage 20 → Storage sebesar 20 GB. --master-username → Username administrator database. --master-user-password → Password administrator. --vpc-security-group-ids → Security Group yang mengontrol network access. --db-subnet-group-name → Menentukan private DB subnets. --no-publicly-accessible → Mencegah database mendapatkan public accessibility. Parameter paling penting dari sisi security adalah --no-publicly-accessible karena database memang harus berada di private network boundary.

```
aws --endpoint-url="$ENDPOINT" \  
    rds create-db-instance \  
    --db-instance-identifier enterprise-postgres \  
    --engine postgres \  
    --db-instance-class db.t3.micro \  
    --allocated-storage 20
```

```

--allocated-storage 20 \
--master-username "$DB_USERNAME" \
--master-user-password "$DB_PASSWORD" \
--vpc-security-group-ids "$SG_RDS_ID" \
--db-subnet-group-name enterprise-db-subnet-group \
--no-publicly-accessible

```

Step 36. Validate RDS Control-Plane Configuration

Step ini digunakan untuk memastikan RDS instance berhasil dibuat dengan engine PostgreSQL, instance class, storage, DB subnet group, security group, dan konfigurasi public accessibility yang sesuai.

36.1 Describe Database, Expected → Minimal: DBInstanceIdentifier = enterprise-postgres, Engine = postgres, DBInstanceClass = db.t3.micro, AllocatedStorage = 20, PubliclyAccessible = false

```

aws --endpoint-url="$ENDPOINT" \
  rds describe-db-instances \
  --db-instance-identifier enterprise-postgres

```

36.2 Validate Subnet Group, Expected → enterprise-db-subnet-group dan VPC harus sama dengan: enterprise-vpc

```

aws --endpoint-url="$ENDPOINT" \
  rds describe-db-instances \
  --db-instance-identifier enterprise-postgres \
  --query
'DBInstances[0].DBSubnetGroup.{Name:DBSubnetGroupName,Status:SubnetGroupStatus,VPC:VpcId}' \
  --output table

```

36.3 Validate Security Group, Expected → SG_RDS_ID

```

aws --endpoint-url="$ENDPOINT" \
  rds describe-db-instances \
  --db-instance-identifier enterprise-postgres \
  --query 'DBInstances[0].VpcSecurityGroups'

```

Step 37. Validate RDS Network Isolation

Step ini digunakan untuk memastikan RDS PostgreSQL tidak publicly accessible dan endpoint database hanya berada pada jaringan private yang telah ditentukan.

37.1 Check Public Accessibility, Expected → False

```
aws --endpoint-url="$ENDPOINT" \
  rds describe-db-instances \
  --db-instance-identifier enterprise-postgres \
  --query 'DBInstances[0].PubliclyAccessible' \
  --output text
```

37.2 Check Database Endpoint, Expected Jika Floci mengimplementasikan endpoint: Address = ... dan Port = 7001

```
aws --endpoint-url="$ENDPOINT" \
  rds describe-db-instances \
  --db-instance-identifier enterprise-postgres \
  --query 'DBInstances[0].Endpoint'
```

Step 38. Validate PostgreSQL Data-Plane Connectivity

Step ini digunakan untuk memverifikasi bahwa RDS PostgreSQL tidak hanya berhasil dibuat secara control plane, tetapi juga menyediakan PostgreSQL data plane yang dapat menerima koneksi TCP melalui endpoint dan port yang diberikan oleh Floci.

38.1 Check RDS Status, Expected → available

```
aws --endpoint-url="$ENDPOINT" \
  rds describe-db-instances \
  --db-instance-identifier enterprise-postgres \
  --query 'DBInstances[0].DBInstanceStatus' \
  --output text
```

38.2 Ambil endpoint dibawah ini

```
export DB_ENDPOINT=$(
  aws --endpoint-url="$ENDPOINT" \
    rds describe-db-instances \
    --db-instance-identifier enterprise-postgres \
    --query 'DBInstances[0].Endpoint.Address' \
    --output text
)

echo "DB_ENDPOINT=$DB_ENDPOINT"
```

38.3 Kemudian dibawah ini, Expected pada environment dengan floci RDS data plane → Connection to ... 7001 port [tcp/postgresql] succeeded

```
nc -zv "$DB_ENDPOINT" 7001
```

Step 39. Final Phase 5 Validation

Melakukan final validation terhadap seluruh database architecture.

**# 39.1 DB Subnet Group, Expected → private-db-subnet-a untuk us-east-1a
sedangkan private-db-subnet-b untuk us-east-1b**

```
aws --endpoint-url="$ENDPOINT" \  
    rds describe-db-subnet-groups \  
    --db-subnet-group-name enterprise-db-subnet-group
```

**# 39.2 RDS Instance, Expected → Identifier = enterprise-postgres, Engine
= postgres, Storage = 20 GB, PublicAccessible = false**

```
aws --endpoint-url="$ENDPOINT" \  
    rds describe-db-instances \  
    --db-instance-identifier enterprise-postgres
```

**# 39.3 Security Group, Expected → TCP 7001, Source = EKS Node Security
Group**

```
aws --endpoint-url="$ENDPOINT" \  
    ec2 describe-security-groups \  
    --group-ids "$SG_RDS_ID"
```

39.4 Data Plane, Expected → Connection succeeded

```
nc -zv "$DB_ENDPOINT" 7001
```

Phase 6 — Storage & Object Security

Phase ini digunakan untuk menyediakan Amazon S3 sebagai object storage layer untuk enterprise application. Bucket dikonfigurasi sebagai private storage, digunakan untuk menyimpan application artifacts/data, serta dilengkapi versioning dan lifecycle management untuk mendukung data protection, recovery, dan storage governance.

Component	Validation	Expected	Status
S3 Bucket	S3 API	enterprise-storage exists	☒
Bucket Region	S3 API	us-east-1	☒
Object Upload	S3 data plane	Object uploaded successfully	☒
Object Download	S3 data plane	Object content retrieved	☒
Object Deletion	S3 data plane	Object deleted	☒
Versioning	S3 API	Enabled	☒
Object Versioning	S3 data plane	Multiple VersionId values	☒
Lifecycle Policy	S3 API	DeleteOldLogs enabled	☒
Public Access Block	S3 API	All four controls enabled	☒

Step 40. Create Private S3 Storage

Step ini digunakan untuk membuat S3 bucket sebagai centralized object storage untuk enterprise application. Bucket akan digunakan sebagai storage layer dan tidak dirancang untuk akses publik.

40.1 Create S3 Bucket, Parameter → s3 mb untuk membuat bucket baru. s3://enterprise-storage untuk nama bucket. --region untuk menentukan AWS Region yang digunakan oleh environment. Expected output → make_bucket: enterprise-storage

```
aws --endpoint-url="$ENDPOINT" \  
    s3 mb s3://enterprise-storage \  
    --region "$AWS_DEFAULT_REGION"
```

40.2 – Validate Bucket, Expected output → 2026-09-13 enterprise-storage

```
aws --endpoint-url="$ENDPOINT" \  
    s3 ls
```

40.3 Validate Bucket Location, Expected bucket location sesuai dengan region lab, atau response yang ekuivalen dengan konfigurasi us-east-1, kalo keluarnya null itu emang bawaan awsnya karena null = us-east-1.

```
aws --endpoint-url="$ENDPOINT" \  
    s3api get-bucket-location \  
    --bucket enterprise-storage
```

Step 41. Validate S3 Object Operations

Step ini digunakan untuk memverifikasi bahwa bucket dapat melakukan operasi dasar object storage seperti upload, listing, download, dan deletion.

41.1 Prepare Test Object

```
echo "Enterprise AWS Security Platform - S3 Test Object" > test-object.txt
```

41.2. Upload Object, Expected → upload: ./test-object.txt to s3://enterprise-storage/test-object.txt

```
aws --endpoint-url="$ENDPOINT" \  
    s3 cp test-object.txt \  
    s3://enterprise-storage/test-object.txt
```

41.3 – List Bucket Contents, Expected → test-object.txt

```
aws --endpoint-url="$ENDPOINT" \  
    s3 ls s3://enterprise-storage
```

41.4. Download Object

```
aws --endpoint-url="$ENDPOINT" \  
    s3 cp \  
    s3://enterprise-storage/test-object.txt \  
    downloaded-test-object.txt
```

41.6 Validate, Expected → Enterprise AWS Security Platform - S3 Test Object

```
cat downloaded-test-object.txt
```

41.7 Delete Test Object, Expected → delete: s3://enterprise-storage/test-object.txt

```
aws --endpoint-url="$ENDPOINT" \  
    s3 rm s3://enterprise-storage/test-object.txt
```

Step 42. Enable S3 Versioning

Step ini digunakan untuk mengaktifkan object versioning sehingga perubahan atau penghapusan object dapat direpresentasikan sebagai versi yang berbeda dan memungkinkan recovery terhadap versi sebelumnya.

42.1 Enable Versioning

```
aws --endpoint-url="$ENDPOINT" \  
    s3api put-bucket-versioning \  
    --bucket enterprise-storage \  
    --versioning-configuration {  
        "Status": "Enabled"  
    }
```

```
--versioning-configuration Status=Enabled

# 42.2 Validate Versioning, Expected → { "Status": "Enabled" }

aws --endpoint-url="$ENDPOINT" \
    s3api get-bucket-versioning \
    --bucket enterprise-storage
```

Step 43. Validate Object Versioning

Step ini digunakan untuk membuktikan bahwa S3 tidak hanya memiliki konfigurasi versioning pada control plane, tetapi benar-benar menghasilkan beberapa object versions.

43.1 Create First Object Version

```
echo "Version 1" > versioned-object.txt

aws --endpoint-url="$ENDPOINT" \
    s3 cp versioned-object.txt \
    s3://enterprise-storage/versioned-object.txt
```

43.2 Create Second Object Version

```
echo "Version 2" > versioned-object.txt

aws --endpoint-url="$ENDPOINT" \
    s3 cp versioned-object.txt \
    s3://enterprise-storage/versioned-object.txt
```

43.3 List Object Versions, Expected → Terdapat minimal dua VersionId untuk versioned-object.txt. Ini menjadi bukti bahwa versioning bekerja pada object data plane.

```
aws --endpoint-url="$ENDPOINT" \
    s3api list-object-versions \
    --bucket enterprise-storage \
    --prefix versioned-object.txt
```

Step 44. Configure S3 Lifecycle Management

Step ini digunakan untuk menerapkan lifecycle policy sehingga object tertentu dapat dikelola secara otomatis berdasarkan umur atau prefix. Untuk lab ini kita gunakan prefix logs/.

44.1 Create Test Log Object

```
echo "Enterprise application log" > application.log

aws --endpoint-url="$ENDPOINT" \
    s3 cp application.log \
```

```
s3://enterprise-storage/logs/application.log
```

44.2 Configure Lifecycle Rule, Parameter → ID untuk identifier lifecycle rule. Status=Enabled untuk mengaktifkan rule. Prefix=logs/ adalah rule hanya berlaku untuk object di bawah prefix tersebut. Expiration.Days=30 adalah object akan ditandai untuk expiration setelah 30 hari.

```
aws --endpoint-url="$ENDPOINT" \  
    s3api put-bucket-lifecycle-configuration \  
    --bucket enterprise-storage \  
    --lifecycle-configuration '{  
        "Rules": [  
            {  
                "ID": "DeleteOldLogs",  
                "Status": "Enabled",  
                "Filter": {  
                    "Prefix": "logs/"  
                },  
                "Expiration": {  
                    "Days": 30  
                }  
            }  
        ]  
    }'
```

44.3 Validate Lifecycle Configuration, Expected → Terdapat rule yaitu ID: DeleteOldLogs, Status: Enabled, Prefix: logs/, Expiration: 30 days. Catatan: validasi konfigurasi lifecycle tidak sama dengan menunggu 30 hari untuk membuktikan deletion. Untuk lab ini, kita memvalidasi bahwa lifecycle policy berhasil dipasang pada bucket.

```
aws --endpoint-url="$ENDPOINT" \  
    s3api get-bucket-lifecycle-configuration \  
    --bucket enterprise-storage
```

Step 45. Validate S3 Public Access Security

Step ini digunakan untuk memastikan bucket storage tidak dikonfigurasi sebagai public object storage.

45.1

```
aws --endpoint-url="$ENDPOINT" \  
    s3api get-public-access
```

```

    s3api put-public-access-block \
    --bucket enterprise-storage \
    --public-access-block-configuration \
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true

# 45.1 Kemudian dibawah ketik command di bawah ini. Expected → BlockPublicAcls: true, IgnorePublicAcls: true, BlockPublicPolicy: true, RestrictPublicBuckets: true

aws --endpoint-url="$ENDPOINT" \
    s3api get-public-access-block \
    --bucket enterprise-storage

```

Step 46. Final Validation of S3 Storage Baseline

46.1 Pastikan environment:

```

export ENDPOINT="http://localhost:4566"
export AWS_DEFAULT_REGION="us-east-1"
export AWS_ACCOUNT_ID="000000000000"
export S3_BUCKET="enterprise-storage"
export S3_KEY="logs/test-object.txt"

```

46.2 Validasi bucket

```

aws --endpoint-url="$ENDPOINT" \
    s3api head-bucket \
    --bucket "$S3_BUCKET"

```

46.3 Validasi versioning, Expected → { "Status": "Enabled" }

```

aws --endpoint-url="$ENDPOINT" \
    s3api get-bucket-versioning \
    --bucket "$S3_BUCKET"

```

46.4 Validasi lifecycle, Pastikan rule DeleteOldLogs masih ada.

```

aws --endpoint-url="$ENDPOINT" \
    s3api get-bucket-lifecycle-configuration \
    --bucket "$S3_BUCKET"

```

Step 47. Prepare Kubernetes Workload for S3 Access

47.1 Pastikan namespace

```

kubectl get namespace enterprise-app

```

```

# 47.2 Pastikan ServiceAccount, Pastikan annotation →
eks.amazonaws.com/role-arn:
arn:aws:iam::000000000000:role/EnterpriseAppIRSARole

kubectl get serviceaccount \

enterprise-app-sa \

-n enterprise-app \

-o yaml

# 47.3 Validasi, Expected →
arn:aws:iam::000000000000:role/EnterpriseAppIRSARole

kubectl get serviceaccount enterprise-app-sa \

-n enterprise-app \

-o jsonpath='{.metadata.annotations.eks\.amazonaws\.com/role-arn}';
echo

```

Step 48. Deploy S3 IRSA Test Pod

48.1 Gunakan projected ServiceAccount token secara explicit

```

mkdir ~/phase6

cd ~/phase 6

cat <<'EOF' > s3-irsa-test.yaml
apiVersion: v1
kind: Pod
metadata:
  name: s3-irsa-test
  namespace: enterprise-app
spec:
  serviceAccountName: enterprise-app-sa
  restartPolicy: Never
  containers:
    - name: aws-cli
      image: amazon/aws-cli:latest
      command:
        - /bin/sh
        - -c
        - |
          sleep 3600
      volumeMounts:
        - name: aws-iam-token
          mountPath: /var/run/secrets/eks.amazonaws.com/serviceaccount
          readOnly: true
  volumes:
    - name: aws-iam-token
      projected:
        sources:
          - serviceAccountToken:
              audience: sts.amazonaws.com
              expirationSeconds: 3600

```

```
path: token
EOF
```

48.2 Apply

```
kubectl apply -f s3-irsa-test.yaml
```

48.3 Validasi, Expected → s3-irsa-test 1/1 Running

```
kubectl get pod s3-irsa-test -n enterprise-app
```

Step 49. Validate Projected ServiceAccount Token

49.1 Cek mount, Expected ada token

```
kubectl exec -n enterprise-app s3-irsa-test -- \
  ls -la /var/run/secrets/eks.amazonaws.com/serviceaccount/
```

49.2 Kemudian ketik command di bawah ini, Expected → IRSA TOKEN EXISTS

```
kubectl exec -n enterprise-app s3-irsa-test -- \
  sh -c '
    test -s /var/run/secrets/eks.amazonaws.com/serviceaccount/token &&
    echo "IRSA TOKEN EXISTS"
  '
```

49.3 Validasi token bisa dibaca, Expected panjang token > 0.

```
kubectl exec -n enterprise-app s3-irsa-test -- \
  sh -c '
    TOKEN=$(cat
/var/run/secrets/eks.amazonaws.com/serviceaccount/token)
    echo "Token length: ${#TOKEN}"
  '
```

Step 50. Validate JWT Claims

Kita validasi claim token tanpa memasukkan token mentah ke output. Pastikan claim penting seperti "aud": ["sts.amazonaws.com"] dan "sub": "system:serviceaccount:enterprise-app:enterprise-app-sa" terlihat. Untuk lab Floci, issuer Kubernetes dapat berbeda dari issuer AWS-native EKS. Jadi jangan mengubah trust policy Phase 3 hanya untuk memaksa issuer ini sama.

```
kubectl exec -n enterprise-app s3-irsa-test -- \
  sh -c '
    TOKEN=$(cat
/var/run/secrets/eks.amazonaws.com/serviceaccount/token)
    PAYLOAD=$(echo "$TOKEN" | cut -d "." -f2)
```

```

case $(( ${#PAYLOAD} % 4 )) in
    2) PAYLOAD="${PAYLOAD}==" ;;
    3) PAYLOAD="${PAYLOAD}=" ;;
esac

echo "$PAYLOAD" | tr "_-" "/" | base64 -d 2>/dev/null

echo
,

```

Step 51. Validate STS AssumeRoleWithWebIdentity

Ini bagian penting. Jangan export `AWS_ROLE_ARN` atau `AWS_WEB_IDENTITY_TOKEN_FILE`. Kita lakukan explicit STS exchange. Expected → `arn:aws:sts::000000000000:assumed-role/EnterpriseAppIRSARole/s3-identity-test`. Kenapa cuma output ARN? Karena response STS juga mengandung `AccessKeyId`, `SecretAccessKey`, `SessionToken`. Kita tidak perlu menampilkan credential tersebut. Kalau command ini berhasil, berarti Kubernetes ServiceAccount JWT → STS `AssumeRoleWithWebIdentity` → `EnterpriseAppIRSARole` → Temporary IAM identity berhasil.

```

kubectl exec -n enterprise-app s3-irsa-test -- \

sh -c '

TOKEN=$(cat
/var/run/secrets/eks.amazonaws.com/serviceaccount/token)

aws --endpoint-url="http://172.17.0.1:4566" \

sts assume-role-with-web-identity \

--role-arn arn:aws:iam::000000000000:role/EnterpriseAppIRSARole \

--role-session-name s3-identity-test \

--web-identity-token "$TOKEN" \

--query "AssumedRoleUser.Arn" \

--output text

,

```

Step 52. Validate Temporary IAM Identity

Sekarang kita ambil satu response STS, lalu gunakan ketiga credential dari response yang sama. Expected:

```

{
  "UserId": "...",
  "Account": "000000000000",

```

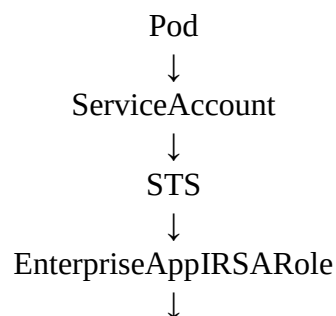
```
"Arn":      "arn:aws:sts::000000000000:assumed-role/EnterpriseAppIRSARole/s3-identity-test"
}
```

Ini membuktikan credential temporary hasil STS bisa dipakai untuk melakukan AWS API call. Catatan: credential tidak ditampilkan di terminal.

```
kubectl exec -n enterprise-app s3-irsa-test -- \
  sh -c '
    export AWS_DEFAULT_REGION=us-east-1
    TOKEN=$(cat
/var/run/secrets/eks.amazonaws.com/serviceaccount/token)
    CREDS=$(aws --endpoint-url="http://172.17.0.1:4566" \
      sts assume-role-with-web-identity \
        --role-arn arn:aws:iam::000000000000:role/EnterpriseAppIRSARole \
        --role-session-name s3-identity-test \
        --web-identity-token "$TOKEN" \
        --query "Credentials.[AccessKeyId,SecretAccessKey,SessionToken]"
    \
    --output text)
    set -- $CREDS
    export AWS_ACCESS_KEY_ID="$1"
    export AWS_SECRET_ACCESS_KEY="$2"
    export AWS_SESSION_TOKEN="$3"
    aws --endpoint-url="http://172.17.0.1:4566" \
      sts get-caller-identity
  '
```

Step 53. Validate Pod-to-S3 Read Access

Sekarang kita test permission S3. Gunakan pola credential yang sama. Expected → enterprise-storage. Kalau bucket muncul, berarti:



S3

sudah berhasil.

```
kubectl exec -n enterprise-app s3-irsa-test -- \
  sh -c '
    TOKEN=$(cat
/var/run/secrets/eks.amazonaws.com/serviceaccount/token)

    CREDS=$(aws --endpoint-url="http://172.17.0.1:4566" \
      sts assume-role-with-web-identity \
      --role-arn arn:aws:iam::000000000000:role/EnterpriseAppIRSARole \
      --role-session-name s3-read-test \
      --web-identity-token "$TOKEN" \
      --query "Credentials.[AccessKeyId,SecretAccessKey,SessionToken]"
\
      --output text)

    set -- $CREDS

    export AWS_ACCESS_KEY_ID="$1"
    export AWS_SECRET_ACCESS_KEY="$2"
    export AWS_SESSION_TOKEN="$3"

    aws --endpoint-url="http://172.17.0.1:4566" \
      s3api list-buckets \
      --query "Buckets[].Name" \
      --output text
  '
```

Step 54. Validate Pod-to-S3 Object Read

54.1 Buat object test terlebih dahulu dari host

```
echo "S3 access test from enterprise application" > /tmp/s3-test.txt
```

54.2 Upload

```
aws --endpoint-url="$ENDPOINT" \
  s3 cp \
```

```

/tmp/s3-test.txt \
"s3://$S3_BUCKET/$S3_KEY"

# 54.2 Kemudian baca dari Pod menggunakan temporary credentials, Expected
→ S3 access test from enterprise application

kubectl exec -n enterprise-app s3-irsa-test -- \
  sh -c '
    TOKEN=$(cat
/var/run/secrets/eks.amazonaws.com/serviceaccount/token)

    CRED=$(aws --endpoint-url="http://172.17.0.1:4566" \
      sts assume-role-with-web-identity \
      --role-arn arn:aws:iam::000000000000:role/EnterpriseAppIRSARole \
      --role-session-name s3-object-read-test \
      --web-identity-token "$TOKEN" \
      --query "Credentials.[AccessKeyId,SecretAccessKey,SessionToken]"
\
      --output text)

    set -- $CRED$
    export AWS_ACCESS_KEY_ID="$1"
    export AWS_SECRET_ACCESS_KEY="$2"
    export AWS_SESSION_TOKEN="$3"
    aws --endpoint-url="http://172.17.0.1:4566" \
      s3 cp \
      s3://enterprise-storage/logs/test-object.txt \
      /tmp/downloaded.txt
    cat /tmp/downloaded.txt
  '

```

Step 55. Validate Pod-to-S3 Write Access

Sekarang workload membuat object sendiri. Expected → S3 WRITE SUCCESS

```

kubectl exec -n enterprise-app s3-irsa-test -- \
  sh -c '
    TOKEN=$(cat
/var/run/secrets/eks.amazonaws.com/serviceaccount/token)

    CRED=$(aws --endpoint-url="http://172.17.0.1:4566" \

```

```

    sts assume-role-with-web-identity \
    --role-arn arn:aws:iam::000000000000:role/EnterpriseAppIRSARole \
    --role-session-name s3-write-test \
    --web-identity-token "$TOKEN" \
    --query "Credentials.[AccessKeyId,SecretAccessKey,SessionToken]"
\

    --output text)

set -- $CREDS

export AWS_ACCESS_KEY_ID="$1"
export AWS_SECRET_ACCESS_KEY="$2"
export AWS_SESSION_TOKEN="$3"

echo "Object created by Kubernetes workload" > /tmp/workload.txt

aws --endpoint-url="http://172.17.0.1:4566" \
    s3 cp \
    /tmp/workload.txt \
    s3://enterprise-storage/logs/from-kubernetes.txt

echo "S3 WRITE SUCCESS"

```

Step 56. Validate Object Created by Kubernetes Workload

56.1 Sekarang validasi dari host, bukan dari Pod.

```

aws --endpoint-url="$ENDPOINT" \
    s3api head-object \
    --bucket enterprise-storage \
    --key logs/from-kubernetes.txt

```

56.2 Kemudian dibawah ini, Expected → Object created by Kubernetes workload. Ini penting karena membuktikan object benar-benar sampai ke S3 layer.

```

aws --endpoint-url="$ENDPOINT" \

```

```
s3 cp \
s3://enterprise-storage/logs/from-kubernetes.txt \
-
```

Step 57. Validate Workload Identity Isolation

57.1 Buat Pod kedua yang tidak menggunakan enterprise-app-sa

```
cat <<'EOF' > unauthorized-s3-test.yaml
apiVersion: v1
kind: Pod
metadata:
  name: unauthorized-s3-test
  namespace: enterprise-app
spec:
  restartPolicy: Never
  containers:
    - name: aws-cli
      image: amazon/aws-cli:latest
      command:
        - /bin/sh
        - -c
        - |
          sleep 3600
EOF
```

57.2 Apply

```
kubectl apply -f unauthorized-s3-test.yaml
```

57.3 Cek token, Expected → PASS: IRSA TOKEN NOT PRESENT. Perhatikan, ini membuktikan identity isolation, bukan otomatis membuktikan seluruh S3 authorization boundary.

```
kubectl exec -n enterprise-app unauthorized-s3-test -- \
  sh -c '
    if [ -f /var/run/secrets/eks.amazonaws.com/serviceaccount/token ];
  then
    echo "UNEXPECTED: IRSA TOKEN EXISTS"
  else
    echo "PASS: IRSA TOKEN NOT PRESENT"
  fi
  '
```

Step 58. Final Phase 6 Validation

```
echo "===== S3 BUCKET ====="
aws --endpoint-url="$ENDPOINT" \
```

```

s3api head-bucket \
  --bucket enterprise-storage
echo "==== VERSIONING ====="
aws --endpoint-url="$ENDPOINT" \
  s3api get-bucket-versioning \
  --bucket enterprise-storage
echo "==== LIFECYCLE ====="
aws --endpoint-url="$ENDPOINT" \
  s3api get-bucket-lifecycle-configuration \
  --bucket enterprise-storage
echo "==== KUBERNETES SERVICEACCOUNT ====="
kubectl get serviceaccount enterprise-app-sa \
  -n enterprise-app \
  -o jsonpath='{.metadata.annotations.eks\.amazonaws\.com/role-arn}';
echo
echo "==== IRSA POD ====="
kubectl get pod s3-irsa-test \
  -n enterprise-app
echo "==== S3 WORKLOAD OBJECT ====="
aws --endpoint-url="$ENDPOINT" \
  s3api head-object \
  --bucket enterprise-storage \
  --key logs/from-kubernetes.txt
echo "==== PHASE 6 VALIDATION COMPLETE ====="

```

Step 59. Cleanup Test Workloads

Setelah semua validasi selesai

59.1 delete pod testing

```

kubectl delete pod \
  s3-irsa-test \
  unauthorized-s3-test \
  -n enterprise-app \
  --ignore-not-found

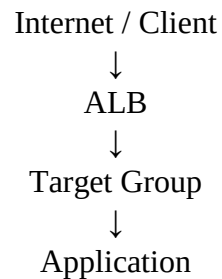
```

59.2 Delete Manifest

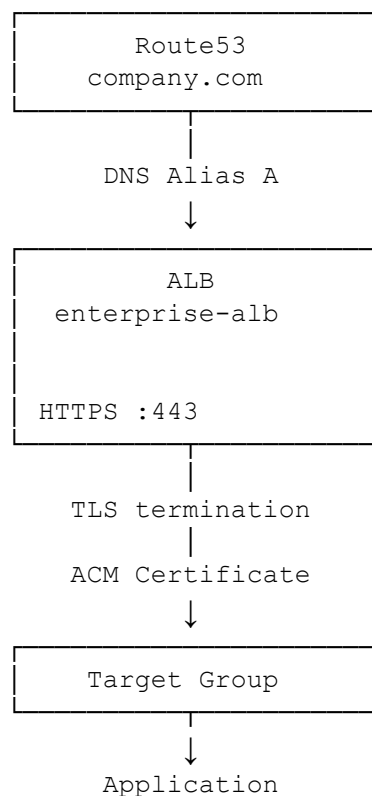
```
rm -f \  
s3-irsa-test.yaml \  
unauthorized-s3-test.yaml \  
/tmp/s3-test.txt
```

Phase 7 — Domain & SSL

Phase 7 menambahkan Domain & TLS/SSL layer di depan application yang sudah dibangun pada Phase 4. Sebelumnya architecture kita kira-kira:



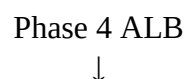
Setelah Phase 7:



Jadi ada 3 komponen utama yang kita hubungkan:

Layer	Komponen	Fungsi
DNS	Route53	company.com diarahkan ke ALB
Certificate	ACM	Menyediakan certificate untuk company.com
TLS ingress	ALB HTTPS :443	Terminate TLS dan forward traffic ke application

Yang penting: Phase 7 tidak membuat ALB baru. Kita menggunakan:



enterprise-alb



Phase 7 menambahkan DNS + ACM + HTTPS

Status Aktual Phase 7

Component	Status	Kenapa
Existing ALB	☑ PASS	enterprise-alb ditemukan
ALB DNS	☑ PASS	DNS name berhasil didapat
ALB Hosted Zone ID	☑ PASS	Z35SXDOTRQ7X7K tersedia
Route53 Hosted Zone	☑ PASS	Hosted zone company.com berhasil dibuat
Route53 A Alias	☑ PASS	company.com menunjuk ke ALB
ACM Certificate	☑ PASS*	Certificate company.com berstatus ISSUED di Floci
ACM DNS validation lifecycle	⚠ NOT VERIFIED	ValidationMethod = None
HTTPS Listener :443	☑ PASS	Listener HTTPS berhasil dibuat
Certificate binding	☑ PASS	HTTPS listener menggunakan certificate ACM
Forward → Target Group	☑ PASS	Listener forward ke enterprise-target-group
ALB DNS resolution	☑ PASS	hostname resolve ke ::1
External HTTPS TCP	✗ NOT VERIFIED	::1:443 connection refused
TLS handshake	✗ NOT VERIFIED	Tidak ada server yang menerima koneksi
HTTPS application response	✗ NOT VERIFIED	Belum mencapai application

* ISSUED di sini berarti status yang direpresentasikan oleh emulator Floci, bukan bukti bahwa AWS production ACM DNS-validation lifecycle benar-benar terjadi.

Step 60. Prepare Existing ALB Information

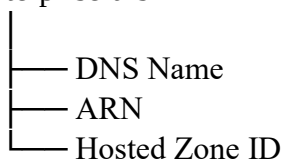
Step ini memastikan Phase 7 menggunakan ALB yang benar-benar dibuat pada Phase 4. Kita tidak ingin Phase 7 diam-diam membuat ALB kedua.

Architecture:

Phase 4



enterprise-alb



Phase 7

60.1 Set Environment, ENDPOINT → endpoint API Floci. AWS_DEFAULT_REGION → region lab. ALB_NAME → nama ALB existing dari Phase 4.

```
export ENDPOINT="http://localhost:4566"
```

```
export AWS_DEFAULT_REGION="us-east-1"
```

```
export ALB_NAME="enterprise-alb"
```

60.2 Retrieve ALB ARN, Expected → Kurang lebih arn:aws:elasticloadbalancing:us-east-1:000000000000:loadbalancer/app/enterprise-alb/.... Artinya ALB memang exists di Floci dan ARN-nya berhasil ditemukan.

```
ALB_ARN=$(  
  aws --endpoint-url="$ENDPOINT" \  
    elbv2 describe-load-balancers \  
    --names "$ALB_NAME" \  
    --query 'LoadBalancers[0].LoadBalancerArn' \  
    --output text  
)  
  
echo "$ALB_ARN"
```

60.3 Retrieve ALB DNS Name, Actual → enterprise-alb-9a2b506e31314ec0.elb.localhost.floci.io. Ini akan menjadi target Route53 Alias.

```
ALB_DNS_NAME=$(  
  aws --endpoint-url="$ENDPOINT" \  
    elbv2 describe-load-balancers \  
    --names "$ALB_NAME" \  
    --query 'LoadBalancers[0].DNSName' \  
    --output text  
)  
  
echo "$ALB_DNS_NAME"
```

60.4 Retrieve ALB Hosted Zone ID, Actual → Z35SXDOTRQ7X7K. Hosted Zone ID ini digunakan oleh Route53 ketika membuat ALB Alias.

```
ALB_HOSTED_ZONE_ID=$(  
  aws --endpoint-url="$ENDPOINT" \  
    elbv2 describe-load-balancers \  
    --names "$ALB_NAME" \  
    --query 'LoadBalancers[0].CanonicalHostedZoneId' \  
    --output text  
)
```

```
echo "$ALB_HOSTED_ZONE_ID"
```

```
# 60.5 Final Validation, Expected → ALB ARN: arn:aws:..., ALB DNS:
enterprise-alb-....elb.localhost.floci.io, ALB Hosted Zone ID:
Z35SXDOTRQ7X7K
```

```
echo "ALB ARN: $ALB_ARN"
```

```
echo "ALB DNS: $ALB_DNS_NAME"
```

```
echo "ALB Hosted Zone ID: $ALB_HOSTED_ZONE_ID"
```

Step 61. Create Route53 Hosted Zone

Membuat DNS zone untuk company.com Sehingga kita mempunyai authoritative DNS zone untuk domain tersebut. Architecture:



```
# 61.1 Create Hosted Zone, Expected → Biasanya
/hostedzone/ZXXXXXXXXXXXXX
```

```
HOSTED_ZONE_ID=$(
  aws --endpoint-url="$ENDPOINT" \
    route53 create-hosted-zone \
    --name company.com \
    --caller-reference "enterprise-security-$(date +%s)" \
    --query 'HostedZone.Id' \
    --output text
)
```

```
# 61.2 Normalize Hosted Zone ID, Mengubah /hostedzone/Z123456789 menjadi
Z123456789 Supaya lebih mudah dipakai oleh command AWS CLI berikutnya.
```

```
HOSTED_ZONE_ID="${HOSTED_ZONE_ID##*/}"
```

61.3 Validate

```
echo "Hosted Zone ID: $HOSTED_ZONE_ID"
```

```
# 61.4 Kemudian di bawah ini, Expected → AWS/Floci mengembalikan
informasi HostedZone, Name: company.com.
```

```
aws --endpoint-url="$ENDPOINT" \
  route53 get-hosted-zone \
  --id "$HOSTED_ZONE_ID"
```

Step 62. Validate Route53 Hosted Zone

Memastikan Hosted Zone benar-benar mempunyai DNS records dasar. Expected → Minimal ada NS dan SOA. Contoh actual flocci adalah

company.com. NS

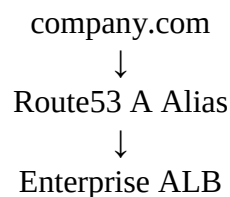
company.com. SOA.

Kenapa NS dan SOA penting? NS: Name Server menentukan authoritative nameserver untuk zone. SOA: Start of Authority menyimpan informasi authority/zone metadata. Hosted Zone berfungsi pada level Route53 API/emulator.

```
aws --endpoint-url="$ENDPOINT" \  
    route53 list-resource-record-sets \  
    --hosted-zone-id "$HOSTED_ZONE_ID"
```

Step 63. Create Alias Record to Existing ALB

Sekarang kita membuat hubungan:



Ini berbeda dari A record biasa karena target-nya adalah AWS resource ALB.

63.1 Create DNS Change Definition, Isi pentingnya → "Name": "company.com" adalah Domain yang dibuat. "Type": "A" adalah IPv4 alias record. "HostedZoneId": "\$ALB_HOSTED_ZONE_ID" adalah Hosted Zone ID ALB. "DNSName": "\$ALB_DNS_NAME" adalah Target ALB. Kenapa UPSERT? UPSERT = create kalau belum ada atau update kalau sudah ada. Jadi command aman dijalankan ulang.

```
mkdir ~/phase7  
cd ~/phase7  
cat <<EOF > record.json  
{  
  "Comment": "Alias company.com to Enterprise ALB",  
  "Changes": [  
    {  
      "Action": "UPSERT",  
      "ResourceRecordSet": {  
        "Name": "company.com",  
        "Type": "A",  
        "AliasTarget": {  
          "HostedZoneId": "$ALB_HOSTED_ZONE_ID",  
          "DNSName": "$ALB_DNS_NAME",  
          "EvaluateTargetHealth": false  
        }  
      }  
    ]  
}
```

```

    }
  }
]
}
EOF

# 63.2 Apply Record, Expected → Ada ChangeInfo

aws --endpoint-url="$ENDPOINT" \
    route53 change-resource-record-sets \
    --hosted-zone-id "$HOSTED_ZONE_ID" \
    --change-batch file://record.json

```

Step 64. Validate Route53 → ALB Association

Memastikan record yang dibuat benar-benar menunjuk ke ALB. Actual floci:

Name: company.com.

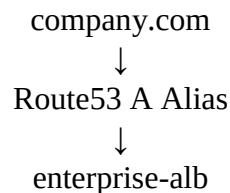
Type: A

AliasTarget:

HostedZoneId: Z35SXDOTRQ7X7K

DNSName: enterprise-alb-9a2b506e31314ec0.elb.localhost.floci.io

Secara control plane:



```

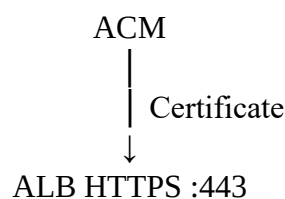
aws --endpoint-url="$ENDPOINT" \
    route53 list-resource-record-sets \
    --hosted-zone-id "$HOSTED_ZONE_ID" \
    --query 'ResourceRecordSets[?Name==`company.com.`]'

```

Step 65. Request ACM Certificate

Membuat certificate untuk company.com. Certificate nantinya dipasang pada ALB HTTPS :443

Architecture:



65.1 Request Certificate

```
CERTIFICATE_ARN=$(  
  aws --endpoint-url="$ENDPOINT" \  
    acm request-certificate \  
    --domain-name company.com \  
    --validation-method DNS \  
    --query 'CertificateArn' \  
    --output text  
)
```

65.2 Save ARN

```
export CERTIFICATE_ARN
```

```
#      65.3      Validate,      Expected      →      arn:aws:acm:us-east-  
1:000000000000:certificate/...
```

```
echo "$CERTIFICATE_ARN"
```

Step 66. Inspect ACM Certificate

Memeriksa metadata certificate yang dibuat.

```
aws --endpoint-url="$ENDPOINT" \  
  acm describe-certificate \  
    --certificate-arn "$CERTIFICATE_ARN"
```

Atau ringkas:

```
aws --endpoint-url="$ENDPOINT" \  
  acm describe-certificate \  
    --certificate-arn "$CERTIFICATE_ARN" \  
    --query 'Certificate.[DomainName,Status,ValidationMethod,Type]' \  
    --output table
```

Actual floci:

```
-----  
|DescribeCertificate|  
+-----+  
| company.com      |  
| ISSUED           |  
| None             |
```

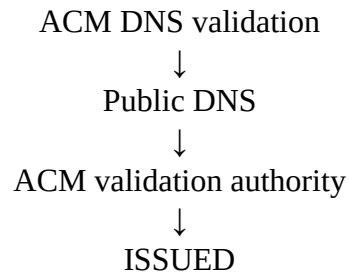
+-----+

Interpretasi

Domain = company.com

Status = ISSUED

Floci merepresentasikan certificate sebagai ISSUED. Tetapi ValidationMethod = None menunjukkan bahwa kita tidak boleh menganggap seluruh lifecycle DNS validation AWS production telah terbukti. Bukan bukti penuh bahwa:



benar-benar terjadi seperti AWS production .

Step 67. Check ACM DNS Validation Records

Mencari apakah Floci memberikan DNS validation record seperti ACM AWS asli. Pada AWS production Kita biasanya mengharapkan informasi seperti DomainName, ValidationDomain, ResourceRecord (Name, Type, Value) Kemudian record tersebut dimasukkan ke Route53. Pada Floci Kalau ResourceRecord tidak tersedia atau lifecycle validation tidak dapat dijalankan alias NOT VERIFIED. Status aktual yaitu NOT VERIFIED — ACM DNS validation lifecycle Bukan berarti certificate object gagal. Yang tidak terbukti adalah full AWS-style DNS validation lifecycle.

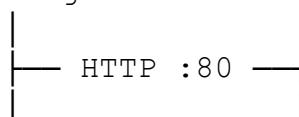
```
aws --endpoint-url="$ENDPOINT" \  
  acm describe-certificate \  
  --certificate-arn "$CERTIFICATE_ARN" \  
  --query 'Certificate.DomainValidationOptions'
```

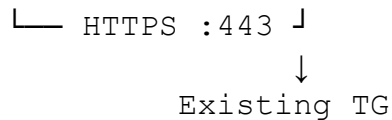
Step 68. Retrieve Existing HTTP Listener

Kita perlu mengetahui target group yang sudah digunakan oleh ALB existing. Jangan membuat target group baru hanya untuk HTTPS.

Architecture:

Existing ALB





68.1 List Listener

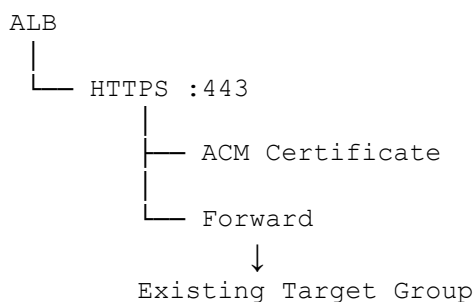
```
aws --endpoint-url="$ENDPOINT" \
  elbv2 describe-listeners \
  --load-balancer-arn "$ALB_ARN"
```

68.2 Retrieve Target Group Automatically, Expected →
arn:aws:elasticloadbalancing:us-east-
1:000000000000:targetgroup/enterprise-target-group/.... Actual:
enterprise-target-group. Kita mempertahankan application backend yang
sudah dibuat pada Phase 4.

```
TG_ARN=$(
  aws --endpoint-url="$ENDPOINT" \
    elbv2 describe-listeners \
    --load-balancer-arn "$ALB_ARN" \
    --query 'Listeners[0].DefaultActions[0].TargetGroupArn' \
    --output text
)
echo "$TG_ARN"
```

Step 69. Create HTTPS Listener

Ini adalah inti TLS layer. Kita membuat:



69.1 Create Listener

```
HTTPS_LISTENER_ARN=$(
  aws --endpoint-url="$ENDPOINT" \
    elbv2 create-listener \
    --load-balancer-arn "$ALB_ARN" \
    --protocol HTTPS \
    --port 443 \
```

```

--certificates "CertificateArn=$CERTIFICATE_ARN" \
--default-actions "Type=forward,TargetGroupArn=$TG_ARN" \
--query 'Listeners[0].ListenerArn' \
--output text
)

# 69.2 Save ARN
export HTTPS_LISTENER_ARN

# 69.3 Validate, Expected → arn:aws:elasticloadbalancing:us-east-1:000000000000:listener/app/enterprise-alb/.... HTTPS listener berhasil dibuat.
echo "$HTTPS_LISTENER_ARN"

```

Step 70. Validate HTTPS Listener Configuration

Memastikan seluruh hubungan:



benar-benar terdaftar pada ALB. Expected → HTTPS 443 arn:...listener/...
arn:...certificate/...

```

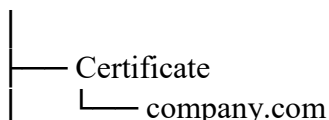
aws --endpoint-url="$ENDPOINT" \
    elbv2 describe-listeners \
    --load-balancer-arn "$ALB_ARN" \
    --query
'Listeners[?Port==`443`].[Protocol,Port,ListenerArn,Certificates[0].CertificateArn]' \
    --output table

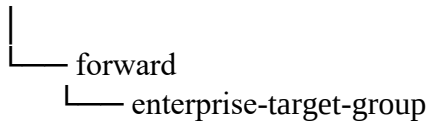
```

Step 71. Validate HTTPS Listener + Target Group

Step ini kita tambahkan untuk memastikan bukan hanya certificate yang terpasang, tetapi listener juga benar-benar forward ke backend. Interpretasi → Kita sudah membuktikan:

HTTPS :443





Ini adalah validasi control-plane yang sangat penting. Actual floci:

```
[
  [
    "HTTPS",
    443,
    "arn:aws:elasticloadbalancing:us-east-1:000000000000:listener/app/enterprise-
alb/9a2b506e31314ec0/e4a1e299b75f4303",
    "arn:aws:acm:us-east-1:000000000000:certificate/2709f3de-12e6-45cd-91be-
68f27b70660c",
    [
      {
        "Type": "forward",
        "TargetGroupArn": "arn:aws:elasticloadbalancing:us-east-
1:000000000000:targetgroup/enterprise-target-group/e4db28e693914456"
      }
    ]
  ]
]
```

71.1 Query Complete HTTPS Listener

```
aws --endpoint-url="$ENDPOINT" \
  elbv2 describe-listeners \
  --load-balancer-arn "$ALB_ARN" \
  --query
'Listeners[?Port==`443`].[Protocol,Port,ListenerArn,Certificates[0].Ce
rtificateArn,DefaultActions]' \
  --output json
```

Step 72. Validate ALB DNS Resolution

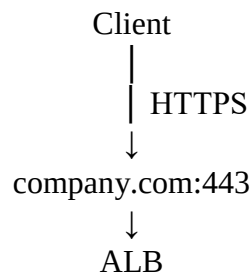
Sekarang kita mulai masuk ke runtime/data-plane validation. Ini penting karena API mengatakan ALB exists belum tentu berarti endpoint benar-benar bisa menerima traffic.

```
# 72.1 Resolve ALB, Actual floci → ::1 localhost.floci.io enterprise-
alb-9a2b506e31314ec0.elb.localhost.floci.io. Interpretasi → Hostname
ALB berhasil di-resolve. Namun targetnya adalah ::1 yaitu IPv6 loopback.
DNS resolution berhasil Tetapi ini belum berarti HTTPS service tersedia.

getent hosts "$ALB_DNS_NAME"
```

Step 73. Test HTTPS Data Plane

Mencoba benar-benar melakukan:



Pertama, command yang sebelumnya lu jalankan:

```
curl -vk \
  --resolve company.com:443:"$ALB_DNS_NAME" \
  https://company.com/
```

Kenapa gagal? Output: **Resolve address 'enterprise-alb-...floci.io' found illegal** Karena syntax curl --resolve tidak menerima hostname pada bagian address. Formatnya HOST:PORT:ADDRESS dan ADDRESS harus berupa IP address. Jadi company.com:443:enterprise-alb-....floci.io itu ✗ salah.

73.1 Resolve hostname ke IP

```
ALB_IP=$(getent hosts "$ALB_DNS_NAME" | awk 'NR==1 {print $1}')
```

```
echo "ALB IP: $ALB_IP"
```

73.1.1 Actual → ALB IP: ::1. Sekarang syntax curl sudah benar, Actual result → Trying [::1]:443... connect to ::1 port 443 ... failed: Connection refused. Artinya DNS resolution: company.com → ALB hostname → ::1 berhasil Tetapi ::1:443 tidak ada service yang menerima koneksi. Sehingga TCP connection → FAILED. Status NOT VERIFIED – ALB HTTPS data plane. Bukan Certificate salah atau Route53 salah atau bahkan HTTPS listener salah. Control-plane evidence justru menunjukkan listener sudah ada.

```
curl -vk \
  --resolve company.com:443:$ALB_IP \
  https://company.com/
```

Step 74. Validate TLS Handshake

Kita juga mencoba langsung menggunakan OpenSSL.

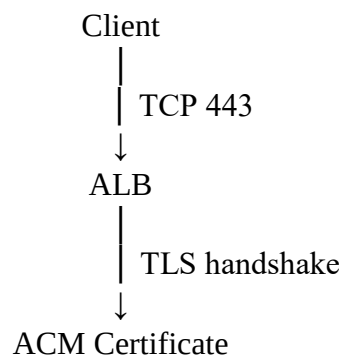
```
openssl s_client \
  -connect "$ALB_IP:443" \
  -servername company.com \
  </dev/null 2>/dev/null \
```

```
| openssl x509 \
    -noout \
    -subject \
    -issuer \
    -dates
```

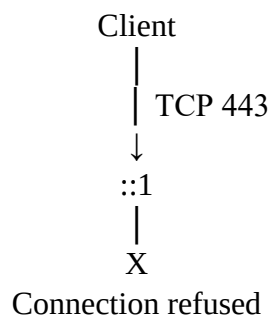
Actual → Could not read certificate from <stdin> Unable to load certificate. terjadi hal ini Karena OpenSSL tidak mendapatkan TLS certificate dari server. Alasannya sudah terlihat pada Step 73:

```
:::1:443
↓
Connection refused
```

Jadi TLS handshake bahkan belum terjadi. Architecture yang seharusnya:



Tetapi actual:



Status NOT VERIFIED — TLS handshake. Ini konsisten dengan Step 73.

Step 75. Final Phase 7 Validation

Sekarang kita pisahkan control plane dan data plane supaya hasil lab tidak misleading.

75.1 Route53 Validation, Expected company.com. A AliasTarget DNSName = enterprise-alb-.... Status PASS

```
aws --endpoint-url="$ENDPOINT" \
    route53 list-resource-record-sets \
```

```

--hosted-zone-id "$HOSTED_ZONE_ID" \
--query 'ResourceRecordSets[?Name==`company.com.`]' \
--output json

# 75.2 ACM Validation, Actual → company.com ISSUED None Status. PASS –
certificate object/status Tetapi DNS validation lifecycle = NOT VERIFIED

aws --endpoint-url="$ENDPOINT" \
    acm describe-certificate \
    --certificate-arn "$CERTIFICATE_ARN" \
    --query 'Certificate.[DomainName,Status,ValidationMethod]' \
    --output table

# 75.3 HTTPS Listener Validation, Expected → HTTPS 443
certificate/2709f3de... forward enterprise-target-group. Status PASS

aws --endpoint-url="$ENDPOINT" \
    elbv2 describe-listeners \
    --load-balancer-arn "$ALB_ARN" \
    --query
'Listeners[?Port==`443`].[Protocol,Port,ListenerArn,Certificates[0].Ce
rtificateArn,DefaultActions]' \
    --output json

# 75.4 Data Plane Validation. Actual → Trying [::1]:443... Connection
refused. Status NOT VERIFIED

curl -vk \
    --resolve company.com:443:$ALB_IP \
    https://company.com/

```

Phase 8 — Secrets Management & Encryption

Phase 8 bertujuan memindahkan database credential yang sensitif keluar dari application configuration, environment variable, YAML manifest, dan source code, kemudian menerapkan mekanisme pengelolaan secret yang terkontrol menggunakan:

- AWS Secrets Manager sebagai tempat penyimpanan dan retrieval secret.
- AWS KMS sebagai encryption key yang melindungi secret.
- IAM + IRSA sebagai authorization layer yang menentukan workload mana yang boleh mengambil secret.
- Kubernetes ServiceAccount sebagai identity workload.
- RDS PostgreSQL sebagai database target.

Dengan demikian, application tidak perlu menyimpan password database secara hardcoded.

Tiga Security Mechanism Utama

1. Secrets Manager — "Di mana secret disimpan?"

AWS Secrets Manager menjadi centralized secret store untuk credential database. Contoh secret:

```
{
  "username": "postgres",
  "password": "[REDACTED]",
  "host": "enterprise-postgres",
  "port": "5432",
  "database": "enterprise_db"
}
```

Application tidak perlu menyimpan credential tersebut di Application source code, Environment variable, Kubernetes YAML, Configuration file, Git repository. Application cukup melakukan:

```
GetSecretValue
    ↓
enterprise/database
    ↓
database credential
```

2. KMS — "Key apa yang melindungi secret?"

AWS KMS menyediakan encryption key yang digunakan oleh Secrets Manager untuk melindungi secret. Dalam Phase 8 dibuat:

alias/enterprise-secrets

↓
Enterprise Secrets KMS Key

Secara konseptual:

Secret
↓
Secrets Manager
| encrypted using KMS
↓
Encrypted secret

KMS bukan tempat penyimpanan password. KMS menyediakan cryptographic key management, sedangkan secret tetap dikelola oleh Secrets Manager.

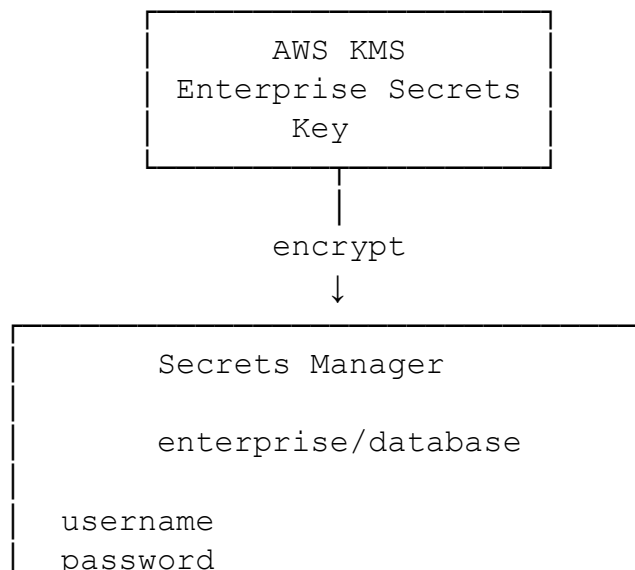
3. IAM + IRSA — "Siapa yang boleh mengambil secret?"

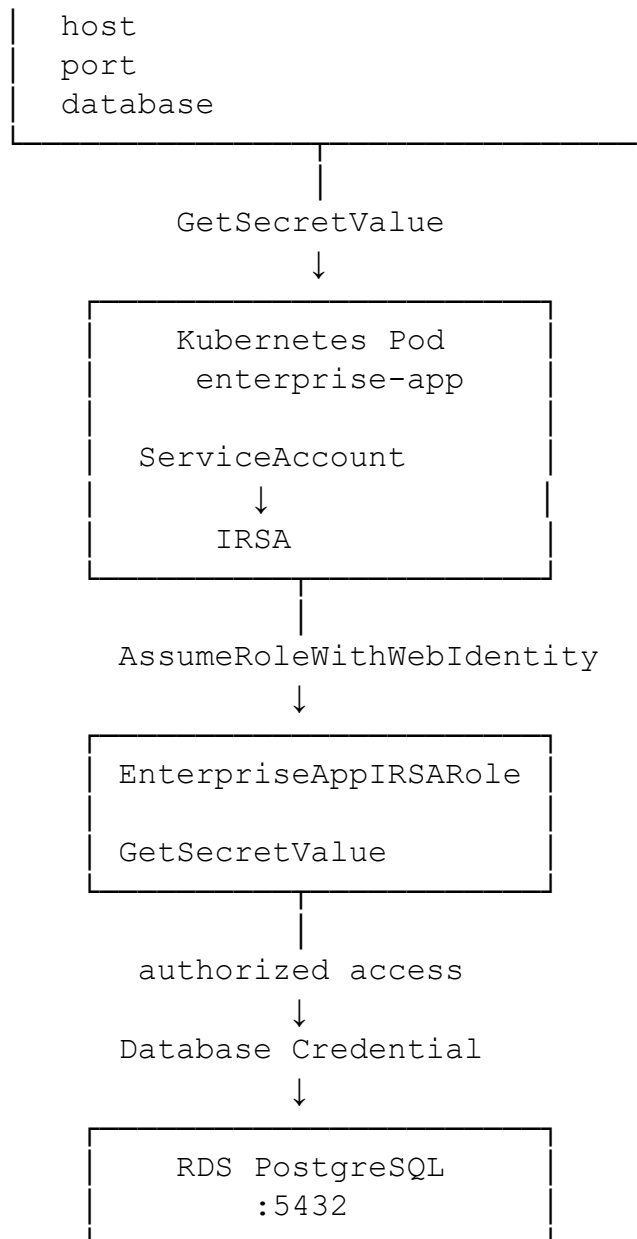
IAM menentukan permission terhadap Secrets Manager. Dalam project ini:

Kubernetes ServiceAccount
↓
enterprise-app-sa
↓
EnterpriseAppIRSARole
↓
secretsmanager:GetSecretValue
↓
enterprise/database

Dengan demikian, application workload memperoleh temporary AWS credentials melalui IRSA/STS dan menggunakan identity tersebut untuk mengakses secret. Tidak diperlukan static AWS access key yang disimpan di Kubernetes Secret atau source code.

Complete Security Flow:





Yang Akan Divalidasi

Security Layer	Validation	Status
KMS	Key berhasil dibuat	☑ PASS
KMS	Key dalam state Enabled	☑ PASS
KMS	Alias alias/enterprise-secrets tersedia	☑ PASS
KMS	Encrypt/decrypt round-trip berhasil	☑ PASS

KMS	AWS-native cryptographic semantics	⚠ Emulator-dependent
Secrets Manager	Secret enterprise/database berhasil dibuat	✓ PASS
Secrets Manager	Secret dapat di-read dari host	✓ PASS
Secrets Manager	Secret menggunakan KMS key	✓ PASS
IAM	EnterpriseAppIRSARole memiliki permission GetSecretValue	✓ PASS
IRSA	ServiceAccount memiliki IAM role annotation	✓ PASS
IRSA	Pod memperoleh projected ServiceAccount token	✓ PASS
IRSA / STS	Pod memperoleh temporary AWS credentials	✓ PASS
Pod → Secrets Manager	Pod berhasil melakukan GetSecretValue	✓ PASS
Application Code	Python/boto3 berhasil mengambil secret	✓ PASS
Application Code	Secret JSON berhasil di-parse	✓ PASS
Secrets Manager → RDS	Credential/metadata secret sesuai dengan RDS yang digunakan	✓ PASS
Unauthorized Workload	Workload tanpa application IRSA tidak memperoleh credential	✓ PASS
Unauthorized Workload	IAM AccessDenied secara eksplisit teruji	⚠ Not Verified — Floci
KMS Key Policy	Explicit authorization behavior	⚠ Not separately verified

Step 76. Prepare Phase 8 Environment

Menyiapkan environment variable yang akan dipakai semua command Phase 8. Karena RDS sudah dibuat pada Phase 5, kita menggunakan database existing.

```
# 76.1 Parameter dan Fungsinya → ENDPOINT adalah Endpoint API Floci,
AWS_DEFAULT_REGION adalah Region lab, AWS_ACCOUNT_Id adalah Account ID
Floci, SECRET_NAME adalah Nama secret database, KMS_ALIAS adalah Alias
KMS, EKS_NAMESPACE adalah Namespace application, EKS_SERVICE_ACCOUNT
adalah ServiceAccount workload, IRSA_ROLE_NAME adalah IAM role workload
```

```
export ENDPOINT="http://localhost:4566"
```

```
export AWS_DEFAULT_REGION="us-east-1"
```

```

export AWS_ACCOUNT_ID="000000000000"
export SECRET_NAME="enterprise/database"
export KMS_ALIAS="alias/enterprise-secrets"
export EKS_NAMESPACE="enterprise-app"
export EKS_SERVICE_ACCOUNT="enterprise-app-sa"
export IRSA_ROLE_NAME="EnterpriseAppIRSARole"

# 76.2 Validate AWS Identity, Expected → Account: 000000000000. Validasi
→ Kalau account ID bukan 000000000000, jangan lanjut karena environment
AWS CLI belum sesuai lab.

aws --endpoint-url="$ENDPOINT" sts get-caller-identity

```

Step 77. Retrieve Existing RDS Information

Kita jangan membuat RDS kedua. Phase 5 sudah membuat enterprise-postgres. Sekarang kita mengambil metadata database existing untuk dimasukkan ke Secret Manager.

77.1 List RDS, Expected → Kurang lebih DescribeDBInstances : enterprise-postgres

```

aws --endpoint-url="$ENDPOINT" \
    rds describe-db-instances \
    --query 'DBInstances[].DBInstanceIdentifier' \
    --output table

```

77.2 Retrieve RDS Endpoint, Parameter → --db-instance-identifier adalah memilih RDS instance yang ingin diperiksa. Endpoint.Address adalah mengambil hostname database. Expected → Kurang lebih enterprise-postgres.... Catatan: pada Floci, endpoint runtime RDS yang sudah tervalidasi sebelumnya dapat berbeda dari logical AWS endpoint. Jangan mengganti endpoint berdasarkan asumsi; gunakan endpoint yang memang dikembalikan/terbukti oleh lab.

```

export DB_IDENTIFIER="enterprise-postgres"
export DB_ENDPOINT=$(
    aws --endpoint-url="$ENDPOINT" \
        rds describe-db-instances \
        --db-instance-identifier "$DB_IDENTIFIER" \
        --query 'DBInstances[0].Endpoint.Address' \
        --output text
)
echo "$DB_ENDPOINT"

```

77.3 Retrieve Port, Expected → 7001

```

export DB_PORT=$(
    aws --endpoint-url="$ENDPOINT" \
        rds describe-db-instances \
        --db-instance-identifier "$DB_IDENTIFIER" \
        --query 'DBInstances[0].Endpoint.Port' \
        --output text
)
echo "$DB_PORT"

```

77.4 Final Validation

```

echo "DB Identifier : $DB_IDENTIFIER"
echo "DB Endpoint   : $DB_ENDPOINT"
echo "DB Port       : $DB_PORT"

```

Step 78. Create AWS KMS Key

Membuat customer-managed KMS key untuk security layer Secrets Manager.

Mental model:

```

KMS Key
├── digunakan service AWS
│   └── ↓
│       encrypt secret data

```

Kita tidak menyimpan password database di KMS. KMS bukan password vault. Secrets Manager menyimpan secret. KMS menyediakan cryptographic key untuk encryption.

78.1 Create Key, Parameter → **create-key** adalah membuat customer-managed symmetric KMS key. **--description** adalah metadata untuk menjelaskan tujuan key. **--query** adalah mengambil hanya KeyId. **Expected** → Misalnya **xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx**

```

KMS_KEY_ID=$(
    aws --endpoint-url="$ENDPOINT" \
        kms create-key \
        --description "Enterprise Secrets Encryption Key" \
        --query 'KeyMetadata.KeyId' \
        --output text
)
export KMS_KEY_ID
echo "$KMS_KEY_ID"

```

Step 79. Inspect KMS Key

Memastikan key yang dibuat benar-benar tersedia. Expected → Konsepnya:

KeyId	ARN	Enabled
xxxx	arn:aws:kms:...	Enabled

Yang kita cari adalah KeyState = Enabled dan key digunakan untuk encryption.

```
aws --endpoint-url="$ENDPOINT" \  
    kms describe-key \  
    --key-id "$KMS_KEY_ID"
```

Atau:

```
aws --endpoint-url="$ENDPOINT" \  
    kms describe-key \  
    --key-id "$KMS_KEY_ID" \  
    --query 'KeyMetadata.[KeyId,Arn,KeyState,KeyUsage,KeySpec]' \  
    --output table
```

Step 80. Create KMS Alias

Memberikan nama manusiawi kepada KMS key. Daripada 12345678-abcd-..., kita punya alias/enterprise-secrets

80.1 Create Alias, Parameter → **--alias-name** adalah nama logical alias. **--target-key-id** adalah KMS key yang ditunjuk alias.

```
aws --endpoint-url="$ENDPOINT" \  
    kms create-alias \  
    --alias-name "$KMS_ALIAS" \  
    --target-key-id "$KMS_KEY_ID"
```

80.2 Validate Alias, Expected → **alias/enterprise-secrets**

```
aws --endpoint-url="$ENDPOINT" \  
    kms list-aliases \  
    --query "Aliases[?AliasName=='$KMS_ALIAS']" \  
    --output table
```

Step 81. Validate KMS Cryptographic Operation

Sebelum menghubungkan KMS ke Secrets Manager, kita validasi bahwa key memang dapat melakukan operasi cryptographic dasar.

81.1 Encrypt. Expected → Output berupa ciphertext/base64 blob, bukan enterprise-kms-validation

```
PLAINTEXT="enterprise-kms-validation"
printf '%s' "$PLAINTEXT" > plaintext.txt
CIPHERTEXT_B64=$(
    aws --endpoint-url="$ENDPOINT" \
        kms encrypt \
        --key-id "$KMS_KEY_ID" \
        --plaintext fileb://plaintext.txt \
        --query 'CiphertextBlob' \
        --output text
)
echo "$CIPHERTEXT_B64"
```

81.2 Decrypt

```
echo "$CIPHERTEXT_B64" | base64 -d > ciphertext.bin
```

82.2.1 Kemudian di bawah ini, Expected → enterprise-kms-validation. Kenapa output Plaintext masih perlu base64 -d? Karena response AWS CLI merepresentasikan binary Plaintext sebagai base64 text.

```
aws --endpoint-url="$ENDPOINT" \
    kms decrypt \
    --ciphertext-blob fileb://ciphertext.bin \
    --query 'Plaintext' \
    --output text \
    | base64 -d
```

81.3 One-shot validation, Expected → Original : enterprise-kms-validation, Decrypted: enterprise-kms-validation. Intinya Flow yang sedang dibuktikan: plaintext → KMS Encrypt → ciphertext → KMS Decrypt → plaintext kembali. Dan ini bukan berarti application harus melakukan encrypt/decrypt credential database sendiri menggunakan KMS. Untuk Phase 8, operasi ini cuma validasi bahwa KMS key bekerja. Nanti hubungan yang lebih realistic adalah Secret → Secrets Manager -encrypted with→ KMS

```
PLAINTEXT="enterprise-kms-validation"
printf '%s' "$PLAINTEXT" > plaintext.txt
CIPHERTEXT_B64=$(
    aws --endpoint-url="$ENDPOINT" \
```

```

    kms encrypt \
    --key-id "$KMS_KEY_ID" \
    --plaintext fileb://plaintext.txt \
    --query 'CiphertextBlob' \
    --output text
)
echo "$CIPHERTEXT_B64" | base64 -d > ciphertext.bin
DECRYPTED=$(
    aws --endpoint-url="$ENDPOINT" \
    kms decrypt \
    --ciphertext-blob fileb://ciphertext.bin \
    --query 'Plaintext' \
    --output text \
    | base64 -d
)
echo "Original : $PLAINTEXT"
echo "Decrypted: $DECRYPTED"
if [ "$PLAINTEXT" = "$DECRYPTED" ]; then
    echo "KMS ENCRYPT/DECRYPT: PASS"
else
    echo "KMS ENCRYPT/DECRYPT: FAIL"
fi

```

Step 82. Create Database Secret

Sekarang kita masuk ke Secrets Manager. Menyimpan credential database di

Secrets Manager

```

└── enterprise/database

```

dan bukan di source code aplikasi.

82.1 Set Database Credentials, Gunakan credential yang memang dipakai ketika RDS dibuat pada Phase 5. Contoh struktur seperti di bawah ini. Untuk lab, password boleh berupa credential lab. Untuk production, jangan hardcode credential seperti ini di shell history/source repository.

```
export DB_USERNAME="postgres"
```

```

export DB_PASSWORD="Password123!"
export DB_NAME="enterprise_db"

# 82.2 Create Secret, Parameter → --name adalah nama secret, --
description adalah dokumentasi, --kms-key-id adalah KMS key untuk
encryption, --secret-string adalah data credential

aws --endpoint-url="$ENDPOINT" \
  secretsmanager create-secret \
    --name "$SECRET_NAME" \
    --description "Enterprise PostgreSQL Database Credential" \
    --kms-key-id "$KMS_KEY_ID" \
    --secret-string "{
      \"username\": \"$DB_USERNAME\",
      \"password\": \"$DB_PASSWORD\",
      \"host\": \"$DB_ENDPOINT\",
      \"port\": \"$DB_PORT\",
      \"database\": \"$DB_NAME\"
    }"

# 82.3 Expected → Response akan berisi ARN, Name, VersionId, Contoh
konsep:
arn:aws:secretsmanager:us-east-1:000000000000:secret:enterprise/database-...

```

Step 83. Validate Secret Metadata

Memastikan secret exists dan menggunakan KMS key yang kita buat. Expected → Kurang lebih:

enterprise/database

arn:aws:secretsmanager:...

<enterprise KMS key ARN/ID>

Enterprise PostgreSQL Database Credential

Yang penting Secrets Manager

└─ KMS Key

sudah terhubung pada metadata service.

```

aws --endpoint-url="$ENDPOINT" \
  secretsmanager describe-secret \
    --secret-id "$SECRET_NAME"

```

Atau:

```
aws --endpoint-url="$ENDPOINT" \
  secretsmanager describe-secret \
  --secret-id "$SECRET_NAME" \
  --query '[Name,ARN,KmsKeyId,Description]' \
  --output table
```

Step 84. Validate Secret Retrieval from Host

Memastikan secret benar-benar dapat diambil menggunakan AWS API. Security note : Untuk dokumentasi project, jangan masukkan output password ke screenshot/Git. Lebih aman gunakan dibawah ini untuk evidence tanpa membocorkan credential. Expected → JSON:

```
{
  "username": "postgres",
  "password": "...",
  "host": "...",
  "port": "7001",
  "database": "enterprise_db"
}

aws --endpoint-url="$ENDPOINT" \
  secretsmanager get-secret-value \
  --secret-id "$SECRET_NAME" \
  --query 'ARN'
```

Step 85. Verify Existing IRSA Configuration

Kita sekarang menghubungkan Secrets Manager dengan workload Kubernetes.

Karena Phase 3 sudah punya:

enterprise-app-sa
↓
EnterpriseAppIRSARole

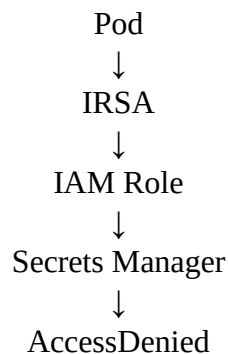
kita reuse.

```
# 85.1 Validate ServiceAccount, Expected → Ada: metadata.annotations:
eks.amazonaws.com/role-arn:
arn:aws:iam::000000000000:role/EnterpriseAppIRSARole
```

```
kubectl get serviceaccount \
  "$EKS_SERVICE_ACCOUNT" \
  -n "$EKS_NAMESPACE" \
  -o yaml
```

Step 86. Add Secrets Manager Permission to IAM Role

IRSA sudah memberikan identity. Sekarang identity tersebut harus diberi permission secretsmanager:GetSecretValue Tanpa permission:



86.1 Create Policy, Penjelasan → Effect nya Allow. Action nya hanya membaca secret. Resource nya sementara * untuk kompatibilitas lab. untuk Production hardening, Nanti idealnya "Resource": "arn:aws:secretsmanager:us-east-1:000000000000:secret:enterprise/database-*" supaya application role tidak dapat membaca semua secret.

```
mkdir ~/phase8
cd phase8
cat > enterprise-secrets-policy.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ],
      "Resource": "*"
    }
  ]
}
EOF
```

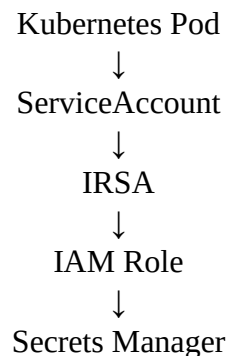
86.2 Attach Policy, Expected → Tidak ada error.

```
aws --endpoint-url="$ENDPOINT" \
  iam put-role-policy \
  --role-name "$IRSA_ROLE_NAME" \
```

```
--policy-name EnterpriseSecretsReadPolicy \  
--policy-document file://enterprise-secrets-policy.json
```

Step 87. Create Secrets Manager Validation Pod

Ini validasi paling penting. Kita tidak cukup membuktikan Host → Secrets Manager, Kita ingin membuktikan:



87.1 Create Pod, Kenapa projected token? Karena ini menyediakan ServiceAccount JWT → STS AssumeRoleWithWebIdentity → temporary AWS credentials. Ini mengikuti mekanisme IRSA yang sudah kita validasi sebelumnya.

```
cat <<'EOF' > secretsmanager-test.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: secretsmanager-test  
  namespace: enterprise-app  
spec:  
  serviceAccountName: enterprise-app-sa  
  restartPolicy: Never  
  containers:  
    - name: aws-cli  
      image: amazon/aws-cli:latest  
      command:  
        - /bin/sh  
        - -c  
        - |  
          sleep 3600  
      volumeMounts:  
        - name: aws-iam-token  
          mountPath: /var/run/secrets/eks.amazonaws.com/serviceaccount  
          readOnly: true  
  volumes:  
    - name: aws-iam-token  
      projected:  
        sources:  
          - serviceAccountToken:  
              audience: sts.amazonaws.com  
              expirationSeconds: 3600  
              path: token
```

EOF

87.2 Deploy Pod

```
kubectl apply -f secretsmanager-test.yaml
```

Step 88. Validate Pod Identity

Memastikan Pod mempunyai projected JWT. Expected → token → ..data/token

88.1

```
kubectl exec -n enterprise-app secretsmanager-test -- \
  ls -l /var/run/secrets/eks.amazonaws.com/serviceaccount/
```

88.2 Kemudian di bawah ini, Expected → TOKEN_PRESENT

```
kubectl exec -n enterprise-app secretsmanager-test -- \
  sh -c 'test -s
/var/run/secrets/eks.amazonaws.com/serviceaccount/token && echo
"TOKEN_PRESENT"'
```

Step 89. Assume IAM Role from Pod

Karena Floci sebelumnya punya compatibility issue pada automatic AWS CLI web-identity credential chain, gunakan explicit STS exchange yang sudah terbukti bekerja.

89.1 Masuk ke Pod:

```
kubectl exec -it -n enterprise-app secretsmanager-test -- /bin/sh
```

89.2 Kemudian:

```
export AWS_DEFAULT_REGION=us-east-1
```

89.3 Ambil token:

```
TOKEN=$(cat /var/run/secrets/eks.amazonaws.com/serviceaccount/token)
```

89.4 Request temporary credentials:

```
CREDS=$(aws --endpoint-url="http://172.17.0.1:4566" sts assume-role-
with-web-identity --role-arn
arn:aws:iam::000000000000:role/EnterpriseAppIRSARole --role-session-
name secretsmanager-validation --web-identity-token "$TOKEN" --query
"Credentials.[AccessKeyId,SecretAccessKey,SessionToken]" --output
text)
```

89.5 Set credentials:

```
set -- $CREDS; export AWS_ACCESS_KEY_ID="$1"; export
AWS_SECRET_ACCESS_KEY="$2"; export AWS_SESSION_TOKEN="$3"
```

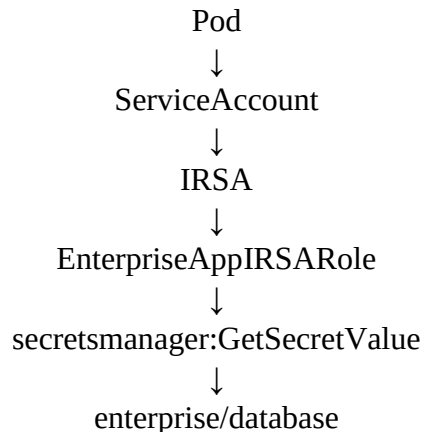
89.6 Validate identity. Expected → Identity menunjukkan role assumed-role/EnterpriseAppIRSARole/...

```
aws --endpoint-url="http://172.17.0.1:4566" \
```

```
sts get-caller-identity
```

Step 90. Pull Secret from Pod

Sekarang baru kita melakukan validasi yang sebenarnya. Expected → JSON database credential. Ini membuktikan:



```
aws --endpoint-url="http://172.17.0.1:4566" \
  secretsmanager get-secret-value \
  --secret-id enterprise/database \
  --query 'SecretString' \
  --output text
```

Step 91. Pull Secret Using Python Application Code

Step ini memvalidasi bahwa application code menggunakan AWS SDK (boto3) untuk mengambil secret dari AWS Secrets Manager, bukan menggunakan AWS CLI. Pada Step 90, validasi Secrets Manager dilakukan menggunakan AWS CLI di dalam Pod secretsmanager-test. Untuk Step 91, kita menggunakan Python application container agar flow lebih menyerupai aplikasi sebenarnya. Catatan: amazon/aws-cli digunakan untuk Step 90 dan tidak memiliki pip/Python runtime yang kita perlukan. Karena itu, Step 91 menggunakan Pod Python terpisah.

```
# 91.1 Exit dari Pod Step 90. Jika saat ini masih berada di dalam sh-5.2# maka keluar terlebih dahulu. Lu akan kembali ke shell WSL floci@DESKTOP-0VO33K8:~$. Jangan hapus secretsmanager-test dulu, karena Pod tersebut masih bisa digunakan sebagai bukti validasi Step 90.
```

```
exit
```

```
# 91.2 Create Python Application Pod. Kita menggunakan image Python resmi dan tetap menggunakan ServiceAccount enterprise-app-sa supaya Pod mendapatkan projected ServiceAccount token yang sama dengan validasi IRSA sebelumnya. Jalankan:
```

```

cat > boto3-secrets-test.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: boto3-secrets-test
  namespace: enterprise-app
spec:
  serviceAccountName: enterprise-app-sa
  restartPolicy: Never

  containers:
  - name: python
    image: python:3.12-slim
    command:
      - /bin/sh
      - -c
      - |
        sleep 3600
    volumeMounts:
      - name: aws-iam-token
        mountPath: /var/run/secrets/eks.amazonaws.com/serviceaccount
        readOnly: true
  volumes:
  - name: aws-iam-token
    projected:
      sources:
        - serviceAccountToken:
            audience: sts.amazonaws.com
            expirationSeconds: 3600
            path: token

```

EOF

91.3 Apply:

```
kubectl apply -f boto3-secrets-test.yaml
```

91.4 Validasi:

```
kubectl get pod boto3-secrets-test -n enterprise-app
```

91.5 Enter Python Application Container. Masuk ke container:

```

kubectl exec -it \
  -n enterprise-app \
  boto3-secrets-test \
  -- /bin/sh

```

91.6 Install boto3. Image python:3.12-slim memiliki pip, jadi:

```
pip install boto3
```

91.6.1 Validasi. Expected kurang lebih → Python 3.12.x boto3: 1.x.x

```
python --version
```

```
python -c "import boto3; print('boto3:', boto3.__version__)"
```

91.7 Obtain Temporary AWS Credentials. Karena pada environment Floci kita sudah membuktikan bahwa explicit AssumeRoleWithWebIdentity berhasil, kita gunakan mekanisme tersebut untuk mendapatkan temporary credentials. Masih di dalam Pod:

```
export AWS_DEFAULT_REGION=us-east-1
```

91.8 Ambil projected token:

```
TOKEN=$(cat /var/run/secrets/eks.amazonaws.com/serviceaccount/token)
```

91.9 Validasi token tersedia. Expected → TOKEN_PRESENT

```
test -s /var/run/secrets/eks.amazonaws.com/serviceaccount/token \
    && echo "TOKEN_PRESENT"
```

91.10 Sekarang kita perlu melakukan STS exchange. Karena python:3.12-slim tidak memiliki AWS CLI, install AWS CLI hanya untuk validation environment ini:

```
pip install awscli
```

91.11 Kemudian:

```
CREDS=$(aws --endpoint-url="http://172.17.0.1:4566" sts assume-role-
with-web-identity --role-arn
arn:aws:iam::000000000000:role/EnterpriseAppIRSARole --role-session-
name boto3-secrets-validation --web-identity-token "$TOKEN" --query
"Credentials.[AccessKeyId,SecretAccessKey,SessionToken]" --output
text)
```

91.12 Export temporary credentials. Jangan echo \$CREDS atau print ketiga credential tersebut.

```
set -- $CREDS; export AWS_ACCESS_KEY_ID="$1"; export
AWS_SECRET_ACCESS_KEY="$2"; export AWS_SESSION_TOKEN="$3"
```

91.13 Validasi identity. Expected adalah identity berupa assumed role EnterpriseAppIRSARole.

```
aws --endpoint-url="http://172.17.0.1:4566" \
    sts get-caller-identity
```

91.14 Create Python Script. Masih di dalam Pod:

```
cat > /tmp/get_secret.py <<'PY'
import boto3
import json
client = boto3.client(
    "secretsmanager",
    endpoint_url="http://172.17.0.1:4566",
    region_name="us-east-1"
)
response = client.get_secret_value(
    SecretId="enterprise/database"
)
secret = json.loads(response["SecretString"])
print("Secret retrieved successfully")
print("Username:", secret["username"])
```

```
print("Host:", secret["host"])
print("Port:", secret["port"])
print("Database:", secret["database"])
print("Password: [REDACTED]")
PY
```

91.15 Execute Python Application, Expected → Secret retrieved successfully; Username: postgres; Host: ...; Port: 7001; Database: enterprise_db; Password: [REDACTED]. Kenapa password tidak di-print? Karena application log bukan tempat untuk mengekspos credential. Aplikasi hanya membutuhkan password sebagai data runtime untuk koneksi ke database. Password tidak perlu ditulis ke stdout, log file, CloudWatch, SIEM, atau terminal. Jadi Secrets Manager -GetSecretValue→ Application (username → digunakan, host → digunakan, port → digunakan, database → digunakan, password → digunakan namun tidak di log)

```
python /tmp/get_secret.py
```

91.16 Exit dari Application Container

```
exit
```

91.17 Jangan langsung hapus Pod kalau masih mau debugging atau mengambil evidence. Nanti setelah seluruh Phase 8 selesai, kedua validation Pod bisa dibersihkan:

```
# kubectl delete pod -n enterprise-app secretsmanager-test boto3-
secrets-test
```

92.1 Pod Unauthorized. Expected → default. Ini penting karena Pod tersebut tidak menggunakan enterprise-app-sa. yang memiliki IRSA role EnterpriseAppIRSARole Pod unauthorized-secret-test sudah dibuat sebelumnya, jadi tidak perlu membuat ulang. Pastikan ServiceAccount yang digunakan:

```
kubectl get pod unauthorized-secret-test \
-n enterprise-app \
-o jsonpath='{.spec.serviceAccountName}{"\n"}'
```

92.2 Masuk ke Unauthorized Pod

```
kubectl exec -it \
-n enterprise-app \
unauthorized-secret-test \
-- /bin/sh
```

92.3 Set AWS Region Di dalam Pod

```
export AWS_DEFAULT_REGION=us-east-1
```

92.4 Attempt to Retrieve Secret. Ada dua hasil yang valid untuk dicatat, tergantung behavior Floci. Expected ideal behavior AccessDeniedException. Artinya request sudah sampai authorization layer, tetapi identity yang digunakan tidak memiliki secretsmanager:GetSecretValue

```
aws --endpoint-url="http://172.17.0.1:4566" \
```

```
secretsmanager get-secret-value \
--secret-id enterprise/database
```

92.5 Jika Mendapat NoCredentials Pada environment lu, hasil yang sudah muncul adalah NoCredentials: Unable to locate credentials. Jangan dianggap sebagai AccessDeniedException. Artinya: Unauthorized Pod -tidak memiliki AWS credentials→ X AWS API. Request berhenti sebelum IAM authorization dilakukan. Jadi catat PASS – Unauthorized workload does not obtain the application's IRSA credentials. IAM AccessDenied: NOT VERIFIED. Ini tetap merupakan security validation yang berguna: Pod dengan default ServiceAccount tidak memperoleh EnterpriseAppIRSARole. Tetapi kita tidak boleh mengklaim bahwa Floci sudah membuktikan IAM AccessDenied.

92.6 Final Result. Untuk hasil lu yang sekarang: Step 92 – PASS / NOT VERIFIED. Lebih tepatnya Validation dan Resultnya → Unauthorized Pod menggunakan default SA PASS. Tidak mendapatkan IRSA application credentials PASS. Secret berhasil diambil NO. IAM AccessDeniedException teruji NOT VERIFIED. Floci IAM enforcement NOT VERIFIED.

92.7 Exit from container

```
exit
```

Step 93. Validate KMS + Secrets Manager Relationship

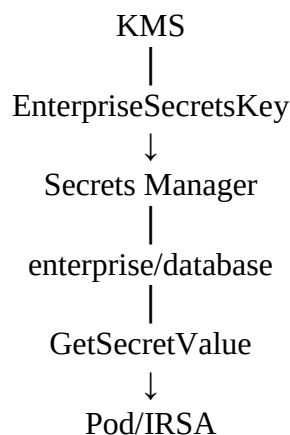
Sekarang kita validasi metadata final. Expected → Kurang lebih:

enterprise/database

arn:aws:kms:us-east-1:000000000000:key/...

arn:aws:secretsmanager:...

Architecture yang terbukti:



```
aws --endpoint-url="$ENDPOINT" \
secretsmanager describe-secret \
--secret-id "$SECRET_NAME" \
--query '[Name,KmsKeyId,ARN]' \
```

```
--output table
```

Step 94 Final Phase 8 Validation

Sekarang lakukan satu final validation.

94.1 KMS. Expected → Enabled ENCRYPT_DECRYPT

```
aws --endpoint-url="$ENDPOINT" \  
    kms describe-key \  
    --key-id "$KMS_KEY_ID" \  
    --query 'KeyMetadata.[KeyId,KeyState,KeyUsage]' \  
    --output table
```

94.2 Secrets Manager

```
aws --endpoint-url="$ENDPOINT" \  
    secretsmanager describe-secret \  
    --secret-id "$SECRET_NAME" \  
    --query '[Name,KmsKeyId]' \  
    --output table
```

94.3 IAM Role, Expected → EnterpriseSecretsReadPolicy

```
aws --endpoint-url="$ENDPOINT" \  
    iam list-role-policies \  
    --role-name "$IRSA_ROLE_NAME"
```

94.4 Kubernetes ServiceAccount, Expected → arn:aws:iam::000000000000:role/EnterpriseAppIRSARole

```
kubectl get sa \  
    "$EKS_SERVICE_ACCOUNT" \  
-n "$EKS_NAMESPACE" \  
-o jsonpath='{.metadata.annotations.eks\.amazonaws\.com/role-  
arn}{"\n"}'
```

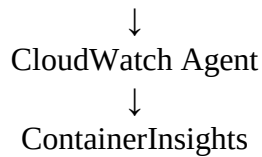
94.5 Pod → STS. Expected → TOKEN_PRESENT

```
kubectl exec -n enterprise-app secretsmanager-test -- \  
    sh -c 'test -s  
/var/run/secrets/eks.amazonaws.com/serviceaccount/token && echo  
TOKEN_PRESENT'
```

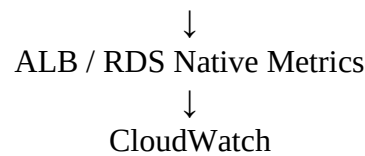
Phase 9 — Monitoring & Observability

Phase 9 bertujuan membangun monitoring, logging, dan alerting layer untuk enterprise application yang berjalan pada Kubernetes/EKS. Pada phase sebelumnya, security control sudah mencakup IAM, IRSA, KMS, Secrets Manager, dan RDS. Phase 9 menambahkan kemampuan untuk mengamati workload Kubernetes, mengumpulkan application logs, memonitor AWS managed services, serta menyediakan application alerting menggunakan CloudWatch Alarm dan SNS. Phase ini dibagi menjadi tiga capability utama:

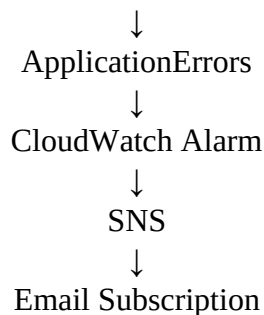
1. Kubernetes / Application Monitoring



2. AWS Service Monitoring

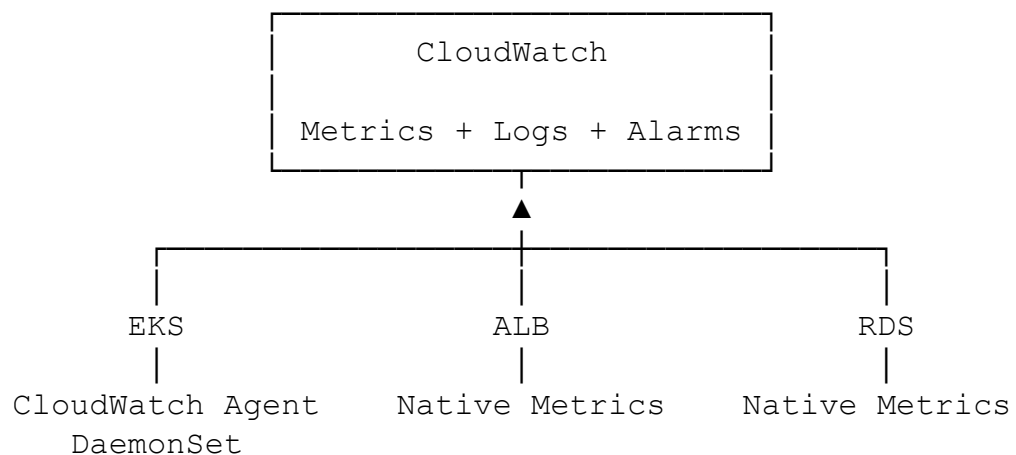


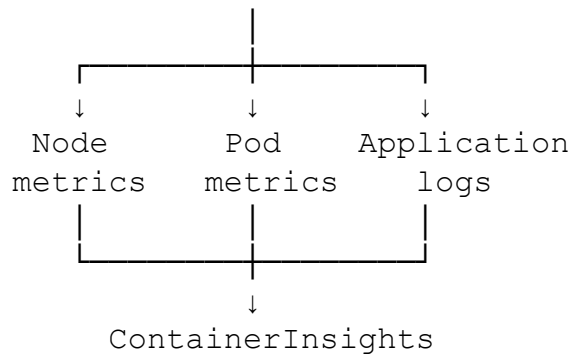
3. Application Alerting



Monitoring Architecture

Untuk workload Kubernetes, CloudWatch Agent dijalankan sebagai DaemonSet pada node Kubernetes.



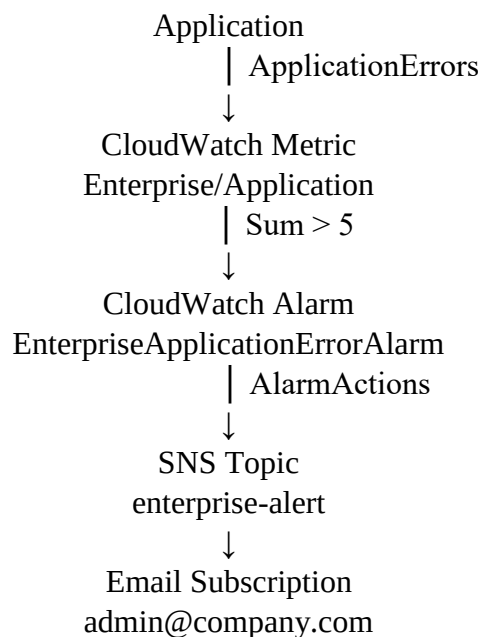


CloudWatch Agent digunakan untuk mengumpulkan informasi dari node dan workload Kubernetes, termasuk CPU, memory, disk, disk I/O, network, serta log container. Application logs dikumpulkan dari log file container pada node `/var/log/containers/*.log`

Sementara itu, ALB dan RDS tidak menggunakan CloudWatch Agent. Secara native AWS, kedua service tersebut mempublikasikan service metrics masing-masing ke CloudWatch.

Application Alerting Architecture

Selain monitoring, Phase 9 membangun satu alerting path khusus untuk application error. Custom metric `ApplicationErrors` digunakan sebagai validation metric agar alerting dapat diuji tanpa bergantung pada automatic CPU/memory metrics yang memiliki keterbatasan pada Floci.



Alarm dikonfigurasi dengan

- Metric : `ApplicationErrors`
- Namespace : `Enterprise/Application`
- Statistic : `Sum`
- Period : 60 seconds
- Evaluation Periods : 1

- Threshold : > 5
- Comparison : GreaterThanThreshold
- Missing Data : notBreaching

Configuration tersebut sudah berhasil divalidasi.

Alarm juga sudah terhubung ke arn:aws:sns:us-east-1:000000000000:enterprise-alert

SNS topic tersebut memiliki email subscription:

- Protocol : email
- Endpoint : admin@company.com

Namun, validation terhadap runtime alerting belum sepenuhnya berhasil. Setelah metric ApplicationErrors=10 dikirim, alarm pada Floci tetap berada pada state INSUFFICIENT_DATA. Oleh karena itu, alarm configuration berhasil divalidasi, tetapi CloudWatch alarm evaluation dan actual notification delivery belum dapat dianggap production-equivalent pada Floci.

Yang Dimonitor

1. EKS / Kubernetes

1.1 Node

- CPU utilization
- Memory utilization
- Disk usage
- Disk I/O
- Network activity
- Node health

1.2 Pod / Container

- CPU
- Memory
- Pod status
- Container status
- Container restart
- Workload health

2. Application

- Application stdout logs
- Error logs
- Access logs
- Exceptions
- Container logs
- Application-specific metrics

Untuk validation alerting, application-specific metric yang digunakan adalah:

- Namespace : Enterprise/Application
- Metric : ApplicationErrors

Metric ini merupakan custom validation metric, bukan native EKS metric.

3. ALB

Native AWS metrics yang menjadi target monitoring:

- RequestCount
- TargetResponseTime
- HTTPCode_Target_4XX_Count
- HTTPCode_Target_5XX_Count
- HealthyHostCount
- UnHealthyHostCount

ALB resource berhasil divalidasi. Namun query terhadap AWS/ApplicationELB tidak mengembalikan metrics pada Floci.

Status:

- ALB Resource : PASS
- Native ALB Metrics : NOT VERIFIED
- Reason : Floci emulator limitation

4. RDS

Native AWS metrics yang menjadi target monitoring:

- CPUUtilization
- DatabaseConnections
- FreeStorageSpace
- ReadIOPS
- WriteIOPS
- FreeableMemory
- ReadLatency
- WriteLatency

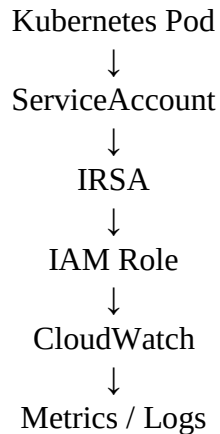
RDS resource berhasil divalidasi. Namun query terhadap AWS/RDS tidak mengembalikan metrics pada Floci.

Status:

- RDS Resource : PASS
- Native RDS Metrics : NOT VERIFIED
- Reason : Floci emulator limitation

Monitoring Security Model

Monitoring tetap menggunakan security boundary berbasis IAM dan Kubernetes ServiceAccount:



CloudWatch Agent menggunakan:

- ServiceAccount : cloudwatch-agent
- IAM Role : CloudWatchAgentRole

Validation sebelumnya membuktikan bahwa Kubernetes ServiceAccount token dapat digunakan untuk explicit STS AssumeRoleWithWebIdentity, dan temporary credentials tersebut dapat digunakan untuk melakukan cloudwatch:PutMetricData. Namun automatic Web Identity credential acquisition oleh CloudWatch Agent mengalami keterbatasan pada Floci. Karena itu, keberhasilan explicit STS exchange tidak dianggap sebagai bukti bahwa automatic CloudWatch Agent credential chain berjalan identik dengan native AWS.

Validation Matrix Phase 9

Layer	Validation	Actual Result	Status
CloudWatch Agent	Package / binary tersedia	Berhasil	☑ PASS
Kubernetes	amazon-cloudwatch namespace	Berhasil	☑ PASS
IAM	CloudWatchAgentRole tersedia	Berhasil	☑ PASS
IRSA	ServiceAccount → IAM Role	Berhasil	☑ PASS
IRSA → STS	Explicit Web Identity exchange	Berhasil	☑ PASS
CloudWatch Agent	DaemonSet deployed	1/1	☑ PASS
CloudWatch Agent	Pod running	1/1 Running	☑ PASS
ContainerInsights	Namespace tersedia	Berhasil	☑ PASS
ContainerInsights	Manual test metric	IRSACloudWatchTest, CredentialBridgeTest	☑ PASS
EKS metrics	Automatic Agent publishing	Tidak terverifikasi	⚠ Floci limitation

Application logs	Container/node log source	Berhasil	☒ PASS
CloudWatch Logs	Automatic Agent ingestion	Tidak terverifikasi	☐ Floci limitation
ALB	Resource	Berhasil	☒ PASS
ALB	Native AWS/ApplicationELB metrics	Empty	☐ Floci limitation
RDS	Resource	Berhasil	☒ PASS
RDS	Native AWS/RDS metrics	Empty	☐ Floci limitation
Application metric	ApplicationErrors	Berhasil	☒ PASS
CloudWatch Alarm	EnterpriseApplicationErrorAlarm	Berhasil	☒ PASS
Alarm configuration	Threshold / period / evaluation	Berhasil	☒ PASS
Alarm → SNS	enterprise-alert	Berhasil	☒ PASS
SNS Topic	enterprise-alert	Berhasil	☒ PASS
SNS Subscription	email → admin@company.com	Berhasil	☒ PASS
SNS Subscription	email → email-lu@gmail.com	Berhasil	☒ PASS
SNS Publish	Test message accepted	MessageId returned	☒ PASS
Alarm evaluation	ApplicationErrors=10 → ALARM	Tetap INSUFFICIENT_DATA	☐ Floci limitation
Actual Email Delivery	Email received by email-lu@gmail.com	Belum dicek/terbukti	☐ NOT VERIFIED
End-to-End Alert	Metric → Alarm → SNS → Email	Belum terverifikasi	☐ NOT VERIFIED

Step 95. Prepare Monitoring Environment

Menyiapkan environment variable dan memastikan kita sedang bekerja terhadap Floci serta cluster EKS yang benar.

95.1 Set Environment. Parameter beserta Fungsinya → **ENDPOINT** adalah Endpoint Floci, **AWS_DEFAULT_REGION** adalah Region simulasi, **AWS_ACCOUNT_ID** adalah Account ID Floci, **EKS_NAMESPACE** adalah Namespace application, **CW_NAMESPACE** adalah Namespace monitoring, **CW_AGENT_ROLE_NAME** adalah IAM role CloudWatch Agent, **CW_AGENT_SA** adalah Kubernetes ServiceAccount agent

```
export ENDPOINT="http://localhost:4566"
```

```
export AWS_DEFAULT_REGION="us-east-1"
```

```
export AWS_ACCOUNT_ID="000000000000"
```

```
export EKS_NAMESPACE="enterprise-app"
```

```

export CW_NAMESPACE="amazon-cloudwatch"
export CW_AGENT_ROLE_NAME="CloudWatchAgentRole"
export CW_AGENT_SA="cloudwatch-agent"

# 95.2 Validate AWS Identity. Expected → Account: 000000000000

aws --endpoint-url="$ENDPOINT" \
    sts get-caller-identity

# 95.3 Validasi Step 95

echo "ENDPOINT=$ENDPOINT"
echo "REGION=$AWS_DEFAULT_REGION"
echo "CW_NAMESPACE=$CW_NAMESPACE"

```

Step 96. Download & Validate Amazon CloudWatch Agent

Step ini menyiapkan Amazon CloudWatch Agent package dari awal. Ada dua hal yang perlu dibedakan:

WSL Host

└─ download/install CloudWatch Agent

digunakan untuk memvalidasi package dan binary. Sedangkan monitoring EKS nanti:

EKS

└─ CloudWatch Agent Pod / DaemonSet

digunakan untuk actual Kubernetes monitoring. Jadi instalasi .deb di WSL bukan berarti WSL host menjadi monitoring agent untuk EKS.

96.1 Download CloudWatch Agent

```

mkdir -p ~/phase9
cd ~/phase9

```

96.2 Download package:

```

wget https://amazoncloudwatch-agent.s3.amazonaws.com/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb

```

96.3 Validate Download

```

ls -lh amazon-cloudwatch-agent.deb

```

96.4 Install CloudWatch Agent

```

sudo dpkg -i amazon-cloudwatch-agent.deb

```

96.5 Validate Installation

```
dpkg -l | grep amazon-cloudwatch-agent
```

Step 97. Create Monitoring Namespace

Memisahkan component monitoring dari application workload.

```
kubectl create namespace amazon-cloudwatch
```

Step 98. Create CloudWatch Agent IAM Policy

Agent membutuhkan permission untuk mengirim metric dan log ke CloudWatch. Untuk baseline, gunakan permission:

cloudwatch:PutMetricData

logs:CreateLogGroup

logs:CreateLogStream

logs:PutLogEvents

logs:DescribeLogStreams

98.1 Buat policy menggunakan command bawah ini, Kenapa Resource: "*" ? PutMetricData memang tidak menggunakan ARN resource seperti resource-based authorization biasa. Untuk lab Floci, kita juga ingin menghindari pembatasan yang belum tentu didukung emulator. Untuk production AWS, policy bisa diperketat pada action/resource yang memang mendukung resource scoping.

```
cat > cloudwatch-agent-policy.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "cloudwatch:PutMetricData"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents",
        "logs:DescribeLogStreams"
      ],
      "Resource": "*"
    }
  ]
}
EOF
```

98.2 Create IAM Role. Catatan penting: OIDC ID di atas mengikuti cluster lu yang sebelumnya sudah divalidasi. Kalau cluster dibuat ulang dan OIDC ID berubah, ambil ulang nilai tersebut.

```
aws --endpoint-url="$ENDPOINT" \  
    iam create-role \  
    --role-name "$CW_AGENT_ROLE_NAME" \  
    --assume-role-policy-document '{  
        "Version": "2012-10-17",  
        "Statement": [  
            {  
                "Effect": "Allow",  
                "Principal": {  
                    "Federated": "arn:aws:iam::000000000000:oidc-  
provider/oidc.eks.us-east-  
1.amazonaws.com/id/50EE392A27204BC383C621D35892EDB9"  
                },  
                "Action": "sts:AssumeRoleWithWebIdentity",  
                "Condition": {  
                    "StringEquals": {  
                        "oidc.eks.us-east-  
1.amazonaws.com/id/50EE392A27204BC383C621D35892EDB9:aud": "sts.amazonaw  
s.com",  
                        "oidc.eks.us-east-  
1.amazonaws.com/id/50EE392A27204BC383C621D35892EDB9:sub": "system:servi  
ceaccount:amazon-cloudwatch:cloudwatch-agent"  
                    }  
                }  
            }  
        ]  
    }'
```

98.3 Attach Policy

```
aws --endpoint-url="$ENDPOINT" \  
    iam put-role-policy \  
    --role-name "$CW_AGENT_ROLE_NAME" \  
    --policy-name CloudWatchAgentPolicy \  
    --policy-document file:///cloudwatch-agent-policy.json
```

98.4 Validasi. Expected → CloudWatchAgentPolicy

```
aws --endpoint-url="$ENDPOINT" \  
  iam list-role-policies \  
  --role-name "$CW_AGENT_ROLE_NAME"
```

Step 99. Create CloudWatch Agent ServiceAccount

Membuat identity Kubernetes untuk CloudWatch Agent.

99.1

```
kubectl create serviceaccount \  
  "$CW_AGENT_SA" \  
  -n "$CW_NAMESPACE"
```

99.2 Annotate dengan IAM role:

```
kubectl annotate serviceaccount \  
  "$CW_AGENT_SA" \  
  -n "$CW_NAMESPACE" \  
  eks.amazonaws.com/role-  
arn="arn:aws:iam::000000000000:role/$CW_AGENT_ROLE_NAME"
```

99.3 Validate. Expected → terdapat annotations:

eks.amazonaws.com/role-arn:

arn:aws:iam::000000000000:role/CloudWatchAgentRole

```
kubectl get serviceaccount \  
  "$CW_AGENT_SA" \  
  -n "$CW_NAMESPACE" \  
  -o yaml
```

Step 100. Create CloudWatch Agent Configuration

Memberitahu agent metric apa yang harus dikumpulkan. Untuk initial lab, jangan langsung memasukkan terlalu banyak metric. Kita mulai dari CPU, Memory, Disk, Network dan Kubernetes/container metrics.

100.1 Buat ConfigMap:

```
cd ~/phase9  
  
cat > cloudwatch-agent-config.yaml <<'EOF'  
apiVersion: v1  
kind: ConfigMap  
metadata:  
  name: cloudwatch-agent-config  
  namespace: amazon-cloudwatch  
data:  
  cwagentconfig.json: |
```

```

{
  "agent": {
    "metrics_collection_interval": 60,
    "run_as_user": 0
  },
  "metrics": {
    "namespace": "ContainerInsights",
    "metrics_collected": {
      "cpu": {
        "measurement": [
          "cpu_usage_idle",
          "cpu_usage_user",
          "cpu_usage_system"
        ],
        "metrics_collection_interval": 60
      },
      "mem": {
        "measurement": [
          "mem_used_percent"
        ],
        "metrics_collection_interval": 60
      },
      "disk": {
        "measurement": [
          "used_percent"
        ],
        "metrics_collection_interval": 60,
        "resources": [
          "*"
        ]
      },
      "diskio": {
        "measurement": [
          "io_time",
          "read_bytes",
          "write_bytes"
        ],
        "metrics_collection_interval": 60
      },
      "net": {
        "measurement": [
          "bytes_sent",
          "bytes_recv"
        ],
        "metrics_collection_interval": 60
      }
    }
  }
}
EOF

```

100.2 Apply

```
kubectl apply -f cloudwatch-agent-config.yaml
```

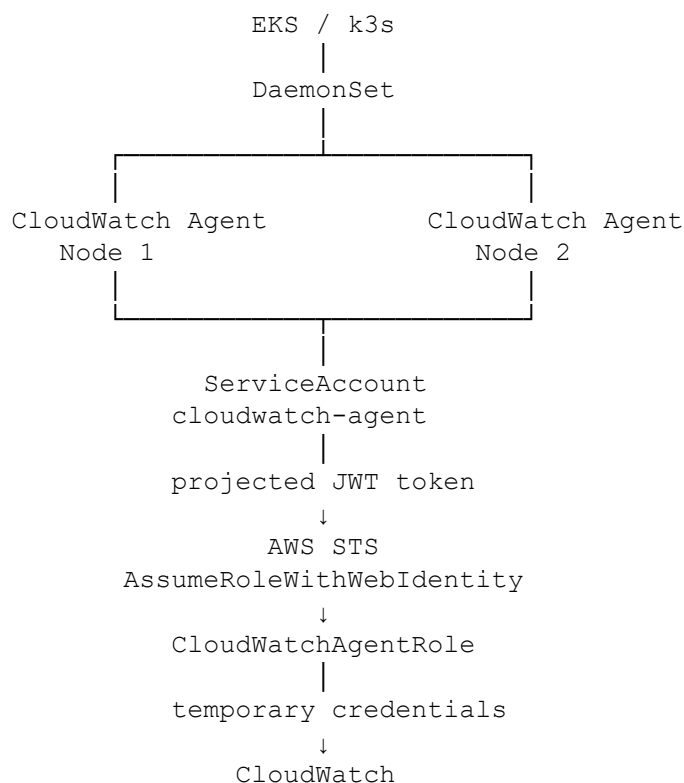
100.3 Validate

```
kubectl get configmap \  
cloudwatch-agent-config \  
-n amazon-cloudwatch
```

Step 101. Deploy CloudWatch Agent DaemonSet with IRSA

Step ini melakukan dua hal sekaligus yaitu Deploy CloudWatch Agent sebagai Kubernetes DaemonSet, sehingga satu Agent Pod berjalan di setiap node EKS. Menghubungkan Agent dengan IAM Role melalui IRSA, sehingga Agent dapat memperoleh temporary AWS credentials melalui Kubernetes ServiceAccount token.

Arsitekturanya:



Dengan ini kita tidak perlu menaruh AWS access key/secret key secara statis di Kubernetes.

101.1 Buat manifest DaemonSet.

```
cat > cloudwatch-agent-daemonset.yaml <<'EOF'  
apiVersion: apps/v1  
kind: DaemonSet  
metadata:  
  name: cloudwatch-agent  
  namespace: amazon-cloudwatch  
spec:  
  selector:  
    matchLabels:  
      app: cloudwatch-agent
```

```

template:
  metadata:
    labels:
      app: cloudwatch-agent
  spec:
    serviceAccountName: cloudwatch-agent
    containers:
      - name: cloudwatch-agent
        image: public.ecr.aws/cloudwatch-agent/cloudwatch-
agent:latest
        resources:
          requests:
            cpu: 50m
            memory: 100Mi
          limits:
            cpu: 250m
            memory: 300Mi
        env:
          - name: AWS_REGION
            value: "us-east-1"

          - name: AWS_ROLE_ARN
            value:
"arn:aws:iam::000000000000:role/CloudWatchAgentRole"
          - name: AWS_WEB_IDENTITY_TOKEN_FILE
            value:
"/var/run/secrets/eks.amazonaws.com/serviceaccount/token"
        volumeMounts:
          - name: config
            mountPath: /etc/cwagentconfig
          - name: rootfs
            mountPath: /rootfs
            readOnly: true
          - name: proc
            mountPath: /host/proc
            readOnly: true
          - name: sys
            mountPath: /host/sys
            readOnly: true

          - name: aws-iam-token
            mountPath:
/var/run/secrets/eks.amazonaws.com/serviceaccount
            readOnly: true
        volumes:
          - name: config
            configMap:
              name: cloudwatch-agent-config
          - name: rootfs
            hostPath:
              path: /
          - name: proc
            hostPath:
              path: /proc

```

```

- name: sys
  hostPath:
    path: /sys
- name: aws-iam-token
  projected:
    sources:
      - serviceAccountToken:
          audience: sts.amazonaws.com
          expirationSeconds: 3600
          path: token

```

EOF

101.2 Penjelasan manifest

A. kind: DaemonSet

Kita tidak menggunakan Deployment karena monitoring agent harus berjalan di setiap node. DaemonSet akan membuat: Node 1 → CloudWatch Agent, Node 2 → CloudWatch Agent, Node 3 → CloudWatch Agent. Jadi agent dapat membaca metric host/node masing-masing.

B. serviceAccountName: cloudwatch-agent

Ini bagian penting untuk security. Pod tidak menggunakan identity Kubernetes default, tetapi ServiceAccount: cloudwatch-agent. ServiceAccount ini sebelumnya sudah kita buat dan diberi annotation: eks.amazonaws.com/role-arn: arn:aws:iam::000000000000:role/CloudWatchAgentRole. Jadi hubungan identity-nya: Pod → cloudwatch-agent ServiceAccount → CloudWatchAgentRole

C. image: public.ecr.aws/cloudwatch-agent/cloudwatch-agent:latest

Ini container image CloudWatch Agent. Jadi .deb yang kita download pada Step 96 bukan yang menjalankan Agent di Kubernetes. Itu penting dibedakan:

Step 96

```

WSL
└── amazon-cloudwatch-agent.deb
    ↓
    validasi package/binary

```

Step 101

```

EKS
└── CloudWatch Agent container
    ↓
    monitoring node/workload

```

Untuk EKS, yang kita gunakan adalah container image.

D. - name: AWS_ROLE_ARN

value: "arn:aws:iam::000000000000:role/CloudWatchAgentRole"

Ini memberi tahu AWS SDK Role IAM mana yang harus digunakan ketika melakukan Web Identity authentication? Role tersebut adalah CloudWatchAgentRole Bukan access key. Ini tetap menggunakan IAM role.

E. - name: AWS_WEB_IDENTITY_TOKEN_FILE
value: "/var/run/secrets/eks.amazonaws.com/serviceaccount/token"

Ini menunjuk ke file JWT ServiceAccount. Flow-nya Kubernetes → JWT token - AWS_WEB_IDENTITY_TOKEN_FILE → AWS SDK → -STS AssumeRoleWithWebIdentity → temporary credentials/. Ini yang sebelumnya hilang dari DaemonSet kita.

F. volumeMounts

F.1 - name: config
mountPath: /etc/cwagentconfig

Mount ConfigMap cloudwatch-agent-config ke /etc/cwagentconfig. Agent kemudian membaca konfigurasi JSON kita dari sana.

F.2 - name: rootfs
mountPath: /rootfs
readOnly: true

Mount filesystem host ke container. Tujuannya agar Agent dapat mengakses informasi filesystem/node host untuk monitoring. readOnly: true penting supaya Agent tidak mendapatkan akses write ke host filesystem.

F.3 - name: proc
mountPath: /host/proc
readOnly: true

/proc berisi informasi runtime Linux seperti CPU, process, memory, system statistics. Agent membutuhkan informasi tersebut untuk host monitoring.

F.4 - name: sys
mountPath: /host/sys
readOnly: true

/sys memberikan informasi kernel/device/system. Sama seperti /proc, kita mount sebagai read-only.

G. aws-iam-token

G.1 - name: aws-iam-token
mountPath: /var/run/secrets/eks.amazonaws.com/serviceaccount
readOnly: true

Ini membuat projected token tersedia di /var/run/secrets/eks.amazonaws.com/serviceaccount/token. Sehingga AWS_WEB_IDENTITY_TOKEN_FILE → /var/run/secrets/eks.amazonaws.com/serviceaccount/token

G.2 - name: aws-iam-token

projected:

sources:

- serviceAccountToken:

audience: sts.amazonaws.com

expirationSeconds: 3600

path: token

menginstruksikan Kubernetes untuk membuat token khusus untuk workload. audience: sts.amazonaws.com Token ditujukan untuk AWS STS karena token ini nantinya digunakan untuk AssumeRoleWithWebIdentity. expirationSeconds: 3600 artinya Token berlaku selama sekitar 3600 seconds = 1 hour, ini lebih aman daripada token statis jangka panjang. path: token artinya Nama file token di dalam mount. Hasil akhirnya /var/run/secrets/eks.amazonaws.com/serviceaccount/token

101.3 Apply DaemonSet. Expected → daemonset.apps/cloudwatch-agent created

```
kubectl apply -f cloudwatch-agent-daemonset.yaml
```

101.4 Validasi Pod

```
kubectl get pods \
```

```
-n amazon-cloudwatch \
```

```
-l app=cloudwatch-agent \
```

```
-o wide
```

101.5 Validasi IRSA environment. Sekarang cek environment yang benar-benar masuk ke Pod. Expected → AWS_REGION=us-east-1, AWS_ROLE_ARN=arn:aws:iam::000000000000:role/CloudWatchAgentRole, AWS_WEB_IDENTITY_TOKEN_FILE=/var/run/secrets/eks.amazonaws.com/serviceaccount/token

```
kubectl get pod \
```

```
-n amazon-cloudwatch \
```

```
-l app=cloudwatch-agent \
```

```
-o jsonpath='{range .items[0].spec.containers[0].env[*]}{.name}={.value}{"\n"}{end}'
```

101.6 Validasi projected token configuration, Expected → ada name: aws-iam-token, projected.sources.serviceAccountToken: audience: sts.amazonaws.com, expirationSeconds: 3600, path: token. Dan juga mountPath: /var/run/secrets/eks.amazonaws.com/serviceaccount, name: aws-iam-token, readOnly: true

```
kubectl get pod \
```

```
-n amazon-cloudwatch \
```

```
-l app=cloudwatch-agent \
```

```
-o yaml \
```

```
| grep -A12 -B3 'aws-iam-token'
```

101.7 Validasi Agent startup, Yang kita cari → Starting AmazonCloudWatchAgent dan tidak ada fatal error seperti failed to start atau configuration validation failed atau permission denied. pesan seperti 169.254.169.254 EC2 metadata fail tidak otomatis berarti failure di environment Floci. Agent memang mencoba mendeteksi apakah environment tersebut EC2/ECS, sedangkan kita menjalankannya pada k3s lokal.

```
kubectl logs \
  -n amazon-cloudwatch \
  daemonset/cloudwatch-agent \
  --since=5m
```

101.8 Validasi STS/IRSA, Karena image CloudWatch Agent tidak memiliki shell, kita jangan kubectl exec ke Agent Pod. Kita validasi mekanisme IRSA menggunakan debug Pod yang memakai ServiceAccount yang sama. Kenapa kita pakai debug Pod? Karena kita ingin mengisolasi ServiceAccount → JWT → STS → IAM Role dari masalah internal image CloudWatch Agent. Ini juga konsisten dengan validasi IRSA yang sebelumnya sudah berhasil di project lu.

```
kubectl run irsa-debug \
  -n amazon-cloudwatch \
  --image=amazon/aws-cli:latest \
  --restart=Never \
  --overrides='
{
  "spec": {
    "serviceAccountName": "cloudwatch-agent",
    "containers": [{
      "name": "aws-cli",
      "image": "amazon/aws-cli:latest",
      "command": ["/bin/sh", "-c", "sleep 3600"],
      "volumeMounts": [{
        "name": "aws-iam-token",
        "mountPath":
"/var/run/secrets/eks.amazonaws.com/serviceaccount",
        "readOnly": true
      }]
    }],
    "volumes": [{
      "name": "aws-iam-token",
      "projected": {
        "sources": [{
          "serviceAccountToken": {
            "audience": "sts.amazonaws.com",
            "expirationSeconds": 3600,
            "path": "token"
          }
        }]
      }
    }]
  }
}
```

```
}'
```

101.9 Validasi token

```
kubectl wait \
  --for=condition=Ready \
  pod/irsa-debug \
  -n amazon-cloudwatch \
  --timeout=60s
```

101.10 Kemudian di bawah ini. Expected → token

```
kubectl exec \
  -n amazon-cloudwatch \
  irsa-debug \
  -- ls -l /var/run/secrets/eks.amazonaws.com/serviceaccount/
```

101.11 Lalu di bawah ini. Expected → eyJhbGciOiJSUzI1NiIs...

```
kubectl exec \
  -n amazon-cloudwatch \
  irsa-debug \
  -- sh -c \
  'head -c 30 /var/run/secrets/eks.amazonaws.com/serviceaccount/token'
```

101.12 Validasi AssumeRoleWithWebIdentity

```
kubectl exec -it \
  -n amazon-cloudwatch \
  irsa-debug \
  -- /bin/sh
```

101.13 Kemudian di bawah ini

```
export AWS_DEFAULT_REGION=us-east-1
TOKEN=$(cat /var/run/secrets/eks.amazonaws.com/serviceaccount/token)
```

```
# 101.14 Lalu di bawah ini. Expected: → { "Credentials": {
"AccessKeyId": "...", "SecretAccessKey": "...", "SessionToken": "...",
}, "AssumedRoleUser": { "Arn": "arn:aws:sts::000000000000:assumed-
role/CloudWatchAgentRole/cloudwatch-agent-validation" } }
```

```
aws \
  --endpoint-url="http://172.17.0.1:4566" \
  sts assume-role-with-web-identity \
  --role-arn arn:aws:iam::000000000000:role/CloudWatchAgentRole \
  --role-session-name cloudwatch-agent-validation \
```

```

--web-identity-token "$TOKEN"

# 101.15 Validate CloudWatch PutMetricData. Masih di dalam irsa-debug:
TOKEN=$(cat /var/run/secrets/eks.amazonaws.com/serviceaccount/token)
CREDS=$(aws \
    --endpoint-url="http://172.17.0.1:4566" \
    sts assume-role-with-web-identity \
    --role-arn arn:aws:iam::000000000000:role/CloudWatchAgentRole \
    --role-session-name cloudwatch-metric-validation \
    --web-identity-token "$TOKEN" \
    --query 'Credentials.[AccessKeyId,SecretAccessKey,SessionToken]' \
    --output text)
set -- $CREDS
export AWS_ACCESS_KEY_ID="$1"
export AWS_SECRET_ACCESS_KEY="$2"
export AWS_SESSION_TOKEN="$3"
export AWS_DEFAULT_REGION=us-east-1

# 101.15.1 Validasi identity. Expected → Arn:
arn:aws:sts::000000000000:assumed-role/CloudWatchAgentRole/cloudwatch-
metric-validation

aws \
    --endpoint-url="http://172.17.0.1:4566" \
    sts get-caller-identity

# 101.15.2 Lalu ketik di bawah ini, Expected → No output

aws \
    --endpoint-url="http://172.17.0.1:4566" \
    cloudwatch put-metric-data \
    --namespace ContainerInsights \
    --metric-data MetricName=IRSACloudWatchTest,Value=1,Unit=Count

# 101.15.3 Validate Metric. Masih di debug Pod. Cari →
IRSACloudWatchTest. Kalau muncul IRSACloudWatchTest maka: CloudWatch
API + IAM permission + temporary credentials = PASS.

aws \
    --endpoint-url="http://172.17.0.1:4566" \
    cloudwatch list-metrics \
    --namespace ContainerInsights

```

```
# 101.16 Cleanup debug Pod.
```

```
exit
```

```
# 101.17 Kemudian dari WSL ketik di bawah ini. Debug Pod hanya untuk
membuktikan authentication path, bukan bagian permanent architecture.
```

```
kubectl delete pod \
  irsa-debug \
  -n amazon-cloudwatch
```

Step 102. Validate CloudWatch Metric Submission

Tujuan step ini adalah memastikan apakah metric berhasil tersedia di CloudWatch dan menentukan apakah metric tersebut berasal dari CloudWatch Agent atau hanya dari validasi manual menggunakan temporary credentials. Pada tahap ini, kita sudah membuktikan bahwa:

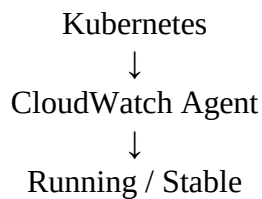
- IRSA → STS berhasil.
- Temporary credentials berhasil diperoleh.
- CloudWatchAgentRole memiliki akses cloudwatch:PutMetricData.
- PutMetricData manual berhasil.
- Metric IRSACloudWatchTest berhasil muncul di namespace ContainerInsights.
- Metric CredentialBridgeTest juga berhasil dikirim menggunakan temporary credentials dari STS.

Namun, CloudWatch Agent sendiri masih mengalami masalah credential provider pada environment Floci. Agent gagal memperoleh credentials melalui automatic WebIdentity provider dan kemudian melakukan fallback ke shared credentials file /root/.aws/credentials, yang tidak tersedia.

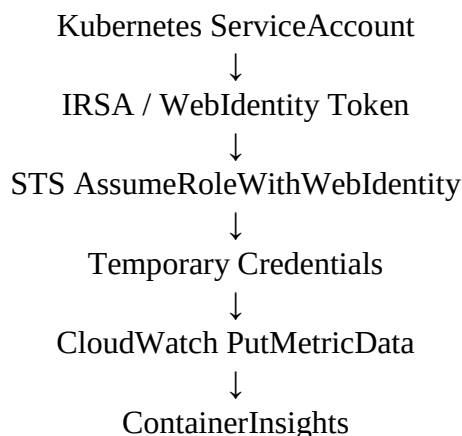
Validation	Namespace/Component	Status
ValidationResultContainerInsights namespace	Container Insights	☑ PASS
Manual PutMetricData	CloudWatch	☑ PASS
IRSACloudWatchTest visible	IRSA / CloudWatch	☑ PASS
CredentialBridgeTest visible	Credential Bridge	☑ PASS
CloudWatch Agent Pod Running	CloudWatch Agent	☑ PASS
CloudWatch Agent automatic WebIdentity	CloudWatch Agent / WebIdentity	✗ FAIL
FlociAgent PutMetricData	FlociAgent / CloudWatch	✗ FAIL
Agent CPU/Memory/Disk/Network metrics	CloudWatch Agent	🕒 Belum tersedia

Kesimpulan → Step 102 PARTIAL PASS

CloudWatch API, IAM authorization, IRSA, STS temporary credentials, dan manual metric submission telah berhasil divalidasi. Namun, CloudWatch Agent belum dapat melakukan automatic metric submission karena credential provider WebIdentity pada CloudWatch Agent tidak kompatibel dengan implementasi Floci yang digunakan pada environment lab ini. Agent kemudian melakukan fallback ke `/root/.aws/credentials`, tetapi file tersebut tidak tersedia. IRSACloudWatchTest dan CredentialBridgeTest tidak boleh dianggap sebagai metric yang berasal dari CloudWatch Agent. Keduanya merupakan metric hasil validasi manual menggunakan temporary credentials. Keterbatasan ini dicatat sebagai Floci emulator limitation. Tidak dilakukan penambahan credential bridge atau komponen workaround tambahan hanya untuk meniru credential chain native AWS. Validasi CloudWatch Agent pada level deployment dan process stability tetap berhasil:



Sedangkan jalur credential dan CloudWatch API berhasil divalidasi secara eksplisit:



Dengan demikian, Step 102 ditutup sebagai PARTIAL PASS dengan Floci limitation. Native Agent-generated CPU, Memory, Disk, dan Network metrics tidak digunakan sebagai dependency untuk melanjutkan architecture/security validation pada phase berikutnya.

```
# 102.1 Check ContainerInsights Metrics. Expected → Metric
IRSACloudWatchTest muncul: { "Metrics": [ { "Namespace":
"ContainerInsights", "MetricName": "IRSACloudWatchTest", "Dimensions":
[] } ] }. Interpretasi → IRSACloudWatchTest membuktikan bahwa CloudWatch
API dapat menerima metric menggunakan temporary credentials yang
diperoleh melalui AssumeRoleWithWebIdentity. Metric ini belum
membuktikan CloudWatch Agent berhasil melakukan submission, karena
metric tersebut dibuat secara manual pada Step 101.
```

```
aws \
  --endpoint-url="$ENDPOINT" \
  cloudwatch list-metrics \
```

```
--namespace ContainerInsights
```

102.2 Query Metric Name. Gunakan query sederhana agar kompatibel dengan `--output table`. Expected saat ini → `IRSACloudWatchTest`. Jika hanya `IRSACloudWatchTest` yang muncul, berarti belum ada bukti metric dari `CloudWatch Agent`.

```
aws \

--endpoint-url="$ENDPOINT" \

cloudwatch list-metrics \

--namespace ContainerInsights \

--query 'Metrics[*].MetricName' \

--output table
```

102.3 Check CloudWatch Agent Submission. Periksa log Agent. Pada environment `Floci` saat ini, ditemukan `"E! cloudwatch: WriteToCloudWatch failure, err: SharedCredsLoad: failed to load shared credentials file"` dan `"E! cloudwatch: code: SharedCredsLoad, message: failed to load shared credentials file original error: FailedRead: unable to open file caused by: open /root/.aws/credentials: no such file or directory"`. Interpretasi → `CloudWatch Agent` belum berhasil melakukan metric submission. Agent telah gagal memperoleh credentials melalui automatic `WebIdentity` credential provider dan kemudian melakukan fallback ke `/root/.aws/credentials`. File tersebut tidak tersedia di dalam Agent Pod sehingga proses `PutMetricData` gagal.

```
kubectl logs \

-n amazon-cloudwatch \

daemonset/cloudwatch-agent \

--since=10m
```

102.4 Validate Agent Status. Interpretasi → `Running` menunjukkan process `CloudWatch Agent` berjalan stabil, tetapi tidak berarti metric berhasil dikirim ke `CloudWatch`.

```
kubectl get pods \

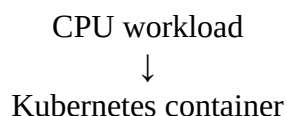
-n amazon-cloudwatch \

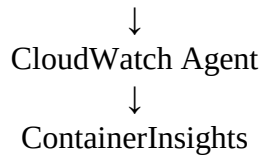
-l app=cloudwatch-agent \

-o wide
```

Step 103. Validate EKS Workload Metrics

Memastikan monitoring benar-benar melihat workload Kubernetes, bukan cuma agent. Tujuan test :





103.1 Cek workload:

```
kubectl get pods -A
```

103.2 Lalu generate CPU activity sederhana:

```
kubectl run monitoring-load \
  -n enterprise-app \
  --image=busybox:latest \
  --restart=Never \
  --command -- \
  /bin/sh -c 'while true; do x=1; done'
```

103.3 Check:

```
kubectl get pod monitoring-load \
  -n enterprise-app
```

103.4 Biarkan berjalan sekitar 60-120 detik. Kemudian cari metric Container Insights:

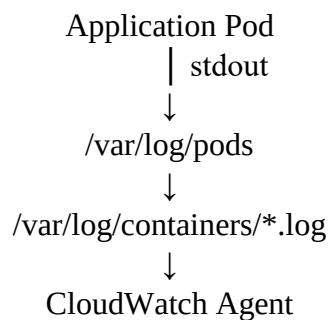
```
aws --endpoint-url="$ENDPOINT" \
  cloudwatch list-metrics \
  --namespace ContainerInsights
```

103.5 Setelah selesai:

```
kubectl delete pod monitoring-load \
  -n enterprise-app
```

Step 104. Validate Application Log Collection to CloudWatch Logs

Tujuan step ini adalah memvalidasi pipeline application log dari Kubernetes workload sampai ke CloudWatch Logs. Arsitektur yang divalidasi:



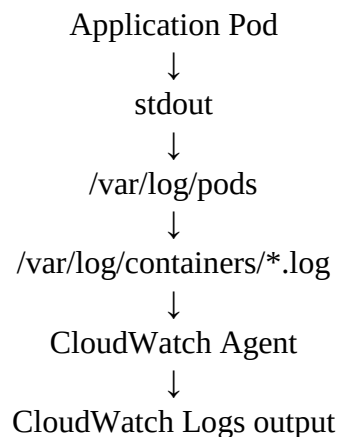
↓
CloudWatch Logs

Pada Floci, tahap AWS authentication juga divalidasi untuk menentukan apakah CloudWatch Agent dapat menggunakan IRSA secara otomatis.

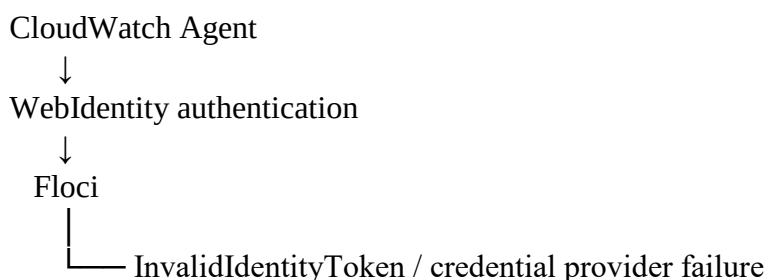
Validation	Result
Application test Pod Running	☑ PASS
Application stdout logs generated	☑ PASS
/var/log/pods contains application log	☑ PASS
/var/log/containers symlink exists	☑ PASS
CloudWatch Agent /var/log mount	☑ PASS
CloudWatch Agent logs configuration	☑ PASS
CloudWatch Agent loads log configuration	☑ PASS
CloudWatch Agent remains Running	☑ PASS
Agent attempts CloudWatch Logs output	☑ PASS
Agent automatic WebIdentity authentication	✗ FAIL
CloudWatch Log Group created	✗ BLOCKED
CloudWatch Log Stream created	✗ BLOCKED
APPLICATION_LOG_TEST visible in CloudWatch	✗ BLOCKED

Conclusion → Step 104 PARTIAL PASS

The Kubernetes application logging pipeline has been successfully validated up to the CloudWatch Logs output stage. Validated path:



The remaining failure occurs during AWS credential acquisition:



↓
 /root/.aws/credentials fallback
 └─ file does not exist

Therefore, the absence of the CloudWatch Log Group and Log Stream is a consequence of the authentication failure, not evidence that the Kubernetes log collection configuration is incorrect. The same Floci WebIdentity limitation was previously observed during CloudWatch metric submission in Step 102. No credential bridge is introduced, because doing so would no longer validate the intended IRSA → CloudWatch Agent authentication path.

104.1 Create Test Application Pod. Buat Pod sederhana yang menghasilkan application log secara periodik.

```
kubectl apply -f - <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: application-log-test
  namespace: enterprise-app
  labels:
    app: application-log-test
spec:
  containers:
    - name: app
      image: busybox:1.36
      command:
        - /bin/sh
        - -c
        - |
          i=1
          while true; do
            echo "APPLICATION_LOG_TEST message=$i timestamp=$(date -
Iseconds)"
            i=$((i+1))
            sleep 5
          done
EOF
```

104.2 Validate Application Pod

```
kubectl get pod \
  -n enterprise-app \
  application-log-test \
  -o wide
```

104.3 Validate Application stdout Logs. Expected: → APPLICATION_LOG_TEST message=1 timestamp=... APPLICATION_LOG_TEST message=2 timestamp=... APPLICATION_LOG_TEST message=3 timestamp=.... Nomor message dan timestamp akan berbeda.

```
kubectl logs \
```

```
-n enterprise-app \
application-log-test \
--since=1m
```

104.4 Validate Kubernetes Node Log Files. CloudWatch Agent membaca log container dari filesystem node.

```
kubect1 run node-log-debug \
  -n amazon-cloudwatch \
  --image=busybox:1.36 \
  --restart=Never \
  --overrides='
{
  "spec": {
    "nodeName": "ee5da7bc52db",
    "containers": [{
      "name": "debug",
      "image": "busybox:1.36",
      "command": ["/bin/sh", "-c", "sleep 3600"],
      "volumeMounts": [
        {
          "name": "host-var-log",
          "mountPath": "/host/var/log",
          "readOnly": true
        }
      ]
    }],
    "volumes": [
      {
        "name": "host-var-log",
        "hostPath": {
          "path": "/var/log"
        }
      }
    ]
  }
}'
```

104.4.1 Validasi directory. Pastikan terdapat file seperti → **application-log-test_enterprise-app_app-<container-id>.log**

```
kubect1 exec \
  -n amazon-cloudwatch \
  node-log-debug \
  -- ls -lah /host/var/log/containers
```

104.4.2 Kemudian validasi actual log file. Expected terdapat → **APPLICATION_LOG_TEST message=1 APPLICATION_LOG_TEST message=2 APPLICATION_LOG_TEST message=3.** Kubernetes application stdout telah tersedia pada node melalui /var/log/pods dan direferensikan melalui /var/log/containers/*.log

```
kubect1 exec \
  -n amazon-cloudwatch \
```

```
node-log-debug \  
  
-- sh -c \  
  
'cat /host/var/log/pods/enterprise-app_application-log-  
test_*/app/0.log'
```

**# 104.5 Configure CloudWatch Agent Log Collection. CloudWatch Agent
dikonfigurasi untuk membaca Kubernetes container logs.**

```
kubectl apply -f - <<'EOF'  
apiVersion: v1  
kind: ConfigMap  
metadata:  
  name: cloudwatch-agent-config  
  namespace: amazon-cloudwatch  
data:  
  cwagentconfig.json: |  
    {  
      "agent": {  
        "metrics_collection_interval": 60,  
        "run_as_user": 0  
      },  
      "metrics": {  
        "namespace": "ContainerInsights",  
        "metrics_collected": {  
  
          "cpu": {  
            "measurement": [  
              "cpu_usage_idle",  
              "cpu_usage_user",  
              "cpu_usage_system"  
            ],  
            "metrics_collection_interval": 60  
          },  
          "mem": {  
            "measurement": [  
              "mem_used_percent"  
            ],  
            "metrics_collection_interval": 60  
          },  
          "disk": {  
            "measurement": [  
              "used_percent"  
            ],  
            "metrics_collection_interval": 60,  
            "resources": [  
              "*"   
            ]  
          },  
          "diskio": {  
            "measurement": [  
              "io_time",  
              "read_bytes",  
              "write_bytes"  
            ],  
            "metrics_collection_interval": 60
```

```

    },
    "net": {
      "measurement": [
        "bytes_sent",
        "bytes_recv"
      ],
      "metrics_collection_interval": 60
    }
  }
},
"logs": {
  "logs_collected": {
    "files": {
      "collect_list": [
        {
          "file_path": "/var/log/containers/*.log",
          "log_group_name": "/aws/containerinsights/enterprise-
app/application",
          "log_stream_name": "{hostname}",
          "timezone": "UTC"
        }
      ]
    }
  }
}
}
EOF

```

104.5.1 Validasi konfigurasi. Expected terdapat → "logs": { "logs_collected": { "files": { "collect_list": [{ "file_path": "/var/log/containers/*.log". INFO: Jangan menggunakan append_dimensions. Konfigurasi tersebut sebelumnya menyebabkan CloudWatch Agent mengaktifkan ec2tagger, yang mencoba mengakses EC2 Instance Metadata Service dan menyebabkan masalah pada environment Floci/k3s.

```

kubectl get configmap \
  -n amazon-cloudwatch \
  cloudwatch-agent-config \
  -o jsonpath='{.data.cwagentconfig\.json}' \
  | grep -A15 '"logs"'

```

104.6 Configure CloudWatch Agent /var/log Access/ CloudWatch Agent harus mempunyai akses read-only terhadap /var/log milik node. Pastikan DaemonSet mempunyai volume mount berikut → volumeMount.name: varlog, mountPath: /var/log, readOnly: true dan volumes.name: varlog, hostPath.path: /var/log

```

cat <<EOF > cloudwatch-agent-daemonset.yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: cloudwatch-agent

```

```

    namespace: amazon-cloudwatch
spec:
  selector:
    matchLabels:
      app: cloudwatch-agent
  template:
    metadata:
      labels:
        app: cloudwatch-agent
    spec:
      serviceAccountName: cloudwatch-agent
      containers:
        - name: cloudwatch-agent
          image: public.ecr.aws/cloudwatch-agent/cloudwatch-
agent:latest
          resources:
            requests:
              cpu: 50m
              memory: 100Mi
            limits:
              cpu: 250m
              memory: 300Mi
          env:
            - name: AWS_REGION
              value: "us-east-1"
            - name: AWS_ROLE_ARN
              value:
"arn:aws:iam::000000000000:role/CloudWatchAgentRole"
            - name: AWS_WEB_IDENTITY_TOKEN_FILE
              value:
"/var/run/secrets/eks.amazonaws.com/serviceaccount/token"
          volumeMounts:
            # CloudWatch Agent configuration
            - name: config
              mountPath: /etc/cwagentconfig
            # Host filesystem
            - name: rootfs
              mountPath: /rootfs
              readOnly: true
            # Host proc
            - name: proc
              mountPath: /host/proc
              readOnly: true
            # Host sys
            - name: sys
              mountPath: /host/sys
              readOnly: true
            # Kubernetes container logs
            - name: varlog
              mountPath: /var/log
              readOnly: true
            # Projected ServiceAccount token for IRSA
            - name: aws-iam-token
              mountPath:
/var/run/secrets/eks.amazonaws.com/serviceaccount

```

```

        readOnly: true
volumes:
  # CloudWatch Agent configuration
  - name: config
    configMap:
      name: cloudwatch-agent-config

  # Host root filesystem
  - name: rootfs
    hostPath:
      path: /
  # Host proc
  - name: proc
    hostPath:
      path: /proc
  # Host sys
  - name: sys
    hostPath:
      path: /sys

  # Kubernetes container logs
  - name: varlog
    hostPath:
      path: /var/log

  # Projected ServiceAccount token
  - name: aws-iam-token
    projected:
      sources:
        - serviceAccountToken:
            audience: sts.amazonaws.com
            expirationSeconds: 3600
            path: token

```

EOF

104.6.1 Apply DaemonSet. Expected → daemonset.apps/cloudwatch-agent configured

```
kubectl apply -f cloudwatch-agent-daemonset.yaml
```

104.7 Restart and Validate CloudWatch Agent

```
kubectl rollout restart daemonset/cloudwatch-agent \
  -n amazon-cloudwatch
```

104.7.1 Tunggu rollout. Expected → daemon set "cloudwatch-agent" successfully rolled out

```
kubectl rollout status daemonset/cloudwatch-agent \
  -n amazon-cloudwatch
```

104.7.2 Validasi Pod:

```
kubectl get pods \
  -n amazon-cloudwatch \
```

```

-l app=cloudwatch-agent \

-o wide

# 104.8 Validate Agent Log Collection. Periksa log CloudWatch Agent.
Pada environment Floci, hasil menunjukkan CloudWatch Agent mencoba
mengirim application logs ke CloudWatch Logs, tetapi gagal memperoleh
AWS credentials secara otomatis [outputs.cloudwatchlogs] Aws error
received when sending logs to /aws/containerinsights/enterprise-
app/application/cloudwatch-agent-xxxxxx: SharedCredsLoad: failed to
load shared credentials file caused by: FailedRead: unable to open
file caused by: open /root/.aws/credentials: no such file or directory

kubectl logs \

-n amazon-cloudwatch \

daemonset/cloudwatch-agent \

--since=5m

# 104.8.1 CloudWatch Agent tetap running. Expected/current result:
NAME cloudwatch-agent-xxxxxx; STATUS Running; READY true; RESTARTS 0.
Result: PARTIAL PASS. Application log collection configuration is
loaded successfully and the Agent attempts to send logs to CloudWatch
Logs.. However, automatic AWS credential acquisition by the CloudWatch
Agent fails under Floci. The failure occurs at CloudWatch Agent → AWS
credential acquisition → WebIdentity provider → Floci WebIdentity
validation -FAIL→ fallback to /root/.aws/credentials (file does not
exist). This is consistent with the WebIdentity limitation already
identified during Step 102.

kubectl get pods \

-n amazon-cloudwatch \

-l app=cloudwatch-agent \

-o custom-
columns='NAME:.metadata.name,STATUS:.status.phase,READY:.status.contai
nerStatuses[0].ready,RESTARTS:.status.containerStatuses[0].restartCoun
t'

# 104.9 Validate CloudWatch Log Group. Check CloudWatch Logs. Current
result → DescribeLogGroups : /aws/ecr/registry,
/aws/rds/instance/enterprise-postgres/error. The expected application
log group: /aws/containerinsights/enterprise-app/application does not
exist. Result: NOT PASS / BLOCKED. The log group was not created
because CloudWatch Agent could not authenticate successfully to the
CloudWatch Logs API. This is not interpreted as a configuration
failure because the Agent already reached the outputs.cloudwatchlogs
stage.

aws \

--endpoint-url="$ENDPOINT" \

logs describe-log-groups \

--query 'logGroups[*].logGroupName' \

```

```

--output table

# 104.10 Validate CloudWatch Log Stream Attempt to retrieve the
expected log stream. Expected in a successful environment →
DescribeLogStreams : cloudwatch-agent-xxxxxx. Actual result →
ResourceNotFoundException: The specified log group does not exist
/aws/containerinsights/enterprise-app/application. Result: BLOCKED. No
log stream can exist because the log group itself was not created.

aws \

--endpoint-url="$ENDPOINT" \
logs describe-log-streams \
--log-group-name \
/aws/containerinsights/enterprise-app/application \
--query 'logStreams[*].logStreamName' \
--output table

# 104.11 Validate Actual Application Log in CloudWatch. If the log
group existed, the final validation would be below here. Expected
successful result → APPLICATION_LOG_TEST message=1
APPLICATION_LOG_TEST message=2 APPLICATION_LOG_TEST message=3.
However, because the log group does not exist, this validation cannot
succeed in the current Floci environment. Result: BLOCKED. The
application log was successfully generated and made available to
CloudWatch Agent, but end-to-end delivery to CloudWatch Logs could not
be completed because of the CloudWatch Agent WebIdentity credential
limitation.

aws \

--endpoint-url="$ENDPOINT" \
logs filter-log-events \
--log-group-name \
/aws/containerinsights/enterprise-app/application \
--filter-pattern 'APPLICATION_LOG_TEST'

# 104.14 Cleanup

kubectl delete pod \
-n amazon-cloudwatch \
node-log-debug

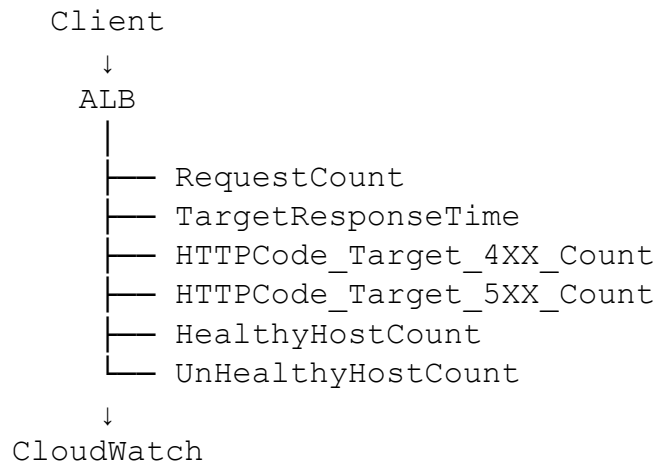
kubectl delete pod \
-n enterprise-app \
application-log-test

```

Step 105 — Validate ALB Metrics

Application Load Balancer memiliki native CloudWatch metrics. ALB tidak membutuhkan CloudWatch Agent untuk mempublikasikan metric karena metric ALB secara native dipublikasikan oleh AWS ke CloudWatch.

Architecture:



Step ini bertujuan memvalidasi apakah Floci mengemulasikan native ALB → CloudWatch metrics.

Item	Expected real AWS	Actual Floci
enterprise-alb	ALB exists	☑ Exists
AWS/ApplicationELB namespace	Contains ALB metrics	✗ Empty
RequestCount	Available	✗ Not returned
TargetResponseTime	Available	✗ Not returned
HTTPCode_Target_4XX_Count	Available	✗ Not returned
HTTPCode_Target_5XX_Count	Available	✗ Not returned
HealthyHostCount	Available	✗ Not returned
UnHealthyHostCount	Available	✗ Not returned

105.1 Validate ALB Exists. Expected → ALB enterprise-alb ditemukan dan mempunyai status aktif. Actual Floci → ALB enterprise-alb sudah tersedia dan sebelumnya telah berhasil divalidasi pada phase ALB/HTTPS.

```

aws \
  --endpoint-url="$ENDPOINT" \
  elbv2 describe-load-balancers \
  --names enterprise-alb \
  --query 'LoadBalancers[*].[LoadBalancerName,DNSName,State.Code]' \
  --output table
  
```

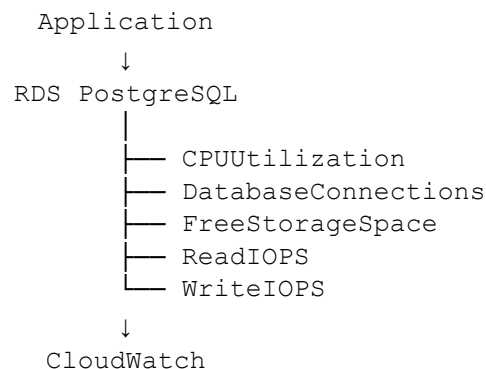
105.2 Query Native ALB CloudWatch Metrics, Expected → Jika native ALB metrics didukung, CloudWatch seharusnya mengembalikan metric pada namespace AWS/ApplicationELB. Metric yang diharapkan antara ,lain

RequestCount, TargetResponseTime, HTTPCode_Target_4XX_Count, HTTPCode_Target_5XX_Count, HealthyHostCount, UnHealthyHostCount. sedangkan floci hanya menghasilkan "ListMetrics" tanpa ada metric yang dikembalikan.

```
aws \
  --endpoint-url="$ENDPOINT" \
  cloudwatch list-metrics \
  --namespace AWS/ApplicationELB \
  --output table
```

Step 106. Validate RDS Metrics

Amazon RDS memiliki native CloudWatch metrics. RDS tidak membutuhkan CloudWatch Agent untuk mempublikasikan metric karena metric database dipublikasikan secara native ke CloudWatch. Architecture:



Step ini bertujuan memvalidasi apakah Floci mengemulasikan native RDS → CloudWatch metrics.

Item	Expected	Actual Floci
enterprise-postgres	RDS exists	☒ Exists
AWS/RDS namespace	Contains RDS metrics	☒ Empty
CPUUtilization	Available	☒ Not returned
DatabaseConnections	Available	☒ Not returned
FreeStorageSpace	Available	☒ Not returned
ReadIOPS	Available	☒ Not returned
WriteIOPS	Available	☒ Not returned

106.1 Validate RDS Instance Exists. Expected → DescribeDBInstances: enterprise-postgres available, postgres. Actual Floci → RDS instance enterprise-postgres sudah tersedia pada project dan sebelumnya telah digunakan sebagai database layer.

```
aws \
  --endpoint-url="$ENDPOINT" \
```

```

rds describe-db-instances \
--db-instance-identifier enterprise-postgres \
--query
'DBInstances[*].[DBInstanceIdentifier,DBInstanceStatus,Engine]' \
--output table

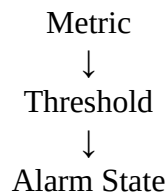
# 106.2 Query Native RDS CloudWatch Metrics. Expected → Jika native
RDS metrics didukung, namespace AWS/RDS. seharusnya mengembalikan
metrics RDS. Metric yang diharapkan → CPUUtilization,
DatabaseConnections, FreeStorageSpace, ReadIOPS, WriteIOPS. Actual
floci → tidak mengembalikan metric RDS.

aws \
--endpoint-url="$ENDPOINT" \
cloudwatch list-metrics \
--namespace AWS/RDS \
--output table

```

Step 107. Create CloudWatch Alarm

Sekarang baru kita masuk ke alerting. Monitoring hanya memberitahu kondisi. Alarm memberikan:



Buat alarm untuk workload metric yang benar-benar tersedia. Untuk contoh awal, kita gunakan custom test metric supaya kita tidak bergantung pada CPU metric Floci.

107.1 Create Test Metric

```

aws --endpoint-url="$ENDPOINT" \
cloudwatch put-metric-data \
--namespace "Enterprise/Application" \
--metric-data \
MetricName=ApplicationErrors,Value=0,Unit=Count

```

107.1 Validasi. Expected → ApplicationErrors

```

aws --endpoint-url="$ENDPOINT" \
cloudwatch list-metrics \
--namespace "Enterprise/Application"

```

107.2 Create Alarm. Parameter beserta Artinya → --alarm-name adalah Nama alarm, --metric-name adalah Metric yang diamati, --namespace Namespace metric, --statistic Sum adalah Menjumlahkan error, --period 60 adalah Window 60 detik, --threshold 5 adalah Alarm jika >5, --evaluation-periods 1 adalah Satu period cukup, --treat-missing-data adalah Missing data dianggap tidak breach.

```
aws --endpoint-url="$ENDPOINT" \  
    cloudwatch put-metric-alarm \  
    --alarm-name EnterpriseApplicationErrorAlarm \  
    --metric-name ApplicationErrors \  
    --namespace Enterprise/Application \  
    --statistic Sum \  
    --period 60 \  
    --threshold 5 \  
    --comparison-operator GreaterThanThreshold \  
    --evaluation-periods 1 \  
    --treat-missing-data notBreaching
```

107.3 Validate Alarm. Expected → EnterpriseApplicationErrorAlarm

```
aws --endpoint-url="$ENDPOINT" \  
    cloudwatch describe-alarms \  
    --alarm-names EnterpriseApplicationErrorAlarm
```

Step 108. Create SNS Alert Topic

Alarm harus memiliki notification destination.

108.1 Buat topic. Expected → arn:aws:sns:us-east-1:000000000000:enterprise-alert

```
SNS_TOPIC_ARN=$(  
    aws --endpoint-url="$ENDPOINT" \  
        sns create-topic \  
        --name enterprise-alert \  
        --query 'TopicArn' \  
        --output text  
)  
  
export SNS_TOPIC_ARN  
echo "$SNS_TOPIC_ARN"
```

108.2 Validate Topic. Cari enterprise-alert

```
aws --endpoint-url="$ENDPOINT" \  
    sns list-topics
```

Step 109. Subscribe Notification Endpoint

SNS subscription created → PASS

Actual email delivery → emulator-dependent

109.1 Kalau Floci mendukung email subscription ketik command di bawah ini. Tapi jangan menganggap email benar-benar terkirim hanya karena subscription berhasil dibuat.

```
aws --endpoint-url="$ENDPOINT" \  
    sns subscribe \  
        --topic-arn "$SNS_TOPIC_ARN" \  
        --protocol email \  
        --notification-endpoint admin@company.com
```

109.2 Validasi. Expected → Protocol: email, Endpoint: admin@company.com

```
aws --endpoint-url="$ENDPOINT" \  
    sns list-subscriptions-by-topic \  
        --topic-arn "$SNS_TOPIC_ARN"
```

Step 110. Connect CloudWatch Alarm to SNS

110.1 Update alarm:

```
aws --endpoint-url="$ENDPOINT" \  
    cloudwatch put-metric-alarm \  
        --alarm-name EnterpriseApplicationErrorAlarm \  
        --metric-name ApplicationErrors \  
        --namespace Enterprise/Application \  
        --statistic Sum \  
        --period 60 \  
        --threshold 5 \  
        --comparison-operator GreaterThanThreshold \  
        --evaluation-periods 1 \  
        --treat-missing-data notBreaching \  
        --alarm-actions "$SNS_TOPIC_ARN"
```

```
# 110.2 Validasi. Expected → EnterpriseApplicationErrorAlarm OK
arn:aws:sns:.... Actual floci → Alarm Resource: PASS, Alarm Action →
SNS: PASS, Alarm State: INSUFFICIENT_DATA
```

```
aws --endpoint-url="$ENDPOINT" \
    cloudwatch describe-alarms \
    --alarm-names EnterpriseApplicationErrorAlarm \
    --query 'MetricAlarms[0].[AlarmName,StateValue,AlarmActions]' \
    --output table
```

Step 111. Trigger Alarm

Sekarang kita sengaja mengirim metric di atas threshold. Kita mengirim 10 errors sementara threshold 5. Jadi $10 > 5$

```
aws --endpoint-url="$ENDPOINT" \
    cloudwatch put-metric-data \
    --namespace "Enterprise/Application" \
    --metric-data \
        MetricName=ApplicationErrors,Value=10,Unit=Count
```

111.1 Check Alarm State

```
sleep 70
```

```
# 111.1.1 Kemudian di bawah ini. Expected ideal →
EnterpriseApplicationErrorAlarm : ALARM Threshold Crossed. Actual
floci → tetap INSUFFICIENT_DATA. NOT VERIFIED – Floci CloudWatch alarm
evaluation behavior
```

```
aws --endpoint-url="$ENDPOINT" \
    cloudwatch describe-alarms \
    --alarm-names EnterpriseApplicationErrorAlarm \
    --query 'MetricAlarms[0].[AlarmName,StateValue,StateReason]' \
    --output table
```

Step 112. End-to-End Monitoring Validation

```
kubectl get pods -n amazon-cloudwatch
kubectl get daemonset -n amazon-cloudwatch
aws --endpoint-url="$ENDPOINT" \
    cloudwatch list-metrics \
    --namespace ContainerInsights
aws --endpoint-url="$ENDPOINT" \
```

```

cloudwatch list-metrics \
--namespace AWS/ApplicationELB
aws --endpoint-url="$ENDPOINT" \
cloudwatch list-metrics \
--namespace AWS/RDS
aws --endpoint-url="$ENDPOINT" \
cloudwatch describe-alarms
aws --endpoint-url="$ENDPOINT" \
sns list-topics

```

Phase 10 — Audit, Compliance & Threat Detection

Tujuan PhaPhase ini membangun security visibility dan threat detection layer untuk environment enterprise. Setelah Phase 9, kita sudah punya Application, EKS, ALB, RDS, S3, IAM, KMS, Secrets Manager, CloudWatch, SNS. Sekarang kita jawab tiga pertanyaan:

1. Apa yang terjadi di environment?



CloudTrail

2. Apakah resource memenuhi security baseline?



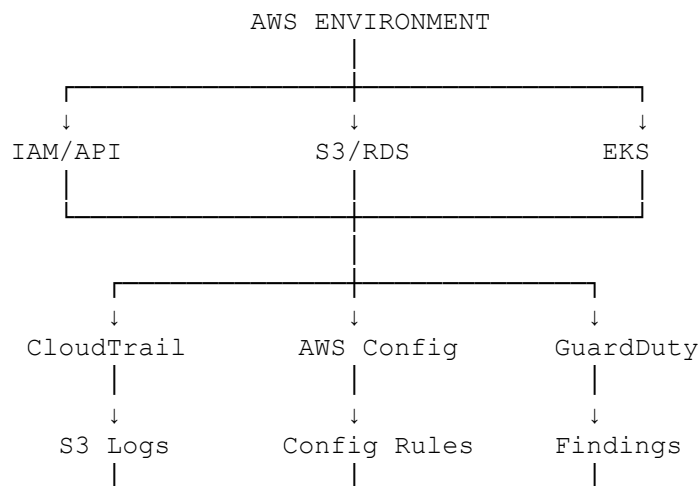
AWS Config

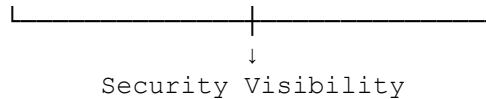
3. Apakah ada indikasi aktivitas berbahaya?



GuardDuty

Architecture Phase 10:





Security model

CloudTrail = "What happened?"

AWS Config = "Is the resource configured securely?"

GuardDuty = "Does the activity look malicious?"

Validation Matrix Phase 10

Layer	Validation	Expected	Status
CloudTrail	Trail exists	Trail ditemukan	PASS
CloudTrail	Multi-region	TRUE	PASS
CloudTrail	Global events	TRUE	PASS
CloudTrail	Log file validation	TRUE	PASS
CloudTrail	Trail status	Logging True	PASS
CloudTrail	S3 delivery	File .json.gz tersedia	PASS
CloudTrail	Event lookup	Event dapat dicari	NOT VERIFIED / Floci
Config	Recorder	Exists	PASS
Config	Recording	True / SUCCESS	PASS
Config	Delivery Channel	Created	PASS
Config	Config Rules	4 rules exists	PASS
Config	Compliance evaluation	COMPLIANT/NON_COMPLIANT	NOT VERIFIED
Config	PutEvaluations	Supported	NOT SUPPORTED
RDS	Private accessibility	FALSE	PASS
RDS	Storage encryption	TRUE	NOT VERIFIED
GuardDuty	Detector	Exists	PASS
GuardDuty	Detector status	ENABLED	PASS
GuardDuty	Finding API	Finding list/get	NOT VERIFIED / Floci
GuardDuty	Finding validation	Real finding	NOT VERIFIED
Phase 10	Security visibility foundation	Available	PARTIAL PASS

Jadi Phase 10 bukan gagal. Foundation-nya berhasil, tetapi dynamic compliance evaluation dan GuardDuty finding retrieval belum bisa diverifikasi karena keterbatasan emulator/API Floci.

Step 113. CloudTrail Security Audit Trail

Membuat centralized audit trail untuk aktivitas AWS API. Contohnya IAM CreateRole, IAM AttachRolePolicy, S3 CreateBucket, KMS Encrypt, RDS ModifyDBInstance, EKS API. Sehingga security engineer dapat menjawab: "Siapa melakukan apa, kapan, dan melalui API apa?"

113.1 Persiapan Phase 10

```
mkdir ~/phase10
cd ~/phase10
export ENDPOINT="http://localhost:4566"
export AWS_DEFAULT_REGION="us-east-1"
export AWS_ACCOUNT_ID="000000000000"
export CLOUDTRAIL_NAME="enterprise-security-trail"
export CONFIG_RECORDER_NAME="default"
export CLOUDTRAIL_BUCKET="enterprise-cloudtrail-logs"
export CONFIG_BUCKET="enterprise-config-logs"
echo "$ENDPOINT"
echo "$CLOUDTRAIL_BUCKET"
echo "$AWS_DEFAULT_REGION"
echo "$CLOUDTRAIL_NAME"
echo "$CONFIG_RECORDER_NAME"
echo "$CONFIG_BUCKET"
```

113.2 Buat CloudTrail S3 bucket. Parameter → --endpoint-url adalah arahkan AWS CLI ke Floci. s3api adalah menggunakan S3 API. create-bucket adalah membuat bucket. --bucket adalah nama bucket penyimpanan CloudTrail.

```
aws --endpoint-url="$ENDPOINT" \
    s3api create-bucket \
    --bucket "$CLOUDTRAIL_BUCKET"
```

113.3 Buat CloudTrail trail. Parameter penting → --name adalah nama trail. --s3-bucket-name adalah destination log. --is-multi-region-trail adalah capture event dari seluruh region. --include-global-service-events adalah event global seperti IAM. --enable-log-file-validation adalah mengaktifkan validasi integritas log.

```
aws --endpoint-url="$ENDPOINT" \
```

```

cloudtrail create-trail \
--name "$CLOUDTRAIL_NAME" \
--s3-bucket-name "$CLOUDTRAIL_BUCKET" \
--is-multi-region-trail \
--include-global-service-events \
--enable-log-file-validation

# 113.4 Validasi konfigurasi Trail. Yang harus diperhatikan →
IsMultiRegionTrail True, IncludeGlobalServiceEvents True,
LogFileValidationEnabled True, S3BucketName enterprise-cloudtrail-logs.
Hasil Floci → IsMultiRegionTrail True, IncludeGlobalServiceEvents True,
LogFileValidationEnabled True.

aws --endpoint-url="$ENDPOINT" \
cloudtrail describe-trails \
--trail-name-list "$CLOUDTRAIL_NAME" \
--output table

```

Step 114. Start CloudTrail Logging

Membuat trail benar-benar aktif merekam event, bukan hanya object konfigurasi.

114.1 Parameter penting → **start-logging** adalah mengaktifkan recording.
--name adalah trail yang akan diaktifkan.

```

aws --endpoint-url="$ENDPOINT" \
cloudtrail start-logging \
--name "$CLOUDTRAIL_NAME"

```

114.2 Validasi status

```

aws --endpoint-url="$ENDPOINT" \
cloudtrail get-trail-status \
--name "$CLOUDTRAIL_NAME" \
--output table

```

Step 115. Validasi CloudTrail Log Delivery

Ini validasi yang lebih kuat. bukan hanya CloudTrail mengatakan logging aktif, tetapi log benar-benar masuk ke S3.

115.1 Expected → **AWSLogs/000000000000/CloudTrail/....** Ini salah satu validasi terkuat Phase 10 karena kita sudah membuktikan AWS API Activity → CloudTrail → S3 → actual .json.gz log

```

aws --endpoint-url="$ENDPOINT" \

```

```

cloudtrail describe-trails \
--trail-name-list "$CLOUDTRAIL_NAME" \
--output table
aws --endpoint-url="$ENDPOINT" \
s3 ls \
"s3://$CLOUDTRAIL_BUCKET/AWSLogs/$AWS_ACCOUNT_ID/CloudTrail/" \
--recursive

```

Step 116. CloudTrail Event Lookup

Menguji apakah CloudTrail dapat digunakan untuk mencari event tertentu. Contoh kita cari ListRoles.

Parameter:

- lookup-events adalah mencari event yang sudah direkam.
- AttributeKey=EventName adalah filter berdasarkan nama API event.
- AttributeValue=ListRoles adalah event yang dicari.
- --max-results 10 adalah maksimal 10 hasil.

Hasil Actual Floci → LookupEvents tanpa event. Status → NOT VERIFIED — Floci limitation. Jangan mengubah CloudTrail hanya karena ini kosong. Karena kita sudah punya bukti yang jauh lebih kuat yaitu CloudTrail Logging = True + S3 log file exists. Jadi kita lewati troubleshooting lebih jauh untuk tahap ini.

```

aws --endpoint-url="$ENDPOINT" \
cloudtrail lookup-events \
--lookup-attributes \
AttributeKey=EventName,AttributeValue=ListRoles \
--max-results 10 \
--output table

```

Step 117. AWS Config Recorder

AWS Config bertugas merekam konfigurasi resource sehingga kita bisa melakukan compliance assessment. Contohnya S3 public? RDS public? RDS encrypted? CloudTrail multi-region?

117.1 Buat Configuration Recorder. Parameter → name adalah nama recorder. roleARN adalah IAM role yang digunakan Config. allSupported=true adalah record supported resources. includeGlobalResourceTypes=true adalah include global resources.

```

aws --endpoint-url="$ENDPOINT" \

```

```

configservice put-configuration-recorder \
--configuration-recorder '{
  "name": "'"$CONFIG_RECORDER_NAME"'",
  "roleARN": "arn:aws:iam::'"$AWS_ACCOUNT_ID"'':role/AWSConfigRole",
  "recordingGroup": {
    "allSupported": true,
    "includeGlobalResourceTypes": true
  }
}'

# 117.2 Validasi recorder. Hasil Actual floci → default
arn:aws:iam::000000000000:role/AWSConfigRole      allSupported      True
includeGlobalResourceTypes True

aws --endpoint-url="$ENDPOINT" \
  configservice describe-configuration-recorders \
  --output table

```

Step 118. Start AWS Config Recording

Recorder yang dibuat belum cukup. Kita harus mengaktifkannya.

118.1 Parameter → **start-configuration-recorder** adalah mulai recording.
--configuration-recorder-name adalah recorder yang digunakan.

```

aws --endpoint-url="$ENDPOINT" \
  configservice start-configuration-recorder \
  --configuration-recorder-name "$CONFIG_RECORDER_NAME"

```

118.2 Validasi status. **Expected** → **recording True lastStatus SUCCESS.**
Hasil Actual floci → **lastStatus SUCCESS recording True.** Ini penting:
AWS Config Recorder-nya memang aktif.

```

aws --endpoint-url="$ENDPOINT" \
  configservice describe-configuration-recorder-status \
  --configuration-recorder-name "$CONFIG_RECORDER_NAME" \
  --output table

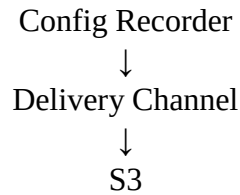
```

Step 119. AWS Config Delivery Channel

Delivery Channel digunakan untuk menentukan destination konfigurasi/history AWS Config. Secara arsitektur:

AWS Resource



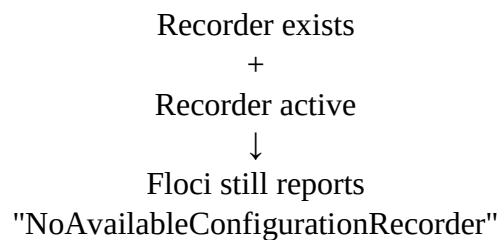


Hasil Actual Floci → NoAvailableConfigurationRecorderException

Padahal ConfigurationRecorder = exists, recording = True, lastStatus = SUCCESS.

Kesimpulan

Jangan ulang-ulang command ini. Ini merupakan behavior/inconsistency Floci:



Jadi status: NOT VERIFIED — Floci delivery-channel limitation.

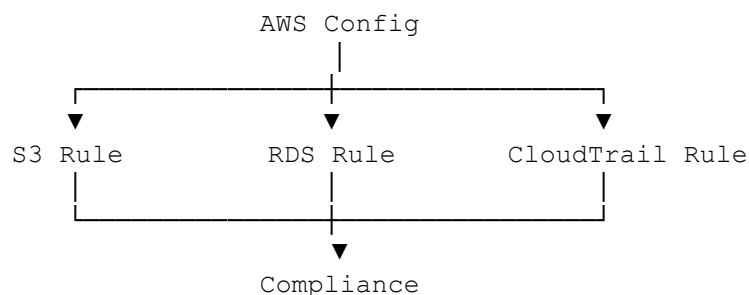
Kita tetap lanjut karena CloudTrail sudah berhasil membuktikan log delivery ke S3, sementara Config recorder sendiri sudah aktif.

```
aws --endpoint-url="$ENDPOINT" \
    configservice put-delivery-channel \
    --delivery-channel \
    "name=default,s3BucketName=$CONFIG_BUCKET"
```

Step 120. AWS Config Security Rules

Sekarang kita membuat security baseline. Kita gunakan empat rule yaitu multi-region-cloudtrail-enabled, s3-bucket-public-read-prohibited, rds-instance-public-access-check, rds-storage-encrypted.

Architecture:



Expected resource → Config Rule exists. Hasil Actual floci → Keempat rule memang ada yaitu rds-storage-encrypted, s3-bucket-public-read-prohibited, rds-instance-public-access-check, multi-region-cloudtrail-enabled.

Validasi:

```
aws --endpoint-url="$ENDPOINT" \
    configservice describe-config-rules \
    --output table
```

Step 121. Validate RDS Security Baseline

Kita jangan cuma melihat Config Rule. Kita juga cross-check resource aslinya.

Parameter query → DBInstanceIdentifier, Engine, PubliclyAccessible, StorageEncrypted.

Hasil Actual Floci → enterprise-postgres, postgres, False, None.

Interpretasi:

PubliclyAccessible = False → PASS

Database memang tidak public. Tetapi StorageEncrypted = None → NOT VERIFIED

Jangan kita tulis RDS encrypted. Karena output tidak membuktikannya.

Check and Result → RDS exists PASS, PostgreSQL PASS, Public access PASS, Storage encryption NOT VERIFIED.

```
aws --endpoint-url="$ENDPOINT" \
    rds describe-db-instances \
    --db-instance-identifier enterprise-postgres \
    --query
'DBInstances[0].[DBInstanceIdentifier,Engine,PubliclyAccessible,StorageEncrypted]' \
    --output table
```

Step 122. AWS Config Compliance Evaluation

Sekarang kita ingin mengetahui apakah rule benar-benar mengevaluasi resource. Expected ideal → COMPLIANT NON_COMPLIANT sedangkan Hasil Actual Floci → Semuanya INSUFFICIENT_DATA. Ini bukan berarti semua resource gagal compliance. Artinya Config rule ada, tetapi Floci belum memberikan evaluation result yang cukup untuk menentukan compliance.

```
aws --endpoint-url="$ENDPOINT" \
    configservice describe-compliance-by-config-rule \
```

```

--config-rule-names \
    multi-region-cloudtrail-enabled \
    s3-bucket-public-read-prohibited \
    rds-instance-public-access-check \
    rds-storage-encrypted \

--query
'ComplianceByConfigRules[*].[ConfigRuleName,Compliance.ComplianceType]
' \

--output table

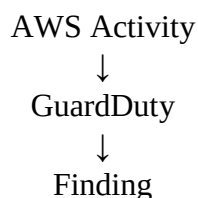
# 122.1 Attempt manual evaluation. Kita sebelumnya mencoba command di
bawah ini. dan Floci menjawab -. InvalidAction. Could not find operation
PutEvaluations. Kesimpulan → API PutEvaluations tidak didukung oleh
implementation Config Floci yang kita gunakan. Jadi kita hapus manual
compliance injection dari Phase 10. Kita tidak akan fake COMPLIANT hanya
supaya tabel kelihatan bagus. Status Step 122 → NOT VERIFIED – Floci
Config evaluation limitation.

configservice put-evaluations

```

Step 123. GuardDuty Detector

Sekarang masuk ke threat detection. GuardDuty bertugas mendeteksi aktivitas yang berpotensi malicious. Architecture:



123.1 Create Detector. Parameter → create-detector adalah membuat GuardDuty detector. --enable adalah langsung mengaktifkan detector. --query adalah mengambil ID detector. --output text adalah output hanya ID.

```

DETECTOR_ID=$(
    aws --endpoint-url="$ENDPOINT" \
        guardduty create-detector \
        --enable \
        --query 'DetectorId' \
        --output text
)

export DETECTOR_ID
echo "$DETECTOR_ID"

```

Step 124. Validate GuardDuty Detector

124.1 Hasil Actual floci → eb3ce729e8144699ab32d09f68c74d17

```
aws --endpoint-url="$ENDPOINT" \  
    guardduty list-detectors \  
    --output table
```

124.2 Kemudian ketik di bawah ini. Expected → Status ENABLED. Hasil Actual floci → ENABLED FIFTEEN_MINUTES. FIFTEEN_MINUTES merupakan frequency/status configuration yang diberikan Floci; yang penting untuk phase ini adalah detector ENABLED.

```
aws --endpoint-url="$ENDPOINT" \  
    guardduty get-detector \  
    --detector-id "$DETECTOR_ID" \  
    --output table
```

Step 125. GuardDuty Finding Validation

Menguji apakah GuardDuty finding dapat diambil.

125.1 Command:

```
aws --endpoint-url="$ENDPOINT" \  
    guardduty list-findings \  
    --detector-id "$DETECTOR_ID" \  
    --output json
```

125.2 Kemudian di bawah ini. Hasil Actual floci → Keduanya mendapatkan InvalidArgument POST requires either ?uploads, ?uploadId, ?restore or ?select parameter. Ini bukan error GuardDuty yang bisa kita interpretasikan sebagai "No findings" Karena error-nya justru menunjukkan request diarahkan/ditangani oleh endpoint API yang salah. Kesimpulan -. Kita tidak boleh mengatakan GuardDuty tidak punya finding. Yang benar → GuardDuty detector berhasil aktif, tetapi Finding API (ListFindings/GetFindings) tidak dapat divalidasi pada Floci karena request mengalami endpoint/API routing incompatibility. Status → NOT VERIFIED – Floci GuardDuty Finding API limitation. Dan jangan bikin fake finding hanya untuk mendapatkan output finding.

```
aws --endpoint-url="$ENDPOINT" \  
    guardduty get-findings \  
    --detector-id "$DETECTOR_ID" \  
    --finding-ids "test-finding-id" \  
    --output json
```

Step 126. PHASE 10 FINAL VALIDATION

```
echo "=====
echo "          Step 126: PHASE 10 FINAL VALIDATION          "
echo "=====
echo -e "\n[1] CloudTrail Trail:"
aws --endpoint-url="$ENDPOINT" \
    cloudtrail describe-trails \
    --trail-name-list "$CLOUDTRAIL_NAME" \
    --query
'trailList[*].[Name,S3BucketName,IsMultiRegionTrail,IncludeGlobalServiceEvents,LogFileValidationEnabled]' \
    --output table
echo -e "\n[2] CloudTrail Status:"
aws --endpoint-url="$ENDPOINT" \
    cloudtrail get-trail-status \
    --name "$CLOUDTRAIL_NAME" \
    --query '[IsLogging,LatestDeliveryTime]' \
    --output table
echo -e "\n[3] CloudTrail S3 Logs:"
aws --endpoint-url="$ENDPOINT" \
    s3 ls \
    "s3://$CLOUDTRAIL_BUCKET/AWSLogs/$AWS_ACCOUNT_ID/CloudTrail/" \
    --recursive
echo -e "\n[4] AWS Config Rules:"
aws --endpoint-url="$ENDPOINT" \
    configservice describe-config-rules \
    --query 'ConfigRules[*].[ConfigRuleName,ConfigRuleState]' \
    --output table
echo -e "\n[5] AWS Config Recorder:"
aws --endpoint-url="$ENDPOINT" \
    configservice describe-configuration-recorder-status \
    --configuration-recorder-name "$CONFIG_RECORDER_NAME" \
```

```

    --query
    'ConfigurationRecordersStatus[*].[name,recording,lastStatus,lastStartTime]' \

    --output table

echo -e "\n[6] AWS Config Compliance:"

aws --endpoint-url="$ENDPOINT" \

    configservice describe-compliance-by-config-rule \

    --config-rule-names \

        multi-region-cloudtrail-enabled \

        s3-bucket-public-read-prohibited \

        rds-instance-public-access-check \

        rds-storage-encrypted \

    --query
    'ComplianceByConfigRules[*].[ConfigRuleName,Compliance.ComplianceType]
    ' \

    --output table

echo -e "\n[7] RDS Security Baseline:"

aws --endpoint-url="$ENDPOINT" \

    rds describe-db-instances \

    --db-instance-identifier enterprise-postgres \

    --query
    'DBInstances[0].[DBInstanceIdentifier,PubliclyAccessible,StorageEncrypted]' \

    --output table

echo -e "\n[8] GuardDuty Detector:"

aws --endpoint-url="$ENDPOINT" \

    guardduty list-detectors \

    --output table

echo -e "\n[9] GuardDuty Status:"

aws --endpoint-url="$ENDPOINT" \

    guardduty get-detector \

    --detector-id "$DETECTOR_ID" \

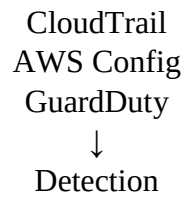
    --query '[Status,FindingPublishingFrequency]' \

    --output table

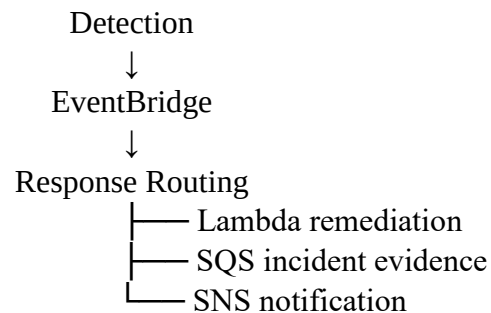
```

Phase 11 — Automated Security Response

Phase 10:



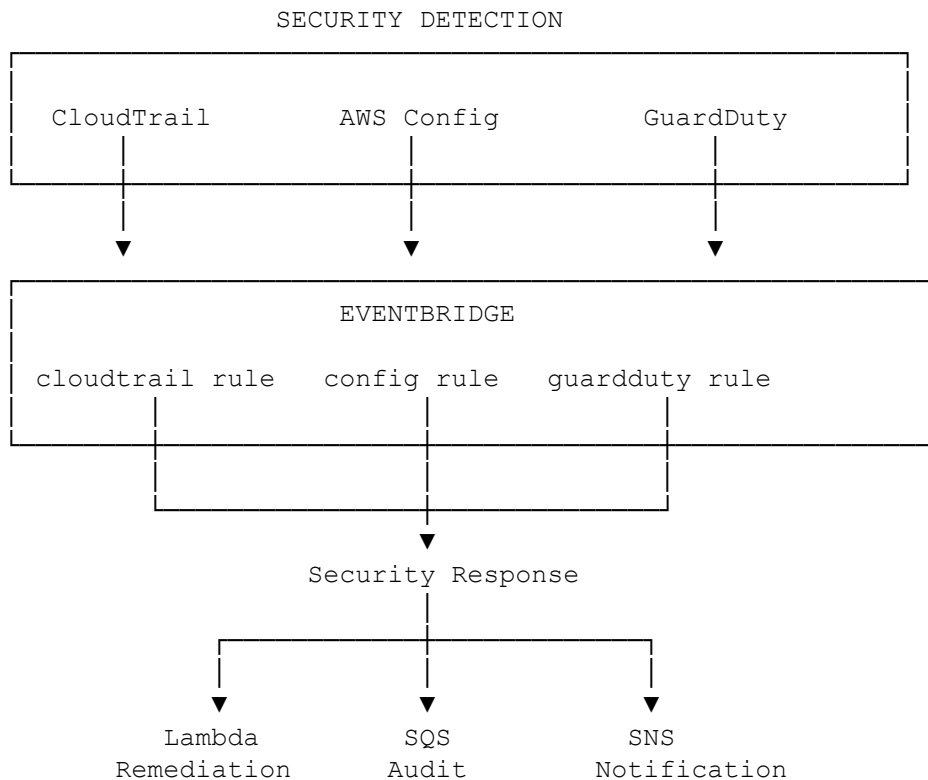
Phase 11:



Jadi pertanyaan yang dijawab Phase 11:

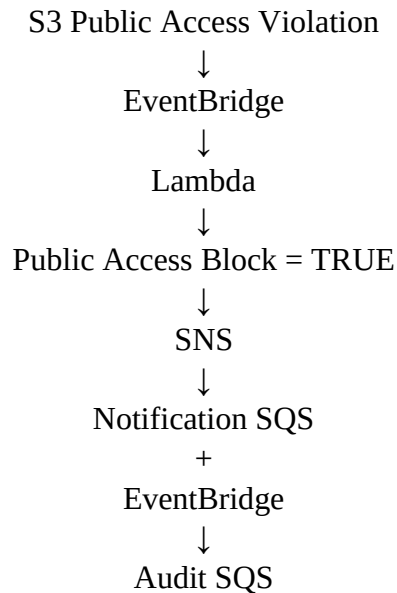
"Kalau security violation terdeteksi, apakah platform bisa merespons secara otomatis, mencatat incident, dan mengirim notification?"

Arsitektur Phase 11

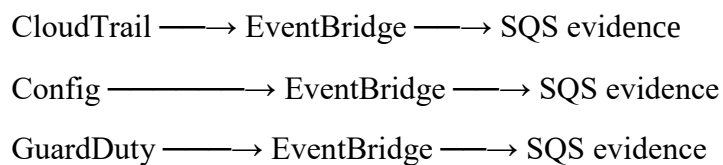




Untuk scenario yang benar-benar kita eksekusi:



Sedangkan tiga native detection integration:

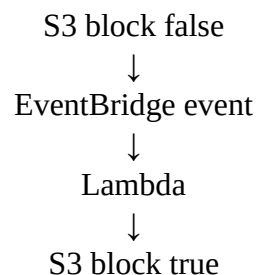


Validation philosophy

Kita akan membedakan tiga level:

1. PASS → Behavior benar-benar terbukti.

Contoh:



2. FLOCI LIMITATION → Capability/resource ada, pattern/rule bisa dibuat, tetapi native runtime event tidak bisa dibuktikan karena limitation yang sudah muncul dari Phase 10.

Contoh:

GuardDuty detector = ENABLED

GuardDuty finding API = InvalidArgument

Bukan NOT TESTED karena memang sudah dicoba.

3. FAIL -. Resource seharusnya bekerja dan kita sudah punya jalur yang seharusnya didukung, tetapi behavior memang gagal. Ini berbeda dari Floci limitation.

Step 127. Preparation & Environment

Menyiapkan workspace dan variable yang digunakan sepanjang Phase 11. Parameter → ENDPOINT → endpoint Floci.

AWS_REGION → region.

AWS_ACCOUNT_ID → account emulator.

RESPONSE_BUCKET → resource yang akan diremediasi.

AUDIT_QUEUE_NAME → queue untuk security evidence.

NOTIFY_QUEUE_NAME → queue notification.

SNS_TOPIC_NAME → notification topic.

LAMBDA_NAME → automated remediation function.

RULE_NAME → normalized security event rule.

tiga variable terakhir → native detection integration rules.

```
mkdir ~/phase11
cd ~/phase11
export ENDPOINT="http://localhost:4566"
export AWS_REGION="us-east-1"
export AWS_ACCOUNT_ID="000000000000"
export RESPONSE_BUCKET="enterprise-response-test"
export AUDIT_QUEUE_NAME="enterprise-security-events"
export NOTIFY_QUEUE_NAME="enterprise-security-notifications"
export SNS_TOPIC_NAME="enterprise-security-response"
export LAMBDA_NAME="enterprise-security-remediation"
export LAMBDA_ROLE_NAME="EnterpriseSecurityAutoRemediationRole"
export RULE_NAME="enterprise-security-auto-remediation"
export CONFIG_RULE_NAME="enterprise-config-security-response"
export GUARDDUTY_RULE_NAME="enterprise-guardduty-security-response"
```

```

export CLOUDTRAIL_RULE_NAME="enterprise-cloudtrail-security-response"
echo "$ENDPOINT"
echo "$AWS_REGION"
echo "$AWS_ACCOUNT_ID"
echo "$RESPONSE_BUCKET"
echo "$LAMBDA_NAME"
echo "$RULE_NAME"
echo "$CONFIG_RULE_NAME"
echo "$GUARDDUTY_RULE_NAME"
echo "$CLOUDTRAIL_RULE_NAME"

```

Step 128. Create Security Response Resources

Membuat resource yang digunakan oleh response plane yaitu S3, SQS audit, SQS notification, SNS

128.1 Create S3 remediation bucket. Parameter → --bucket adalah Nama bucket yang dibuat. Expected → { "Location": "/enterprise-response-test" }

```

aws --endpoint-url="$ENDPOINT" \
    s3api create-bucket \
    --bucket "$RESPONSE_BUCKET"

```

128.2 Establish secure baseline. Tujuan → Membuat initial state: BlockPublicAcls = true, IgnorePublicAcls = true, BlockPublicPolicy = true, RestrictPublicBuckets = true

```

aws --endpoint-url="$ENDPOINT" \
    s3api put-public-access-block \
    --bucket "$RESPONSE_BUCKET" \
    --public-access-block-configuration \

```

```

BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true

```

128.3 Validasi. Expected → keempatnya true.

```

aws --endpoint-url="$ENDPOINT" \
    s3api get-public-access-block \
    --bucket "$RESPONSE_BUCKET"

```

128.3 Create audit queue

```

export AUDIT_QUEUE_URL=$(
    aws --endpoint-url="$ENDPOINT" \

```

```

    sqs create-queue \
    --queue-name "$AUDIT_QUEUE_NAME" \
    --query 'QueueUrl' \
    --output text
)

# 128.3.1 Ambil ARN:
export AUDIT_QUEUE_ARN=$(
    aws --endpoint-url="$ENDPOINT" \
    sqs get-queue-attributes \
    --queue-url "$AUDIT_QUEUE_URL" \
    --attribute-names QueueArn \
    --query 'Attributes.QueueArn' \
    --output text
)

# 128.4 Create SNS topic
export SNS_TOPIC_ARN=$(
    aws --endpoint-url="$ENDPOINT" \
    sns create-topic \
    --name "$SNS_TOPIC_NAME" \
    --query 'TopicArn' \
    --output text
)

# 128.5 Create notification queue
export NOTIFY_QUEUE_URL=$(
    aws --endpoint-url="$ENDPOINT" \
    sqs create-queue \
    --queue-name "$NOTIFY_QUEUE_NAME" \
    --query 'QueueUrl' \
    --output text
)

# 128.5.1 Ambil ARN:
export NOTIFY_QUEUE_ARN=$(
    aws --endpoint-url="$ENDPOINT" \

```

```

    sqs get-queue-attributes \
    --queue-url "$NOTIFY_QUEUE_URL" \
    --attribute-names QueueArn \
    --query 'Attributes.QueueArn' \
    --output text
)

# 128.6 Validasi akhir Step 128. Expected queue/topic tersedia.
echo "AUDIT_QUEUE_URL=$AUDIT_QUEUE_URL"
echo "AUDIT_QUEUE_ARN=$AUDIT_QUEUE_ARN"
echo "NOTIFY_QUEUE_URL=$NOTIFY_QUEUE_URL"
echo "NOTIFY_QUEUE_ARN=$NOTIFY_QUEUE_ARN"
echo "SNS_TOPIC_ARN=$SNS_TOPIC_ARN"
aws --endpoint-url="$ENDPOINT" \
    sqs list-queues \
    --output table
aws --endpoint-url="$ENDPOINT" \
    sns list-topics \
    --output table

```

Step 129. IAM Execution Role

Lambda automated remediation harus memakai dedicated least-privilege role.

Permission:

S3:

GetPublicAccessBlock

PutPublicAccessBlock

SNS:

Publish

CloudWatch Logs:

CreateLogGroup

CreateLogStream

PutLogEvents

Bukan:

AdministratorAccess

129.1 Trust policy. Tujuan → Mengizinkan Lambda service assume role.

```
cat > lambda-trust-policy.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "lambda.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
EOF
```

129.2 Create role

```
aws --endpoint-url="$ENDPOINT" \
  iam create-role \
    --role-name "$LAMBDA_ROLE_NAME" \
    --assume-role-policy-document file://lambda-trust-policy.json
```

129.2.1 Ambil ARN:

```
export LAMBDA_ROLE_ARN=$(
  aws --endpoint-url="$ENDPOINT" \
    iam get-role \
      --role-name "$LAMBDA_ROLE_NAME" \
      --query 'Role.Arn' \
      --output text
)
```

129.3 Permission policy

```
cat > lambda-permission-policy.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "RemediateS3PublicAccess",
      "Effect": "Allow",
      "Action": [
        "s3:GetPublicAccessBlock",
        "s3:PutPublicAccessBlock"
      ],
      "Resource": "arn:aws:s3:::$RESPONSE_BUCKET"
    },
  ]
}
```

```

        "Sid": "PublishSecurityAlert",
        "Effect": "Allow",
        "Action": [
            "sns:Publish"
        ],
        "Resource": "$SNS_TOPIC_ARN"
    },
    {
        "Sid": "LambdaLogs",
        "Effect": "Allow",
        "Action": [
            "logs:CreateLogGroup"
        ],
        "Resource": "arn:aws:logs:$AWS_REGION:$AWS_ACCOUNT_ID:log-
group:/aws/lambda/$LAMBDA_NAME"
    },
    {
        "Sid": "LambdaLogStreams",
        "Effect": "Allow",
        "Action": [
            "logs:CreateLogStream",
            "logs:PutLogEvents"
        ],
        "Resource": "arn:aws:logs:$AWS_REGION:$AWS_ACCOUNT_ID:log-
group:/aws/lambda/$LAMBDA_NAME:*"
    }
]
}
EOF

```

129.3.1 Attach:

```

aws --endpoint-url="$ENDPOINT" \
    iam put-role-policy \
        --role-name "$LAMBDA_ROLE_NAME" \
        --policy-name SecurityRemediationPolicy \
        --policy-document file://lambda-permission-policy.json

```

129.3.2 Validasi. Expected → Role exists, Policy exists, Lambda trust = lambda.amazonaws.com

```

aws --endpoint-url="$ENDPOINT" \
    iam get-role \
        --role-name "$LAMBDA_ROLE_NAME" \
        --query 'Role.[RoleName,Arn]' \
        --output table

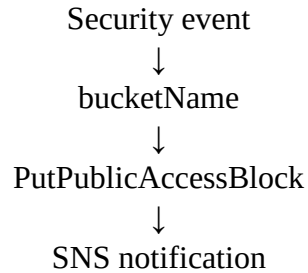
aws --endpoint-url="$ENDPOINT" \
    iam get-role-policy \
        --role-name "$LAMBDA_ROLE_NAME" \

```

```
--policy-name SecurityRemediationPolicy \  
--output json
```

Step 130. Build Lambda Automated Remediation

Lambda ini khusus remediation S3 Public Access.



130.1 Buat:

```
cat > lambda_function.py << 'EOF'  
import json  
import os  
import boto3  
endpoint_url = os.environ.get("AWS_ENDPOINT_URL")  
s3 = boto3.client("s3", endpoint_url=endpoint_url)  
sns = boto3.client("sns", endpoint_url=endpoint_url)  
SNS_TOPIC_ARN = os.environ["SNS_TOPIC_ARN"]  
def handler(event, context):  
    detail = event.get("detail", {})  
    bucket = detail.get("bucketName")  
    if not bucket:  
        request_parameters = detail.get("requestParameters", {})  
        bucket = request_parameters.get("bucketName")  
    if not bucket:  
        raise ValueError("bucketName missing from security event")  
    s3.put_public_access_block(  
        Bucket=bucket,  
        PublicAccessBlockConfiguration={  
            "BlockPublicAcls": True,  
            "IgnorePublicAcls": True,  
            "BlockPublicPolicy": True,  
            "RestrictPublicBuckets": True  
        }  
    )  
    result = {  
        "action": "S3_PUBLIC_ACCESS_REMEDIATION",  
        "bucket": bucket,  
        "status": "REMEDIATED",  
        "control": detail.get("control"),  
        "reason": detail.get("reason")  
    }  
    sns.publish(  
        TopicArn=SNS_TOPIC_ARN,  
        Subject="Automated Security Remediation",  
        Message=json.dumps(result)  
    )
```

```

    print(json.dumps(result))
    return result
EOF

# 130.2 Package:

zip security-remediation.zip lambda_function.py

# 130.3 Create Lambda. Parameter penting → --runtime adalah runtime
Python. --role adalah IAM execution role. --handler adalah
lambda_function.handler. --timeout 30 adalah maksimum execution 30
detik. --memory-size 256 adalah 256 MB. --environment adalah endpoint
Floci + SNS ARN. --zip-file adalah source package Lambda.

aws --endpoint-url="$ENDPOINT" \

    lambda create-function \

    --function-name "$LAMBDA_NAME" \

    --runtime python3.12 \

    --role "$LAMBDA_ROLE_ARN" \

    --handler lambda_function.handler \

    --timeout 30 \

    --memory-size 256 \

    --environment
"Variables={AWS_ENDPOINT_URL=http://localhost.floci.io:4566,AWS_DEFAULT_REGION=$AWS_REGION,SNS_TOPIC_ARN=$SNS_TOPIC_ARN}" \

    --zip-file fileb://security-remediation.zip

# 130.4 Validasi. Expected → enterprise-security-remediation,
python3.12 ... lambda_function.handler Active

aws --endpoint-url="$ENDPOINT" \

    lambda get-function \

    --function-name "$LAMBDA_NAME" \

    --query 'Configuration.[FunctionName,Runtime,Role,Handler,State]' \

    --output table

```

Step 131. Direct Lambda Validation

Sebelum EventBridge dilibatkan, kita isolasi Lambda.

Lambda

- ├── IAM
- ├── S3
- └── SNS

131.1 Buat test event:

```

cat > security-event.json <<EOF
{
  "version": "0",
  "id": "phase11-direct-test",
  "source": "enterprise.security",
  "detail-type": "Security Control Violation",
  "account": "$AWS_ACCOUNT_ID",
  "time": "$(date -u +%Y-%m-%dT%H:%M:%SZ)",
  "region": "$AWS_REGION",
  "resources": [
    "arn:aws:s3:::$RESPONSE_BUCKET"
  ],
  "detail": {
    "control": "S3_PUBLIC_ACCESS",
    "bucketName": "$RESPONSE_BUCKET",
    "reason": "PublicAccessBlockRemoved"
  }
}
EOF

```

131.2 Invoke:

```

aws --endpoint-url="$ENDPOINT" \
  lambda invoke \
    --function-name "$LAMBDA_NAME" \
    --payload fileb://security-event.json \
    --cli-binary-format raw-in-base64-out \
    lambda-direct-response.json

```

```

# 131.3 Cek → Expected → { "action": "S3_PUBLIC_ACCESS_REMEDIATION",
"bucket": "enterprise-response-test", "status": "REMEDIATED",
"control": "S3_PUBLIC_ACCESS", "reason": "PublicAccessBlockRemoved" }

```

```
cat lambda-direct-response.json
```

```

# 131.4 Validasi S3. Expected → BlockPublicAcls true, IgnorePublicAcls
true, BlockPublicPolicy true, RestrictPublicBuckets true.

```

```

aws --endpoint-url="$ENDPOINT" \
  s3api get-public-access-block \
    --bucket "$RESPONSE_BUCKET"

```

Step 132. EventBridge Security Rule

Menghubungkan normalized security event ke response pipeline.

132.1 Buat pattern:

```

cat > event-pattern.json <<EOF
{
  "source": [
    "enterprise.security"
  ]
}

```

```

    ],
    "detail-type": [
        "Security Control Violation"
    ],
    "detail": {
        "control": [
            "S3_PUBLIC_ACCESS"
        ],
        "bucketName": [
            "$RESPONSE_BUCKET"
        ]
    }
}
EOF

```

132.2 Create:

```

aws --endpoint-url="$ENDPOINT" \
    events put-rule \
    --name "$RULE_NAME" \
    --event-bus-name default \
    --event-pattern file://event-pattern.json \
    --state ENABLED \
    --description "Automated remediation for S3 public access security
violations"

```

132.2.1 Ambil ARN:

```

export RULE_ARN=$(
    aws --endpoint-url="$ENDPOINT" \
        events describe-rule \
        --name "$RULE_NAME" \
        --event-bus-name default \
        --query 'Arn' \
        --output text
)

```

132.3 Validasi. Expected → enterprise-security-auto-remediation ENABLED ...

```

aws --endpoint-url="$ENDPOINT" \
    events describe-rule \
    --name "$RULE_NAME" \
    --event-bus-name default \
    --query '[Name,State,Arn,EventPattern]' \
    --output table

```

Step 133. Resource Permissions

Membuat trust path:

- EventBridge → Lambda
- EventBridge → SQS
- SNS → SQS

133.1 EventBridge → Lambda

```
aws --endpoint-url="$ENDPOINT" \  
    lambda add-permission \  
        --function-name "$LAMBDA_NAME" \  
        --statement-id AllowEventBridgeSecurityRule \  
        --action lambda:InvokeFunction \  
        --principal events.amazonaws.com \  
        --source-arn "$RULE_ARN"
```

133.2 Validasi. Expected → Principal = events.amazonaws.com, Action = lambda:InvokeFunction, SourceArn = enterprise-security-auto-remediation.

```
aws --endpoint-url="$ENDPOINT" \  
    lambda get-policy \  
        --function-name "$LAMBDA_NAME" \  
        --output json
```

133.3 EventBridge → SQS

```
cat > audit-queue-policy.json <<EOF  
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "AllowEventBridgeSecurityRule",  
      "Effect": "Allow",  
      "Principal": {  
        "Service": "events.amazonaws.com"  
      },  
      "Action": "sqs:SendMessage",  
      "Resource": "$AUDIT_QUEUE_ARN",  
      "Condition": {  
        "ArnEquals": {  
          "aws:SourceArn": "$RULE_ARN"  
        }  
      }  
    }  
  ]  
}
```

EOF

133.4 Apply:

```
aws --endpoint-url="$ENDPOINT" \  
    sqs set-queue-attributes \  
    --queue-url "$AUDIT_QUEUE_URL" \  
    --attributes Policy=file:///audit-queue-policy.json
```

133.5 Validasi. Expected events.amazonaws.com + sqs:SendMessage.

```
aws --endpoint-url="$ENDPOINT" \  
    sqs get-queue-attributes \  
    --queue-url "$AUDIT_QUEUE_URL" \  
    --attribute-names Policy \  
    --output json
```

133.6 SNS → Notification SQS

```
cat > notification-queue-policy.json <<EOF  
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "AllowSecurityResponseTopic",  
      "Effect": "Allow",  
      "Principal": {  
        "Service": "sns.amazonaws.com"  
      },  
      "Action": "sqs:SendMessage",  
      "Resource": "$NOTIFY_QUEUE_ARN",  
      "Condition": {  
        "ArnEquals": {  
          "aws:SourceArn": "$SNS_TOPIC_ARN"  
        }  
      }  
    }  
  ]  
}  
EOF
```

133.7 Apply:

```
aws --endpoint-url="$ENDPOINT" \  
    sqs set-queue-attributes \  
    --queue-url "$NOTIFY_QUEUE_URL" \  
    --attributes Policy=file:///notification-queue-policy.json
```

133.8 Subscribe:

```
aws --endpoint-url="$ENDPOINT" \  
    sns subscribe \  
    --notification-endpoint "$NOTIFY_QUEUE_URL"
```

```

--topic-arn "$SNS_TOPIC_ARN" \
--protocol sqs \
--notification-endpoint "$NOTIFY_QUEUE_ARN"

# 133.9 Validasi Step 133. Expected → sqs, arn:aws:sqs:...enterprise-
security-notifications.

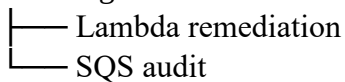
aws --endpoint-url="$ENDPOINT" \
    sns list-subscriptions-by-topic \
    --topic-arn "$SNS_TOPIC_ARN" \
    --output table

```

Step 134. Attach EventBridge Targets

Normalized security rule punya dua target:

EventBridge



134.1 Ambil Lambda ARN:

```

export LAMBDA_ARN=$(
    aws --endpoint-url="$ENDPOINT" \
        lambda get-function \
        --function-name "$LAMBDA_NAME" \
        --query 'Configuration.FunctionArn' \
        --output text
)

```

134.2 Buat:

```

cat > targets.json <<EOF
[
  {
    "Id": "RemediationLambda",
    "Arn": "$LAMBDA_ARN"
  },
  {
    "Id": "SecurityAuditQueue",
    "Arn": "$AUDIT_QUEUE_ARN"
  }
]
EOF

```

134.3 Apply. Expected → { "FailedEntryCount": 0, "FailedEntries": [] }

```

aws --endpoint-url="$ENDPOINT" \

```

```

events put-targets \
--rule "$RULE_NAME" \
--event-bus-name default \
--targets file://targets.json

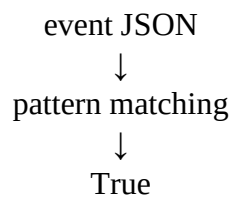
# 134.4 Validasi. Expected → RemediationLambda, SecurityAuditQueue
aws --endpoint-url="$ENDPOINT" \
events list-targets-by-rule \
--rule "$RULE_NAME" \
--event-bus-name default \
--output table

```

Step 135. Validate Event Pattern

Expected → True

Arti



berarti EventBridge rule memang secara logika akan match event tersebut.

```

aws --endpoint-url="$ENDPOINT" \
events test-event-pattern \
--event-pattern file://event-pattern.json \
--event file://security-event.json \
--output text

```

Step 136. Simulate Security Incident

Sengaja menghilangkan Public Access Block.

```

aws --endpoint-url="$ENDPOINT" \
s3api delete-public-access-block \
--bucket "$RESPONSE_BUCKET"

```

136.1 Validasi. Expected configuration tidak ditemukan. Ini bukan Phase 11 failure. Ini adalah Security Control = intentionally violated karena kita sedang membuat kondisi untuk menguji detection/response.

```

aws --endpoint-url="$ENDPOINT" \

```

```
s3api get-public-access-block \  
--bucket "$RESPONSE_BUCKET"
```

Step 137. Trigger Automated Security Response

137.1 Kirim event. Expected → FailedEntryCount = 0. Tapi ini belum membuktikan target berhasil.

```
EVENT_ENTRIES=$(  
  jq -c '  
    [{  
      Source: .source,  
      DetailType: ".detail-type",  
      Detail: (.detail | tojson),  
      EventBusName: "default",  
      Resources: .resources  
    }]  
  ' security-event.json  
)  
aws --endpoint-url="$ENDPOINT" \  
  events put-events \  
  --entries "$EVENT_ENTRIES" \  
  --output json
```

Step 138. Validate Automated Remediation

138.1 Validasi utama. Expected → { "PublicAccessBlockConfiguration": { "BlockPublicAcls": true, "IgnorePublicAcls": true, "BlockPublicPolicy": true, "RestrictPublicBuckets": true } }. Kalau output Actual Floci → BlockPublicAcls true, IgnorePublicAcls true, BlockPublicPolicy true, RestrictPublicBuckets true. AUTOMATED REMEDIATION = PASS

```
aws --endpoint-url="$ENDPOINT" \  
  s3api get-public-access-block \  
  --bucket "$RESPONSE_BUCKET" \  
  --output json
```

138.2 Lambda log validation

```
aws --endpoint-url="$ENDPOINT" \  
  logs describe-log-groups \  
  --log-group-name "/aws/lambda/lambda-function-name"
```

```

--log-group-name-prefix "/aws/lambda/$LAMBDA_NAME" \
--output table
# 138.3 Kalau log group tersedia:
export LOG_STREAM_NAME=$(
  aws --endpoint-url="$ENDPOINT" \
    logs describe-log-streams \
      --log-group-name "/aws/lambda/$LAMBDA_NAME" \
      --order-by LastEventTime \
      --descending \
      --query 'logStreams[0].logStreamName' \
      --output text
)
# 138.3.1 Kemudian di bawah ini. Expected ada →
S3_PUBLIC_ACCESS_REMEDIATION REMEDIATED enterprise-response-test
aws --endpoint-url="$ENDPOINT" \
  logs get-log-events \
    --log-group-name "/aws/lambda/$LAMBDA_NAME" \
    --log-stream-name "$LOG_STREAM_NAME" \
    --output json

```

Step 139 . Validate EventBridge → Audit SQS

Output Actual floci sudah membuktikan ini:

source:

enterprise.security

detail-type:

Security Control Violation

control:

S3_PUBLIC_ACCESS

bucketName:

enterprise-response-test

```

aws --endpoint-url="$ENDPOINT" \
  sqs receive-message \

```

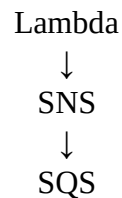
```
--queue-url "$AUDIT_QUEUE_URL" \  
--max-number-of-messages 1 \  
--wait-time-seconds 5 \  
--output json
```

Step 140. Validate Lambda → SNS → SQS

Output lu sekarang sudah membuktikan

- Subject: Automated Security Remediation
- action: S3_PUBLIC_ACCESS_REMEDIATION
- status: REMEDIATED

Jadi:



140.1 Cek subscription:

```
aws --endpoint-url="$ENDPOINT" \  
    sns list-subscriptions-by-topic \  
    --topic-arn "$SNS_TOPIC_ARN" \  
    --output table
```

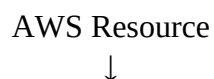
140.1.1 Kemudian:

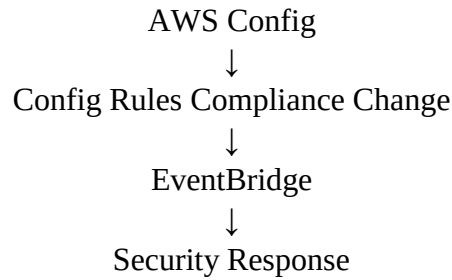
```
aws --endpoint-url="$ENDPOINT" \  
    sqs receive-message \  
    --queue-url "$NOTIFY_QUEUE_URL" \  
    --max-number-of-messages 1 \  
    --wait-time-seconds 5 \  
    --output json
```

Step 141. AWS Config → EventBridge

Nah, mulai sini kita masuk native detection integration. Menguji apakah perubahan compliance dari AWS Config dapat menghasilkan EventBridge event.

Architecture:





AWS Config memang menyediakan service event Config Rules Compliance Change dengan:

source = aws.config

detail-type = Config Rules Compliance Change

dan event tersebut dikirim langsung ke EventBridge.

141.1 Buat event pattern. Tujuan → Rule akan match native Config compliance change, bukan normalized enterprise.security.

```
cat > config-event-pattern.json <<'EOF'
{
  "source": [
    "aws.config"
  ],
  "detail-type": [
    "Config Rules Compliance Change"
  ]
}
EOF
```

141.2 Create EventBridge rule. Parameter -. --name adalah nama rule. --event-bus-name default adalah default event bus. --event-pattern adalah pattern native Config. --state ENABLED adalah rule aktif.

```
aws --endpoint-url="$ENDPOINT" \
  events put-rule \
    --name "$CONFIG_RULE_NAME" \
    --event-bus-name default \
    --event-pattern file://config-event-pattern.json \
    --state ENABLED
```

141.3 Validasi rule. Expected → Name: enterprise-config-security-response; State: ENABLED; EventPattern: source = aws.config, detail-type = Config Rules Compliance Change

```
aws --endpoint-url="$ENDPOINT" \
  events describe-rule \
    --name "$CONFIG_RULE_NAME" \
    --event-bus-name default \
```

```

--output json

# 141.4 Validasi Config compliance. Expected native AWS → COMPLIANT
atau NON_COMPLIANT. Tapi Phase 10 lu menghasilkan INSUFFICIENT_DATA.
Jadi ini penting adalah Config rules exist PASS, Config recorder
active PASS, EventBridge Config rule PASS, Actual compliance state
FLOCI LIMITATION, Config → native EventBridge FLOCI LIMITATION. Karena
PutEvaluations juga menghasilkan InvalidAction Could not find
operation PutEvaluations kita sudah mencoba jalur evaluasinya, bukan
belum mengetes. Jangan lakukan Jangan inject fake PutEvaluations hanya
supaya muncul COMPLIANT. Itu akan membuat hasil lab misleading.

aws --endpoint-url="$ENDPOINT" \

  configservice describe-compliance-by-config-rule \

  --config-rule-names \

    multi-region-cloudtrail-enabled \

    s3-bucket-public-read-prohibited \

    rds-instance-public-access-check \

    rds-storage-encrypted \

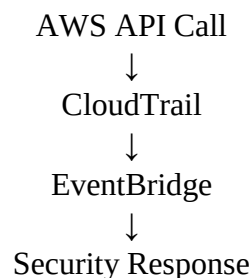
  --output table

```

Step 142. CloudTrail → EventBridge

Menguji integration antara CloudTrail dan EventBridge.

Native architecture:



CloudTrail management events dapat digunakan sebagai input EventBridge pada default event bus. AWS mendokumentasikan event type AWS API Call via CloudTrail.

142.1 Buat security-focused event pattern Kita jangan match seluruh CloudTrail event. Kita fokus ke event yang berbahaya terhadap logging StopLogging, DeleteTrail. ini lebih bagus Karena kita bukan sekadar CloudTrail → EventBridge tetapi CloudTrail → security-sensitive API → EventBridge

```

cat > cloudtrail-event-pattern.json <<'EOF'
{
  "source": [
    "aws.cloudtrail"

```

```

    ],
    "detail-type": [
        "AWS API Call via CloudTrail"
    ],
    "detail": {
        "eventSource": [
            "cloudtrail.amazonaws.com"
        ],
        "eventName": [
            "StopLogging",
            "DeleteTrail"
        ]
    }
}
EOF

```

142.2 Create rule

```

aws --endpoint-url="$ENDPOINT" \
    events put-rule \
    --name "$CLOUDTRAIL_RULE_NAME" \
    --event-bus-name default \
    --event-pattern file://cloudtrail-event-pattern.json \
    --state ENABLED

```

142.3 Validasi. ini lebih bagus Karena kita bukan sekadar CloudTrail → EventBridge tetapi CloudTrail → security-sensitive API → EventBridge

```

aws --endpoint-url="$ENDPOINT" \
    events describe-rule \
    --name "$CLOUDTRAIL_RULE_NAME" \
    --event-bus-name default \
    --query '[Name,State,EventPattern]' \
    --output table

```

142.4 Test event pattern

```

cat > cloudtrail-test-event.json <<'EOF'
{
    "version": "0",
    "id": "phase11-cloudtrail-test",
    "source": "aws.cloudtrail",
    "detail-type": "AWS API Call via CloudTrail",
    "account": "000000000000",
    "region": "us-east-1",
    "detail": {
        "eventSource": "cloudtrail.amazonaws.com",
        "eventName": "StopLogging"
    }
}
EOF

```

142.4.1 Test. Expected → True

```
aws --endpoint-url="$ENDPOINT" \  
  events test-event-pattern \  
  --event-pattern file://cloudtrail-event-pattern.json \  
  --event file://cloudtrail-test-event.json \  
  --output text
```

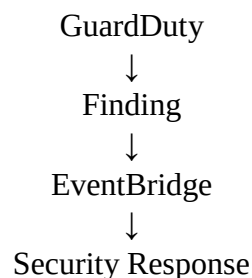
142.5 Coba native CloudTrail event validation. Phase 10 lu sebelumnya LookupEvents tanpa event. Jadi CloudTrail trail PASS, CloudTrail logging PASS, CloudTrail → S3 PASS, CloudTrail event lookup FLOCI LIMITATION, Native CloudTrail → EventBridge FLOCI LIMITATION. Jangan stop CloudTrail asli. Kita tidak perlu menjalankan StopLogging sungguhan hanya demi test karena itu justru merusak audit trail. Pattern sudah diuji dengan test-event-pattern; native event delivery tetap dicatat sebagai Floci limitation karena upstream CloudTrail event availability di emulator belum terbukti.

```
aws --endpoint-url="$ENDPOINT" \  
  cloudtrail lookup-events \  
  --max-results 5 \  
  --output table
```

Step 143. GuardDuty → EventBridge

GuardDuty adalah threat detection engine dari Phase 10.

Native architecture:



AWS mendokumentasikan bahwa GuardDuty mengirim GuardDuty Finding langsung ke EventBridge dengan:

- source: aws.guardduty
- detail-type: GuardDuty Finding

143.1 Buat pattern

```
cat > guardduty-event-pattern.json <<'EOF'  
{  
  "source": [  

```

```

        "aws.guarddduty"
    ],
    "detail-type": [
        "GuardDuty Finding"
    ]
}
EOF

```

143.2 Create rule

```

aws --endpoint-url="$ENDPOINT" \
    events put-rule \
        --name "$GUARDDUTY_RULE_NAME" \
        --event-bus-name default \
        --event-pattern file://guarddduty-event-pattern.json \
        --state ENABLED

```

143.3 Validasi rule. Expected → enterprise-guarddduty-security-response ENABLED ...

```

aws --endpoint-url="$ENDPOINT" \
    events describe-rule \
        --name "$GUARDDUTY_RULE_NAME" \
        --event-bus-name default \
        --query '[Name,State,EventPattern]' \
        --output table

```

143.4 Validate GuardDuty detector. Expected → DetectorId eb3ce729e8144699ab32d09f68c74d17

```

aws --endpoint-url="$ENDPOINT" \
    guarddduty list-detectors \
        --output table

```

143.4.1 Kemudian di bawah ini. Expected dari hasil Phase 10 → ENABLED FIFTEEN_MINUTES. Jadi → Detector exists = PASS, Detector enabled = PASS.

```

aws --endpoint-url="$ENDPOINT" \
    guarddduty get-detector \
        --detector-id "$DETECTOR_ID" \
        --query '[Status,FindingPublishingFrequency]' \
        --output table

```

143.5 Coba Finding API. Phase 10 lu sudah mendapatkan InvalidArgument POST requires either ?uploads, ?uploadId, ?restore or ?select parameter. Jadi kita catat GuardDuty detector PASS, GuardDuty enabled PASS, EventBridge rule PASS, Finding retrieval FLOCI

LIMITATION, Native finding delivery FLOCI LIMITATION. Bukan NOT TESTED. Kita sudah memanggil API-nya dan emulator yang gagal meroute operation tersebut.

```
aws --endpoint-url="$ENDPOINT" \  
    guardduty list-findings \  
    --detector-id "$DETECTOR_ID" \  
    --output json
```

Step 144. Validate All Detection → Response Rules

Sekarang kita pastikan empat security detection pathways sudah punya EventBridge rule:

1. Normalized security event
2. CloudTrail
3. AWS Config
4. GuardDuty

144.1 Normalized security response. Expected → enterprise-security-auto-remediation ENABLED

```
aws --endpoint-url="$ENDPOINT" \  
    events describe-rule \  
    --name "$RULE_NAME" \  
    --event-bus-name default \  
    --query '[Name,State]' \  
    --output table
```

144.2 CloudTrail. Expected → enterprise-cloudtrail-security-response ENABLED

```
aws --endpoint-url="$ENDPOINT" \  
    events describe-rule \  
    --name "$CLOUDTRAIL_RULE_NAME" \  
    --event-bus-name default \  
    --query '[Name,State]' \  
    --output table
```

144.3 Config. Expected → enterprise-config-security-response ENABLED

```
aws --endpoint-url="$ENDPOINT" \  
    events describe-rule \  
    --name "$CONFIG_RULE_NAME" \  
    --event-bus-name default \  
    --output table
```

```
--query '[Name,State]' \  
--output table  
  
# 144.4 GuardDuty. Expected → enterprise-guardduty-security-response  
ENABLED
```

```
aws --endpoint-url="$ENDPOINT" \  
  events describe-rule \  
    --name "$GUARDDUTY_RULE_NAME" \  
    --event-bus-name default \  
    --query '[Name,State]' \  
    --output table
```

144.5 Validate pattern matching

144.5.1 CloudTrail. Expected → True

```
aws --endpoint-url="$ENDPOINT" \  
  events test-event-pattern \  
    --event-pattern file://cloudtrail-event-pattern.json \  
    --event file://cloudtrail-test-event.json \  
    --output text
```

144.5.2 Config dan GuardDuty juga bisa diuji dengan synthetic native-shaped event. Config test event

```
cat > config-test-event.json <<'EOF'  
{  
  "version": "0",  
  "id": "phase11-config-test",  
  "source": "aws.config",  
  "detail-type": "Config Rules Compliance Change",  
  "account": "000000000000",  
  "region": "us-east-1",  
  "detail": {  
    "configRuleName": "rds-storage-encrypted",  
    "newEvaluationResult": {  
      "complianceType": "NON_COMPLIANT"  
    }  
  }  
}  
EOF
```

144.5.3 Test. Expected → True

```
aws --endpoint-url="$ENDPOINT" \  
  events test-event-pattern \  
    --event-pattern file://config-event-pattern.json \  
    --event file://config-test-event.json \  
    --output text
```

```

--output text

# 144.5.4 GuardDuty test event

cat > guardduty-test-event.json <<'EOF'
{
  "version": "0",
  "id": "phasell-guardduty-test",
  "source": "aws.guardduty",
  "detail-type": "GuardDuty Finding",
  "account": "000000000000",
  "region": "us-east-1",
  "detail": {
    "type": "Test/Finding",
    "severity": 5
  }
}
EOF

```

144.5.5 Test. Expected → True. Kenapa synthetic test ini penting?
 Karena kita memisahkan dua hal yaitu EventBridge pattern correctness ≠ Native AWS service event delivery. Kalau test-event-pattern = True, pattern benar. Kalau native event tidak muncul karena Floci: Pattern = PASS Native delivery = FLOCi LIMITATION. Ini jauh lebih akurat daripada menyebut seluruh integration gagal.

```

aws --endpoint-url="$ENDPOINT" \
  events test-event-pattern \
  --event-pattern file://guardduty-event-pattern.json \
  --event file://guardduty-test-event.json \
  --output text

```

Step 145 . DETECTION → RESPONSE INTEGRATION

```

echo "=====
echo " Step 145: DETECTION → RESPONSE INTEGRATION      "
echo "=====
echo -e "\n[1] CloudTrail → EventBridge:"
aws --endpoint-url="$ENDPOINT" \
  events describe-rule \
  --name "$CLOUDTRAIL_RULE_NAME" \
  --event-bus-name default \
  --query '[Name,State,EventPattern]' \
  --output table
echo -e "\n[2] AWS Config → EventBridge:"

```

```

aws --endpoint-url="$ENDPOINT" \
    events describe-rule \
    --name "$CONFIG_RULE_NAME" \
    --event-bus-name default \
    --query '[Name,State,EventPattern]' \
    --output table
echo -e "\n[3] GuardDuty → EventBridge:"
aws --endpoint-url="$ENDPOINT" \
    events describe-rule \
    --name "$GUARDDUTY_RULE_NAME" \
    --event-bus-name default \
    --query '[Name,State,EventPattern]' \
    --output table
echo -e "\n[4] GuardDuty Detector:"
aws --endpoint-url="$ENDPOINT" \
    guardduty get-detector \
    --detector-id "$DETECTOR_ID" \
    --query '[Status,FindingPublishingFrequency]' \
    --output table
echo -e "\n[5] CloudTrail Events:"
aws --endpoint-url="$ENDPOINT" \
    cloudtrail lookup-events \
    --max-results 5 \
    --output table
echo -e "\n[6] Config Compliance:"
aws --endpoint-url="$ENDPOINT" \
    configservice describe-compliance-by-config-rule \
    --config-rule-names \
        multi-region-cloudtrail-enabled \
        s3-bucket-public-read-prohibited \
        rds-instance-public-access-check \
        rds-storage-encrypted \
    --query
'ComplianceByConfigRules[*].[ConfigRuleName,Compliance.ComplianceType]
' \

```

```
--output table
echo -e "\n[7] Normalized Security Response:"
aws --endpoint-url="$ENDPOINT" \
    events describe-rule \
    --name "$RULE_NAME" \
    --event-bus-name default \
    --query '[Name,State]' \
    --output table
echo -e "\n[8] Remediation Target:"
aws --endpoint-url="$ENDPOINT" \
    s3api get-public-access-block \
    --bucket "$RESPONSE_BUCKET" \
    --query 'PublicAccessBlockConfiguration' \
    --output json
echo -e "\n[9] Audit Evidence:"
aws --endpoint-url="$ENDPOINT" \
    sqs receive-message \
    --queue-url "$AUDIT_QUEUE_URL" \
    --max-number-of-messages 1 \
    --wait-time-seconds 2 \
    --output json
echo -e "\n[10] Notification Evidence:"
aws --endpoint-url="$ENDPOINT" \
    sqs receive-message \
    --queue-url "$NOTIFY_QUEUE_URL" \
    --max-number-of-messages 1 \
    --wait-time-seconds 2 \
    --output json
```